HR MANAGEMENT SYSTEM

COMPLETE PROJECT DOCUMENTATION

ABSTRACT

The Human Resource Management System (HRMS) is a comprehensive web-based application designed to streamline and automate various HR processes within organizations. This project addresses the critical need for efficient human resource management in modern enterprises by providing a centralized platform for recruitment, employee management, attendance tracking, leave management, and performance appraisal.

The system is developed using PHP as the server-side scripting language, MySQL for database management, HTML5, CSS3, and JavaScript for the frontend, creating a robust and scalable solution. The application follows a modular architecture with separate modules for different HR functions, ensuring maintainability and extensibility.

Key features include a secure authentication system with role-based access control, real-time dashboard analytics, comprehensive employee lifecycle management from recruitment to performance evaluation, automated attendance tracking, and a sophisticated leave management system with approval workflows. The system also provides detailed reporting capabilities and data visualization using Chart.js for better decision-making.

The project demonstrates practical implementation of database design principles, web security measures, responsive UI/UX design, and modern software development practices. It serves as an effective solution for small to medium-sized organizations seeking to digitize their HR operations while maintaining data integrity and security.

This documentation provides a comprehensive analysis of the system architecture, implementation details, database design, testing methodologies, and deployment procedures, serving as a complete reference for understanding and maintaining the HR Management System.

GLOSSARY AND ABBREVIATIONS

Technical Terms

- **API** - Application Programming Interface
- **CRUD** - Create, Read, Update, Delete operations
- **CSS** - Cascading Style Sheets
- **DBMS** - Database Management System
- **DOM** - Document Object Model

- **HTML** - HyperText Markup Language
- **HTTP** - HyperText Transfer Protocol
- **HTTPS** - HyperText Transfer Protocol Secure
- **JS** - JavaScript
- **JSON** - JavaScript Object Notation
- **MVC** - Model-View-Controller
- **PHP** - PHP: Hypertext Preprocessor
- **SQL** - Structured Query Language
- **UI** - User Interface
- **UX** - User Experience
- **URL** - Uniform Resource Locator

HR Terminology

- **KPI** - Key Performance Indicator
- **KRA** - Key Result Area
- **HRMS** - Human Resource Management System
- **HRIS** - Human Resource Information System
- **LMS** - Learning Management System
- **ATS** - Applicant Tracking System

Database Terms

- **DDL** - Data Definition Language
- **DML** - Data Manipulation Language
- **DQL** - Data Query Language
- **TCL** - Transaction Control Language
- **PK** - Primary Key
- **FK** - Foreign Key
- **ERD** - Entity-Relationship Diagram

Development Terms

- **IDE** - Integrated Development Environment
- **OOP** - Object-Oriented Programming
- **REST** - Representational State Transfer
- **SDLC** - Software Development Life Cycle
- **UML** - Unified Modeling Language
- **W3C** - World Wide Web Consortium

TABLE OF CONTENTS

ABSTRACT 1

GLOSSARY AND ABBREVIATIONS ……. 2

TABLE OF CONTENTS ……. 3

CHAPTER 1: INTRODUCTION ……. 5

1.1 Background of the Study ……. 6

1.2 Problem Statement ……. 7

1.3 Objectives of the Study ……. 8

1.4 Scope of the Project ……. 9

1.5 Significance of the Study ……. 10

1.6 Project Methodology …… 11

1.7 Organization of the Report ……. 12

CHAPTER 2: SYSTEM ANALYSIS ……. 13

2.1 Introduction to System Analysis …… 14

2.2 Feasibility Study ……. 15

2.2.1 Technical Feasibility ……. 16

2.2.2 Economic Feasibility ……. 17

2.2.3 Operational Feasibility …… 18

2.2.4 Legal Feasibility ……. 19

2.3 Requirements Analysis ……. 20

2.3.1 Functional Requirements ……. 21

2.3.2 Non-Functional Requirements ……. 22

2.4 System Modeling …… 23

2.4.1 Use Case Diagram 24

2.4.2 Data Flow Diagram 25

2.4.3 Entity Relationship Diagram 26

2.5 System Design Considerations 27

CHAPTER 3: HYPERTEXT PREPROCESSOR (PHP) 28

3.1 Introduction to PHP 29

3.2 History and Evolution of PHP 30

3.3 PHP Features and Characteristics 31

3.4 PHP Syntax and Structure 32

3.5 Variables and Data Types in PHP 33

3.6 Control Structures 34

3.7 Functions and Object-Oriented Programming 35

3.8 PHP and Database Connectivity 36

3.9 PHP Security Features 37

3.10 PHP Frameworks and Libraries 38

3.11 PHP in Web Development 39

3.12 Advantages and Disadvantages of PHP 40

CHAPTER 4: MYSQL 41

4.1 Introduction to MySQL 42

4.2 History and Development of MySQL ……. 43

4.3 MySQL Architecture …… 44

4.4 MySQL Features and Capabilities …… 45

4.5 Data Types in MySQL ……. 46

4.6 MySQL Commands and Syntax ……. 47

4.7 Database Design Principles ……. 48

4.8 Indexing and Performance Optimization ……. 49

4.9 MySQL Security ……. 50

4.10 MySQL Administration and Maintenance ……. 51

4.11 MySQL vs Other Database Systems ……. 52

CHAPTER 6: HR-MANAGEMENT-SYSTEM …… 53

6.1 System Architecture …… 54

6.2 Database Design and Schema ……. 55

6.3 Module Structure and Organization …… 56

6.4 Authentication and Security System ……. 57

6.5 User Interface Design ……. 58

6.6 Core Modules Implementation …… 59

6.6.1 Recruitment Module …… 60

6.6.2 Employee Management Module …… 61

6.6.3 Attendance Management Module 62

6.6.4 Leave Management Module 63

6.6.5 Performance Management Module 64

6.6.6 Dashboard and Analytics Module 65

6.7 File Upload and Management 66

6.8 Error Handling and Validation 67

6.9 Session Management 68

6.10 Responsive Design Implementation 69

6.11 Integration with External Services 70

CHAPTER 7: SYSTEM TESTING 71

7.1 Introduction to Software Testing 72

7.2 Testing Methodology 73

7.3 Unit Testing 74

7.4 Integration Testing 75

7.5 System Testing 76

7.6 Performance Testing 77

7.7 Security Testing 78

7.8 Usability Testing 79

7.9 Test Cases and Test Results 80

7.10 Bug Tracking and Resolution 81

7.11 Testing Tools and Frameworks 82

7.12 Quality Assurance Measures 83

CONCLUSION 84

7.1 Summary of the Project 85

7.2 Achievements and Contributions 86

7.3 Limitations of the System 87

7.4 Future Enhancements 88

7.5 Lessons Learned 89

7.6 Recommendations 90

REFERENCES 91

APPENDICES 92

Appendix A: Source Code Listings 93

Appendix B: Database Schema 94

Appendix C: User Manual 95

Appendix D: Installation Guide 96

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

In today's rapidly evolving business environment, effective human resource management has become a critical factor for organizational success. Traditional HR management methods, which rely heavily on manual processes and paper-based documentation, are increasingly proving inadequate to meet the demands of

modern enterprises. The digital transformation era has necessitated the adoption of sophisticated technological solutions that can streamline HR operations, enhance efficiency, and provide strategic insights for decision-making.

Human Resource Management Systems (HRMS) have emerged as essential tools for organizations seeking to optimize their HR processes. These systems integrate various HR functions into a unified platform, enabling seamless data management, improved compliance, and enhanced employee experiences. The global HRMS market has experienced significant growth in recent years, driven by factors such as increasing workforce complexity, regulatory requirements, and the need for data-driven HR strategies.

The traditional approach to HR management involves numerous challenges including data redundancy, inefficient processes, lack of real-time information, and difficulties in generating comprehensive reports. Manual record-keeping systems are prone to errors, time-consuming, and often result in delayed decision-making. Furthermore, the growing trend of remote work and distributed teams has created additional complexities that traditional HR systems cannot effectively address.

Modern organizations require HR solutions that are not only efficient and reliable but also scalable, secure, and user-friendly. The need for integrated systems that can handle recruitment, onboarding, performance management, payroll, and other HR functions has become increasingly apparent. This has led to the development of comprehensive HR Management Systems that leverage modern web technologies to provide centralized, automated, and intelligent HR management capabilities.

The project addresses these challenges by developing a web-based HR Management System that combines the power of PHP and MySQL to create a robust, scalable, and feature-rich solution. The system is designed to meet the specific needs of small to medium-sized enterprises while maintaining the flexibility to adapt to larger organizational requirements.

1.2 Problem Statement

The management of human resources in organizations faces numerous challenges that hinder operational efficiency and strategic decision-making. Traditional HR management systems suffer from several critical limitations that this project aims to address:

Data Management Issues: - Fragmented data storage across multiple systems and spreadsheets - Inconsistent data formats and standards - Difficulty in maintaining data accuracy and integrity - Time-consuming data reconciliation processes - Limited data visibility and accessibility

Process Inefficiencies: - Manual and repetitive administrative tasks - Lack of automation in routine HR processes - Delayed approval workflows for leave

requests and other HR transactions - Inefficient recruitment and onboarding processes - Poor coordination between different HR functions

Reporting and Analytics Limitations: - Inability to generate real-time reports and insights - Limited analytical capabilities for workforce planning - Difficulty in tracking key HR metrics and KPIs - Lack of predictive analytics for talent management - Insufficient data for strategic decision-making

Security and Compliance Concerns: - Vulnerability of sensitive employee data to unauthorized access - Difficulty in maintaining audit trails and compliance records - Challenges in implementing role-based access control - Risk of data loss and backup issues - Compliance with data protection regulations

User Experience and Accessibility: - Complex and unintuitive user interfaces - Limited accessibility for remote employees - Lack of mobile-friendly interfaces - Poor integration with other business systems - Inadequate training and support resources

Scalability and Maintenance: - Inability to scale with organizational growth - High maintenance costs for legacy systems - Difficulty in implementing new features and updates - Limited integration capabilities with third-party applications - Obsolete technology stack and architecture

These problems result in decreased productivity, increased operational costs, poor employee satisfaction, and missed opportunities for organizational improvement. The development of a modern, web-based HR Management System is essential to address these challenges and provide organizations with the tools they need to effectively manage their human resources.

1.3 Objectives of the Study

The primary objective of this project is to design, develop, and implement a comprehensive web-based Human Resource Management System that addresses the limitations of traditional HR management approaches. The specific objectives are:

Primary Objectives: 1. To develop a centralized HR management platform that integrates all major HR functions 2. To automate routine HR processes to improve operational efficiency 3. To implement robust security measures to protect sensitive employee data 4. To create an intuitive and user-friendly interface for HR administrators and employees 5. To provide comprehensive reporting and analytics capabilities

Secondary Objectives: 1. To design a scalable system architecture that can accommodate organizational growth 2. To implement role-based access control for different user types 3. To develop mobile-responsive interfaces for accessibility across devices 4. To create efficient database schemas for optimal data management 5. To integrate modern web technologies for enhanced functionality

Technical Objectives: 1. To utilize PHP for server-side scripting and business logic implementation 2. To implement MySQL for efficient data storage and retrieval 3. To apply responsive web design principles using HTML5, CSS3, and JavaScript 4. To implement secure authentication and session management mechanisms 5. To develop RESTful APIs for system integration and extensibility

Functional Objectives: 1. To create a comprehensive recruitment module for candidate management 2. To develop an employee management system for personnel administration 3. To implement an attendance tracking system with automated calculations 4. To create a leave management system with approval workflows 5. To develop a performance appraisal system for employee evaluation

Quality Objectives: 1. To ensure system reliability and availability 2. To implement comprehensive error handling and logging mechanisms 3. To achieve optimal system performance and response times 4. To maintain code quality and documentation standards 5. To ensure compliance with data protection regulations

Business Objectives: 1. To reduce administrative overhead and operational costs 2. To improve data accuracy and decision-making capabilities 3. To enhance employee satisfaction and engagement 4. To provide strategic insights for workforce planning 5. To create a foundation for digital HR transformation

1.4 Scope of the Project

The HR Management System project encompasses a comprehensive set of features and functionalities designed to address the core needs of modern human resource management. The scope of this project is defined to ensure focused development while maintaining flexibility for future enhancements.

In Scope:

Core HR Functions: - Employee information management and record-keeping - Recruitment and candidate management system - Attendance tracking and monitoring - Leave management with approval workflows - Performance appraisal and evaluation system - Dashboard and analytics for HR metrics

User Management: - Multi-level user authentication system - Role-based access control (Admin, HR Manager, Employee) - User profile management and permissions - Session management and security features

Technical Implementation: - Web-based application using PHP and MySQL - Responsive design for desktop and mobile devices - Real-time data updates and notifications - File upload and document management - Email integration for notifications and communications

Data Management: - Centralized database for all HR-related information - Data backup and recovery mechanisms - Data validation and integrity checks - Audit trail for system activities - Export functionality for reports and data

Reporting and Analytics: - Real-time dashboard with key metrics - Customizable reports for various HR functions - Data visualization using charts and graphs - Historical data analysis and trend identification - Export capabilities for external analysis

Out of Scope:

Advanced HR Functions: - Payroll and compensation management - Benefits administration - Time and labor management with complex scheduling - Training and learning management system - Succession planning and career development

Enterprise Features: - Multi-company or multi-tenant architecture - Advanced workflow automation - Artificial intelligence and machine learning capabilities - Advanced integration with ERP systems - Global compliance and localization features

Technical Limitations: - Mobile application development (native apps) - Advanced security features like biometric authentication - Cloud deployment and scalability features - Advanced caching and performance optimization - Real-time collaboration features

Regulatory Compliance: - Industry-specific compliance requirements - Advanced data privacy regulations (GDPR, etc.) - Labor law integration and compliance checking - Advanced audit and compliance reporting - Legal document management

The project scope is designed to deliver a comprehensive yet focused solution that addresses the most critical HR management needs while remaining achievable within the project timeline and resource constraints. Future enhancements can extend the system to include advanced features based on user feedback and organizational requirements.

1.5 Significance of the Study

The development of this HR Management System holds significant importance for various stakeholders including organizations, HR professionals, employees, and the academic community. The significance of this project can be understood from multiple perspectives:

Organizational Benefits: - **Operational Efficiency:** Automation of routine HR tasks reduces administrative overhead and allows HR professionals to focus on strategic initiatives - **Cost Reduction:** Elimination of manual processes, reduced paperwork, and improved resource allocation lead to significant cost savings - **Data-Driven Decision Making:** Access to real-time analytics and comprehensive reports enables informed decision-making and strategic planning - **Compliance Management:** Automated tracking and reporting ensure adherence to labor laws and regulatory requirements - **Scalability:** The system

can grow with the organization, accommodating increasing complexity and user numbers

HR Professional Advantages: - **Process Standardization:** Consistent procedures and workflows across all HR functions improve quality and reliability - **Time Savings:** Automation of repetitive tasks frees up time for value-added activities like employee development and strategic planning - **Improved Accuracy:** Reduced manual data entry minimizes errors and improves data quality - **Enhanced Productivity:** Streamlined processes and better tools increase overall HR department productivity - **Professional Development:** Exposure to modern HR technology enhances skills and career prospects

Employee Experience Improvements: - **Self-Service Capabilities:** Employees can manage their own information, request leaves, and access HR services independently - **Transparency:** Clear processes and visibility into HR operations improve trust and satisfaction - **Accessibility:** Web-based access allows employees to use the system from anywhere, at any time - **Faster Response Times:** Automated workflows reduce processing times for various HR requests - **Better Communication:** Integrated notification systems keep employees informed about relevant HR matters

Academic and Research Contributions: - **Practical Implementation:** The project demonstrates the practical application of theoretical concepts in software engineering and database management - **Case Study:** Serves as a comprehensive case study for students and researchers studying web application development - **Best Practices:** Illustrates industry best practices in system design, security implementation, and user experience - **Technology Integration:** Shows effective integration of multiple technologies (PHP, MySQL, HTML, CSS, JavaScript) in a real-world application - **Documentation:** Comprehensive documentation provides valuable learning material for future developers

Industry Impact: - **Digital Transformation:** Contributes to the broader trend of digital transformation in HR management - **Small Business Enablement:** Provides affordable solutions for small and medium-sized enterprises that cannot afford expensive commercial HRMS - **Innovation:** Demonstrates innovative approaches to common HR challenges using modern web technologies - **Knowledge Sharing:** Open-source nature of the project allows knowledge sharing and community improvement - **Standardization:** Helps establish standards for HR system development and implementation

Societal Benefits: - **Employment:** Creates opportunities for developers and IT professionals in the HR technology sector - **Skill Development:** Contributes to the development of technical skills in the workforce - **Economic Growth:** Enables businesses to operate more efficiently, contributing to economic productivity - **Work-Life Balance:** Efficient HR systems can indirectly contribute to better work-life balance for employees - **Modernization:** Helps organizations modernize their operations and stay competitive in the digital age

1.6 Project Methodology

The development of the HR Management System follows a structured methodology that ensures systematic progress, quality deliverables, and successful project completion. The methodology combines elements of traditional software engineering practices with modern agile approaches to optimize development efficiency and product quality.

Development Approach: The project employs a hybrid methodology that combines the structured planning of the Waterfall model for initial phases with the flexibility of Agile practices for development and testing. This approach allows for thorough planning and requirement analysis while maintaining adaptability during implementation.

Phase 1: Planning and Analysis (Weeks 1-2) - Requirement gathering and analysis - Feasibility study and risk assessment - Technology stack selection and justification - Project timeline and resource allocation - Documentation of project scope and objectives

Phase 2: System Design (Weeks 3-4) - Database schema design and normalization - System architecture and module design - User interface design and prototyping - Security framework design - API design and integration planning

Phase 3: Development and Implementation (Weeks 5-10) - Backend development using PHP - Database implementation using MySQL - Frontend development using HTML, CSS, and JavaScript - Module-wise development and integration - Continuous testing and debugging

Phase 4: Testing and Quality Assurance (Weeks 11-12) - Unit testing of individual components - Integration testing of modules - System testing and performance optimization - Security testing and vulnerability assessment - User acceptance testing

Phase 5: Deployment and Documentation (Weeks 13-14) - System deployment and configuration - User training and documentation - Performance monitoring and optimization - Final documentation and project submission

Development Tools and Technologies: - **Programming Languages:** PHP, HTML5, CSS3, JavaScript - **Database Management:** MySQL 8.0+ - **Web Server:** Apache or Nginx - **Development Environment:** XAMPP/WAMP/MAMP - **Version Control:** Git - **IDE:** Visual Studio Code or similar - **Design Tools:** Figma or Adobe XD for UI design - **Testing Tools:** PHPUnit for PHP testing, browser developer tools

Quality Assurance Measures: - Code review and peer programming - Automated testing where applicable - Performance monitoring and optimization - Security audits and vulnerability scanning - User experience testing and feedback collection - Documentation review and updates

Project Management Practices: - Regular progress tracking and milestone

monitoring - Risk management and mitigation strategies - Stakeholder communication and feedback incorporation - Change management and version control - Resource allocation and timeline management - Quality metrics and success criteria definition

Documentation Strategy: - Comprehensive technical documentation - User manuals and training materials - API documentation for future integration - Database documentation and schema diagrams - Installation and deployment guides - Maintenance and troubleshooting guides

This methodology ensures that the project is developed systematically while maintaining flexibility to adapt to changing requirements and technical challenges. The structured approach facilitates quality deliverables and successful project completion within the specified timeline.

1.7 Organization of the Report

This comprehensive documentation is organized into logical chapters and sections to provide a clear understanding of the HR Management System project. The report structure is designed to guide readers through the project from conception to implementation and testing.

Chapter 1: Introduction Provides an overview of the project including background information, problem statement, objectives, scope, significance, methodology, and report organization. This chapter sets the context for the entire project and helps readers understand the motivation behind the system development.

Chapter 2: System Analysis Details the analysis phase of the project, including feasibility studies, requirements analysis, system modeling, and design considerations. This chapter covers technical, economic, operational, and legal feasibility, along with functional and non-functional requirements.

Chapter 3: Hypertext Preprocessor (PHP) Focuses on the server-side programming language used in the project. This chapter covers PHP fundamentals, features, syntax, database connectivity, security aspects, and its role in web development. It provides technical background for understanding the implementation.

Chapter 4: MySQL Covers the database management system used for data storage and retrieval. This chapter includes MySQL architecture, features, data types, commands, design principles, security, and administration. It provides the foundation for understanding the database implementation.

Chapter 6: HR-Management-System The core implementation chapter that details the actual system development. This chapter covers system architecture, database design, module structure, authentication, UI design, and implementation of all major modules including recruitment, employee management, attendance, leave management, and performance evaluation.

Chapter 7: System Testing Comprehensive coverage of the testing phase including testing methodology, unit testing, integration testing, system testing, performance testing, security testing, and usability testing. This chapter also includes test cases, results, and quality assurance measures.

Conclusion Summarizes the project achievements, discusses limitations, suggests future enhancements, and provides recommendations. This chapter reflects on the project outcomes and provides insights for future development.

References Lists all the sources, books, articles, and online resources referenced during the project development and documentation.

Appendices Contains supplementary materials including source code listings, database schema, user manual, installation guide, and other relevant documentation that supports the main report.

Report Features: - **Progressive Disclosure:** Information is presented in a logical sequence, building from basic concepts to detailed implementation - **Cross-Referencing:** Related topics are cross-referenced throughout the document for easy navigation - **Visual Elements:** Diagrams, charts, and illustrations are used to enhance understanding - **Technical Depth:** Sufficient technical detail is provided for developers and system administrators - **Practical Focus:** Emphasis is placed on practical implementation and real-world application - **Academic Rigor:** The documentation maintains academic standards while being practically useful

This organization ensures that readers can easily navigate through the documentation and find relevant information based on their specific needs and interests.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Introduction to System Analysis

System analysis is a critical phase in the software development life cycle that involves studying the existing system or processes to identify problems, opportunities, and objectives. In the context of HR Management System development, system analysis encompasses thorough examination of current HR practices, identification of inefficiencies, and definition of requirements for the new system.

The analysis phase serves as foundation for successful system development by ensuring that the proposed solution addresses real business needs and aligns with organizational objectives. This phase involves multiple stakeholders including HR professionals, employees, management, and IT staff to gather comprehensive requirements and understand various perspectives.

Purpose of System Analysis: - To understand current HR processes and identify pain points - To define clear objectives and scope for the new system - To assess feasibility of proposed solutions - To identify functional and non-functional requirements - To establish baseline metrics for measuring system success - To minimize risks and ensure project alignment with business goals

Analysis Methodology: The system analysis for HR Management System follows a structured approach that combines various techniques and tools:

1. **Requirements Gathering:** Through interviews, surveys, and observation of current HR processes
2. **Process Mapping:** Documentation of existing workflows and identification of improvement opportunities
3. **Stakeholder Analysis:** Identification of all system users and their specific needs
4. **Gap Analysis:** Comparison between current capabilities and desired functionalities
5. **Feasibility Assessment:** Evaluation of technical, economic, operational, and legal aspects
6. **Risk Assessment:** Identification of potential challenges and mitigation strategies

Analysis Deliverables: - Detailed requirements specification document - Process flow diagrams and use cases - Feasibility study report - Risk assessment matrix - System design recommendations - Implementation roadmap

This comprehensive analysis ensures that the HR Management System is built on solid foundation of understanding and addresses all critical aspects of modern HR management.

2.2 Feasibility Study

A feasibility study is conducted to evaluate the viability of the HR Management System project from multiple perspectives. This analysis helps in making informed decisions about project continuation and resource allocation. The feasibility study examines four key areas: technical, economic, operational, and legal feasibility.

Feasibility Study Objectives: - To assess whether the project can be successfully implemented with available resources - To identify potential risks and challenges that may affect project success - To determine the return on investment and economic benefits - To evaluate alignment with organizational capabilities and constraints - To ensure compliance with legal and regulatory requirements

Methodology: The feasibility study employs various data collection and analysis techniques including market research, technical evaluation, cost-benefit analysis, stakeholder interviews, and compliance review. Each feasibility aspect is evaluated independently and then integrated to provide overall project viability assessment.

The following sections detail each feasibility aspect with specific considerations for the HR Management System project.

2.2.1 Technical Feasibility

Technical feasibility assesses whether the proposed HR Management System can be successfully developed and implemented using available technology resources and expertise. This evaluation considers current technology infrastructure, development capabilities, and system requirements.

Technology Stack Assessment: - **PHP Development:** Evaluate team expertise in PHP programming and related frameworks - **MySQL Database:** Assess database administration skills and infrastructure capacity - **Web Technologies:** Review proficiency in HTML5, CSS3, JavaScript, and responsive design - **Server Infrastructure:** Evaluate hosting capabilities and server requirements - **Security Implementation:** Assess knowledge of web security best practices

System Requirements Analysis: - **Hardware Requirements:** Server specifications, storage capacity, and network bandwidth - **Software Requirements:** Operating systems, web servers, and development tools - **Performance Requirements:** Expected user load, response times, and scalability needs - **Integration Requirements:** Compatibility with existing systems and third-party services

Development Capabilities: - **Team Skills:** Assessment of development team's technical competencies - **Development Tools:** Availability of necessary software and development environments - **Testing Infrastructure:** Capability to conduct comprehensive testing and quality assurance - **Deployment Expertise:** Knowledge of deployment procedures and maintenance

Risk Assessment: - **Technology Risks:** Potential technical challenges and mitigation strategies - **Scalability Concerns:** Ability to handle future growth and increased user load - **Security Vulnerabilities:** Potential security threats and protection measures - **Maintenance Requirements:** Ongoing technical support and update needs

Technical Feasibility Conclusion: The HR Management System is technically feasible given the widespread availability of PHP and MySQL expertise, mature development tools, and robust hosting infrastructure. The technology stack is well-established with extensive community support and documentation. The modular architecture allows for incremental development and testing, reducing technical risks.

2.2.2 Economic Feasibility

Economic feasibility evaluates the financial viability of the HR Management System project by analyzing costs, benefits, and return on investment. This as-

sessment helps determine whether the project provides sufficient value to justify the investment.

Cost Analysis:

Development Costs: - **Personnel Costs:** Salaries for developers, designers, and project managers - **Infrastructure Costs:** Server hosting, domain registration, and software licenses - **Training Costs:** Staff training for system usage and maintenance - **Testing Costs:** Quality assurance, user acceptance testing, and bug fixing - **Documentation Costs:** Technical documentation and user manuals

Operational Costs: - **Maintenance Costs:** Ongoing system updates, bug fixes, and technical support - **Hosting Costs:** Server maintenance, bandwidth, and backup services - **Security Costs:** SSL certificates, security audits, and vulnerability scanning - **Training Costs:** New employee training and refresher courses - **Upgrade Costs:** System enhancements and feature additions

Benefit Analysis:

Quantitative Benefits: - **Cost Savings:** Reduction in administrative overhead and paperwork processing - **Time Savings:** Automation of routine tasks and faster processing times - **Error Reduction:** Decreased data entry errors and improved data accuracy - **Productivity Gains:** Increased efficiency in HR operations and decision-making

Qualitative Benefits: - **Improved Decision Making:** Better data availability and analytics capabilities - **Enhanced Employee Experience:** Self-service capabilities and faster response times - **Compliance Improvement:** Better adherence to labor laws and regulations - **Strategic Alignment:** Support for organizational growth and digital transformation

Return on Investment (ROI) Analysis: - **Implementation Period:** 3-4 months for full system deployment - **Payback Period:** Estimated 12-18 months based on cost savings - **ROI Percentage:** Projected 150-200% ROI over 3-year period - **Net Present Value:** Positive NPV indicating financial viability

Cost-Benefit Ratio: - **Total Investment:** Estimated $25,000-35,000 for complete system - **Annual Benefits:** Estimated $20,000-30,000 in operational savings - **Benefit-Cost Ratio:** 1.5:1 to 2.0:1 indicating favorable economics

Economic Feasibility Conclusion: The HR Management System demonstrates strong economic feasibility with positive ROI, reasonable payback period, and significant long-term benefits. The investment is justified by substantial operational savings and strategic advantages.

2.2.3 Operational Feasibility

Operational feasibility assesses whether the HR Management System can be successfully integrated into existing organizational processes and workflows. This

evaluation considers user acceptance, organizational readiness, and operational impacts.

Organizational Readiness Assessment: - **Management Support:** Evaluation of executive sponsorship and commitment - **User Acceptance:** Assessment of employee willingness to adopt new system - **Change Management:** Capability to manage organizational transition - **Training Infrastructure:** Availability of training resources and programs - **Support Structure:** Technical support and help desk capabilities

Process Integration Analysis: - **Workflow Compatibility:** Alignment with existing HR processes and procedures - **Data Migration:** Feasibility of transferring existing data to new system - **Parallel Operations:** Ability to run old and new systems simultaneously during transition - **Business Continuity:** Minimization of disruption during implementation - **Process Improvement:** Opportunities to streamline and optimize workflows

User Considerations: - **User Skills:** Assessment of computer literacy and technical capabilities - **Learning Curve:** Time required for users to become proficient with new system - **Accessibility:** Availability of system access for all user groups - **Language and Localization:** Support for multiple languages if required - **Disability Access:** Compliance with accessibility standards

Organizational Impact: - **Job Role Changes:** Modifications to job descriptions and responsibilities - **Department Structure:** Changes to HR department organization - **Communication Channels:** New methods for employee-HR interactions - **Decision Making:** Changes in approval processes and authority levels - **Performance Metrics:** New ways to measure HR department effectiveness

Training and Support Requirements: - **Initial Training:** Comprehensive training programs for all user groups - **Ongoing Support:** Help desk and technical assistance infrastructure - **Documentation:** User manuals, quick reference guides, and online help - **Super User Program:** Training of power users to support their colleagues - **Refresher Training:** Periodic training updates and skill enhancement

Risk Mitigation Strategies: - **Change Resistance:** Strategies to overcome resistance to new system - **Data Quality:** Measures to ensure data accuracy during migration - **System Downtime:** Minimization of service interruptions - **User Errors:** Prevention and correction of user mistakes - **Security Breaches:** Protection of sensitive HR information

Operational Feasibility Conclusion: The HR Management System is operationally feasible with strong management support, reasonable user acceptance, and adequate training infrastructure. The modular implementation approach allows for gradual adoption and minimizes operational disruption. Comprehensive change management strategies ensure successful system integration.

2.2.4 Legal Feasibility

Legal feasibility evaluates whether the HR Management System complies with applicable laws, regulations, and industry standards. This assessment is crucial for HR systems due to the sensitive nature of employee data and strict regulatory requirements.

Data Protection and Privacy Laws: - **General Data Protection Regulation (GDPR):** Compliance with EU data protection standards - **California Consumer Privacy Act (CCPA):** Adherence to California privacy regulations - **Personal Information Protection Act (PIPA):** Compliance with local data protection laws - **Data Breach Notification:** Requirements for reporting security incidents - **Data Retention Policies:** Legal requirements for data storage and deletion

Employment and Labor Laws: - **Equal Employment Opportunity (EEO):** Compliance with anti-discrimination laws - **Fair Labor Standards Act (FLSA):** Adherence to wage and hour regulations - **Family and Medical Leave Act (FMLA):** Compliance with leave regulations - **Occupational Safety and Health Act (OSHA):** Workplace safety requirements - **Immigration Reform and Control Act:** Employment eligibility verification

Industry Standards and Best Practices: - **ISO 27001:** Information security management standards - **SOC 2 Type II:** Security and compliance auditing standards - **HIPAA Compliance:** Healthcare industry data protection (if applicable) - **Payment Card Industry (PCI):** Credit card data security (if applicable) - **Web Accessibility Standards:** WCAG 2.1 compliance for accessibility

Contractual and Legal Obligations: - **Software Licensing:** Compliance with open source and commercial licenses - **Service Level Agreements:** Adherence to contractual commitments - **Third-Party Integrations:** Legal compliance of external service providers - **Employee Agreements:** Alignment with employment contracts and policies - **Union Requirements:** Compliance with collective bargaining agreements

Intellectual Property Considerations: - **Copyright Compliance:** Use of licensed software and content - **Trademark Usage:** Proper use of company and third-party trademarks - **Patent Considerations:** Avoidance of patent infringement - **Open Source Licenses:** Compliance with open source software requirements - **Custom Code Ownership:** Clarification of intellectual property rights

Audit and Compliance Requirements: - **Audit Trails:** Comprehensive logging of system activities - **Report Generation:** Ability to produce compliance reports - **Data Export:** Capability to provide data for legal proceedings - **Document Retention:** Legal requirements for document storage - **Access Controls:** Restriction of sensitive data access

Risk Assessment and Mitigation: - **Legal Risks:** Potential lawsuits and regulatory penalties - **Compliance Monitoring:** Regular review of legal requirements - **Legal Consultation:** Engagement with legal experts for compliance - **Policy Updates:** Regular updates to policies and procedures - **Employee Training:** Legal compliance education for HR staff

Legal Feasibility Conclusion: The HR Management System is legally feasible with proper implementation of data protection measures, compliance features, and regular legal reviews. The system design incorporates necessary security controls, audit trails, and compliance reporting capabilities to meet legal requirements.

2.3 Requirements Analysis

Requirements analysis is a critical process that identifies, documents, and manages the needs of various stakeholders for the HR Management System. This phase ensures that the developed system meets all functional and non-functional requirements while aligning with organizational objectives.

Requirements Gathering Process: - **Stakeholder Interviews:** Direct conversations with HR professionals, management, and employees - **Surveys and Questionnaires:** Structured data collection from large user groups - **Observation Studies:** Monitoring of current HR processes and workflows - **Document Analysis:** Review of existing HR policies, procedures, and forms - **Competitive Analysis:** Study of existing HRMS solutions and industry best practices - **Use Case Development:** Detailed scenarios of system usage and interactions

Requirements Classification: Requirements are categorized into functional and non-functional requirements to ensure comprehensive coverage and proper prioritization during development.

Requirements Management: - **Requirements Documentation:** Detailed specification documents with clear descriptions - **Traceability Matrix:** Linking requirements to design elements and test cases - **Change Control:** Process for managing requirement changes and updates - **Priority Setting:** Classification of requirements as critical, important, or desirable - **Validation Process:** Review and approval of requirements by stakeholders

Quality Assurance: - **Requirements Review:** Regular review sessions with all stakeholders - **Consistency Checking:** Ensuring requirements do not conflict with each other - **Completeness Verification:** Confirming all necessary requirements are identified - **Testability Assessment:** Ensuring requirements can be verified through testing - **Feasibility Validation:** Confirming requirements can be implemented within constraints

The following sections detail the functional and non-functional requirements identified for the HR Management System.

2.3.1 Functional Requirements

Functional requirements define what the HR Management System should do, specifying the specific behaviors, functions, and features that the system must provide. These requirements are derived from business needs and user expectations.

Authentication and Authorization Requirements: - **FR-001:** System shall provide secure login functionality for administrators and employees - **FR-002:** System shall implement role-based access control with different permission levels - **FR-003:** System shall support password recovery and reset functionality - **FR-004:** System shall maintain session management with automatic timeout - **FR-005:** System shall provide audit trail of all user activities

Dashboard and Analytics Requirements: - **FR-006:** System shall display real-time HR metrics and statistics on main dashboard - **FR-007:** System shall provide interactive charts and graphs for data visualization - **FR-008:** System shall allow customization of dashboard widgets and layouts - **FR-009:** System shall generate automated reports for key HR indicators - **FR-010:** System shall support export of dashboard data in various formats

Employee Management Requirements: - **FR-011:** System shall maintain comprehensive employee profiles with personal and professional information - **FR-012:** System shall support employee onboarding from selected candidates - **FR-013:** System shall generate unique employee identification numbers automatically - **FR-014:** System shall track employee career progression and role changes - **FR-015:** System shall maintain employee status and employment history

Recruitment Management Requirements: - **FR-016:** System shall support job posting and vacancy management - **FR-017:** System shall handle candidate applications and document uploads - **FR-018:** System shall track candidate status through recruitment pipeline - **FR-019:** System shall provide candidate search and filtering capabilities - **FR-020:** System shall support communication with candidates via email integration

Attendance Management Requirements: - **FR-021:** System shall record daily attendance for all employees - **FR-022:** System shall support different attendance statuses (present, absent, late, half-day) - **FR-023:** System shall calculate attendance statistics and summaries - **FR-024:** System shall provide calendar view of attendance records - **FR-025:** System shall generate attendance reports for payroll processing

Leave Management Requirements: - **FR-026:** System shall support multiple leave types with different policies - **FR-027:** System shall allow employees to submit leave requests online - **FR-028:** System shall implement approval workflow for leave requests - **FR-029:** System shall track and manage leave balances for each employee - **FR-030:** System shall generate leave reports and analytics

Performance Management Requirements: - **FR-031:** System shall support performance review scheduling and management - **FR-032:** System shall provide customizable performance evaluation criteria - **FR-033:** System shall track performance history and trends - **FR-034:** System shall support goal setting and progress tracking - **FR-035:** System shall generate performance reports and development plans

Document Management Requirements: - **FR-036:** System shall support file upload for resumes and other documents - **FR-037:** System shall provide secure storage and retrieval of documents - **FR-038:** System shall implement file type and size restrictions - **FR-039:** System shall maintain version control for important documents - **FR-040:** System shall provide document search and filtering capabilities

Reporting Requirements: - **FR-041:** System shall generate standard HR reports for management - **FR-042:** System shall support custom report creation and scheduling - **FR-043:** System shall provide data export in multiple formats (PDF, Excel, CSV) - **FR-044:** System shall maintain report templates and formats - **FR-045:** System shall support scheduled report generation and distribution

Communication Requirements: - **FR-046:** System shall send automated email notifications for various events - **FR-047:** System shall maintain communication logs with employees and candidates - **FR-048:** System shall support bulk email communication to user groups - **FR-049:** System shall provide email template management - **FR-050:** System shall track communication history and responses

2.3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints that the HR Management System must satisfy. These requirements specify how the system should perform rather than what it should do.

Performance Requirements: - **NFR-001:** System shall respond to user interactions within 3 seconds under normal load - **NFR-002:** System shall support concurrent access for 100+ users without degradation - **NFR-003:** System shall complete database queries within 2 seconds for standard operations - **NFR-004:** System shall handle file uploads up to 10MB within 30 seconds - **NFR-005:** System shall maintain 99.5% uptime during business hours

Security Requirements: - **NFR-006:** System shall encrypt sensitive data both in transit and at rest - **NFR-007:** System shall implement secure password policies with minimum complexity requirements - **NFR-008:** System shall prevent SQL injection and cross-site scripting attacks - **NFR-009:** System shall maintain comprehensive audit logs for all system activities - **NFR-010:** System shall comply with data protection regulations (GDPR, CCPA)

Usability Requirements: - **NFR-011:** System shall provide intuitive user

interface requiring minimal training - **NFR-012:** System shall be accessible to users with disabilities (WCAG 2.1 AA) - **NFR-013:** System shall support multiple browsers (Chrome, Firefox, Safari, Edge) - **NFR-014:** System shall provide responsive design for mobile and tablet devices - **NFR-015:** System shall maintain consistent design patterns across all modules

Reliability Requirements: - **NFR-016:** System shall perform data backups automatically every 24 hours - **NFR-017:** System shall recover from failures within 5 minutes - **NFR-018:** System shall maintain data integrity during concurrent operations - **NFR-019:** System shall provide error handling with user-friendly messages - **NFR-020:** System shall validate all input data before processing

Scalability Requirements: - **NFR-021:** System shall support growth to 1000+ employees without performance degradation - **NFR-022:** System shall handle database growth up to 10GB without optimization - **NFR-023:** System shall support horizontal scaling through load balancing - **NFR-024:** System shall maintain performance during peak usage periods - **NFR-025:** System shall support addition of new modules without architectural changes

Maintainability Requirements: - **NFR-026:** System shall be developed using modular architecture for easy maintenance - **NFR-027:** System shall include comprehensive documentation for all components - **NFR-028:** System shall support automated testing for quality assurance - **NFR-029:** System shall implement coding standards and best practices - **NFR-030:** System shall provide logging and debugging capabilities

Compatibility Requirements: - **NFR-031:** System shall be compatible with PHP 7.4+ and MySQL 5.7+ - **NFR-032:** System shall support modern web standards (HTML5, CSS3, ES6) - **NFR-033:** System shall integrate with existing email systems (SMTP) - **NFR-034:** System shall support data import/export in standard formats - **NFR-035:** System shall be deployable on common hosting platforms

Data Management Requirements: - **NFR-036:** System shall maintain referential integrity in database relationships - **NFR-037:** System shall implement proper data normalization to avoid redundancy - **NFR-038:** System shall provide data archiving capabilities for historical records - **NFR-039:** System shall support data migration and backup/restore operations - **NFR-040:** System shall implement data retention policies compliant with regulations

2.4 System Modeling

System modeling involves creating visual representations of the HR Management System to better understand its structure, behavior, and interactions. Various modeling techniques are used to document different aspects of the system and facilitate communication among stakeholders.

Modeling Objectives: - To provide clear visual representation of system ar-

chitecture and processes - To facilitate communication between technical and non-technical stakeholders - To identify potential design issues and optimization opportunities - To serve as blueprint for system development and implementation - To support system maintenance and future enhancements

Modeling Techniques Used: - **Use Case Diagrams:** Illustrate system functionality from user perspective - **Data Flow Diagrams:** Show data movement through the system - **Entity Relationship Diagrams:** Represent database structure and relationships - **Class Diagrams:** Document object-oriented design and relationships - **Sequence Diagrams:** Show interaction between system components over time - **State Diagrams:** Represent system states and transitions

Modeling Standards: - **UML 2.5:** Unified Modeling Language for consistent notation - **BPMN 2.0:** Business Process Model and Notation for process modeling - **ERD Conventions:** Standard entity-relationship diagram notation - **Flowchart Symbols:** Standard flowchart symbols for process representation

The following sections present key system models that provide comprehensive understanding of the HR Management System.

2.4.1 Use Case Diagram

Use case diagrams provide a high-level view of system functionality by showing interactions between users (actors) and the system. These diagrams help identify system boundaries, user roles, and major functions.

Primary Actors: - **Administrator:** System superuser with full access to all functions - **HR Manager:** Manages HR operations with limited administrative privileges - **Employee:** Accesses personal information and submits requests - **Candidate:** Applies for jobs and tracks application status - **Manager:** Reviews and approves requests from team members

Core Use Cases:

Administrator Use Cases: - **UC-001:** Manage System Configuration - **UC-002:** User Account Management - **UC-003:** System Backup and Restore - **UC-004:** Generate System Reports - **UC-005:** Monitor System Performance

HR Manager Use Cases: - **UC-006:** Employee Management - **UC-007:** Recruitment Management - **UC-008:** Leave Approval - **UC-009:** Performance Review Management - **UC-010:** Attendance Monitoring

Employee Use Cases: - **UC-011:** View Personal Information - **UC-012:** Update Profile - **UC-013:** Submit Leave Request - **UC-014:** View Attendance Records - **UC-015:** Access Performance Reviews

Candidate Use Cases: - **UC-016:** Search Job Openings - **UC-017:** Submit Application - **UC-018:** Upload Resume - **UC-019:** Track Application Status - **UC-020:** Update Application Information

Manager Use Cases: - **UC-021:** Approve Team Leave Requests - **UC-022:** Review Team Performance - **UC-023:** Submit Team Attendance - **UC-024:** Assign Tasks to Team Members - **UC-025:** Generate Team Reports

Use Case Relationships: - **Include Relationships:** Common functionality shared across multiple use cases - **Extend Relationships:** Optional functionality that extends base use cases - **Generalization Relationships:** Inheritance between actor types - **Association Relationships:** Communication between actors and use cases

Use Case Priorities: - **Critical:** Core functionality required for system operation - **Important:** Significant functionality that enhances system value - **Desirable:** Nice-to-have features that improve user experience

2.4.2 Data Flow Diagram

Data Flow Diagrams (DFDs) illustrate how data moves through the HR Management System, showing inputs, outputs, processes, and data stores. These diagrams help understand system behavior and identify data transformation points.

DFD Levels: - **Level 0 DFD:** Context diagram showing system as single process - **Level 1 DFD:** Major system processes and data flows - **Level 2 DFD:** Detailed processes within major functions - **Level 3 DFD:** Elementary processes with detailed specifications

Level 0 DFD - Context Diagram: - **External Entities:** Users, HR Department, Management, Candidates - **Central Process:** HR Management System - **Data Flows:** Input requests, output reports, data updates - **System Boundary:** Clearly defined scope of HRMS

Level 1 DFD - Major Processes: - **Process 1:** Authentication and Authorization - **Process 2:** Employee Management - **Process 3:** Recruitment Management - **Process 4:** Attendance Management - **Process 5:** Leave Management - **Process 6:** Performance Management - **Process 7:** Reporting and Analytics

Data Stores: - **DS-1:** Employee Database - **DS-2:** Candidate Database - **DS-3:** Attendance Database - **DS-4:** Leave Database - **DS-5:** Performance Database - **DS-6:** System Configuration

Data Flow Examples: - **Input Flows:** User login, employee data, leave requests, performance reviews - **Output Flows:** Dashboard data, reports, notifications, confirmations - **Internal Flows:** Data validation, processing, storage, retrieval - **Control Flows:** Error messages, status updates, system logs

Process Specifications: - **Mini-Specs:** Detailed descriptions of each process - **Input/Output Definitions:** Data structures and formats - **Processing Logic:** Algorithms and business rules - **Error Handling:** Exception processing and recovery

Data Dictionary: - **Data Elements:** Definitions of all data items - **Data Structures:** Organization of related data elements - **Data Flows:** Composition and meaning of data flows - **Data Stores:** Content and organization of databases

2.4.3 Entity Relationship Diagram

Entity Relationship Diagrams (ERDs) represent the database structure of the HR Management System, showing entities, attributes, and relationships. These diagrams are essential for database design and implementation.

Primary Entities: - **Admin:** System administrators and superusers - **Employee:** Organizational employees and staff members - **Candidate:** Job applicants and recruitment candidates - **Department:** Organizational departments and divisions - **Job:** Job positions and vacancies - **Attendance:** Daily attendance records - **Leave:** Leave requests and approvals - **Performance:** Performance reviews and evaluations - **Leave_Type:** Types of leave with policies - **Leave_Balance:** Employee leave balance tracking

Entity Attributes:

Admin Entity: - **Primary Key:** admin_id (INT, AUTO_INCREMENT) - **Attributes:** username, email, password, full_name, created_at, updated_at - **Constraints:** Unique username and email, encrypted password

Employee Entity: - **Primary Key:** employee_id (INT, AUTO_INCREMENT) - **Attributes:** employee_code, first_name, last_name, email, phone, department_id, role, salary, joining_date, status - **Foreign Keys:** department_id (references Department), candidate_id (references Candidate)

Candidate Entity: - **Primary Key:** candidate_id (INT, AUTO_INCREMENT) - **Attributes:** first_name, last_name, email, phone, job_id, resume_path, status, applied_date, notes - **Foreign Keys:** job_id (references Job)

Department Entity: - **Primary Key:** department_id (INT, AUTO_INCREMENT) - **Attributes:** name, description, created_at

Job Entity: - **Primary Key:** job_id (INT, AUTO_INCREMENT) - **Attributes:** title, department_id, description, requirements, salary_range, location, job_type, status - **Foreign Keys:** department_id (references Department)

Relationships: - **Department-Employee:** One-to-Many (One department has many employees) - **Job-Candidate:** One-to-Many (One job has many candidates) - **Employee-Attendance:** One-to-Many (One employee has many attendance records) - **Employee-Leave:** One-to-Many (One employee has many leave requests) - **Employee-Performance:** One-to-Many (One employee has many performance reviews) - **Leave_Type-Leave:** One-to-Many (One leave type has many leave requests) - **Employee-Leave_Balance:** One-to-Many (One employee has many leave balance records)

Relationship Cardinalities: - **One-to-One:** Admin-System_Configuration - **One-to-Many:** Department-Employee, Job-Candidate, Employee-Attendance - **Many-to-Many:** Employee-Skills, Employee-Training (if implemented)

Normalization: - **First Normal Form (1NF):** All attributes contain atomic values - **Second Normal Form (2NF):** No partial dependencies on composite keys - **Third Normal Form (3NF):** No transitive dependencies - **Boyce-Codd Normal Form (BCNF):** Advanced normalization for complex relationships

Integrity Constraints: - **Entity Integrity:** Primary keys cannot be null - **Referential Integrity:** Foreign keys must reference existing records - **Domain Integrity:** Data values must conform to defined domains - **User-Defined Integrity:** Business rule constraints

2.5 System Design Considerations

System design considerations encompass the architectural and technical decisions that guide the development of the HR Management System. These considerations ensure that the system meets requirements while maintaining quality, performance, and maintainability.

Design Principles: - **Modularity:** System divided into independent, interchangeable modules - **Scalability:** Architecture supports growth in users, data, and functionality - **Security:** Multi-layered security approach protecting sensitive data - **Performance:** Optimized for speed and efficiency under various load conditions - **Maintainability:** Clean code structure and comprehensive documentation - **Usability:** Intuitive interface design requiring minimal training - **Reliability:** Consistent operation with minimal downtime - **Flexibility:** Adaptable to changing business requirements

Architectural Considerations: - **Client-Server Architecture:** Separation of presentation and business logic - **Three-Tier Architecture:** Presentation, application, and data layers - **Service-Oriented Architecture:** Modular services with defined interfaces - **Microservices Approach:** Independent services for different HR functions - **API-First Design:** RESTful APIs for integration and extensibility

Database Design Considerations: - **Normalization:** Proper database normalization to reduce redundancy - **Indexing Strategy:** Optimized indexes for query performance - **Data Partitioning:** Logical separation of data for better management - **Backup Strategy:** Regular backups and recovery procedures - **Security Measures:** Data encryption and access controls - **Performance Optimization:** Query optimization and caching strategies

User Interface Design Considerations: - **Responsive Design:** Adaptation to different screen sizes and devices - **Accessibility:** Compliance with WCAG 2.1 accessibility standards - **User Experience:** Intuitive navigation and minimal cognitive load - **Consistency:** Uniform design patterns across

all modules - **Progressive Enhancement:** Basic functionality works on all devices - **Mobile-First Approach:** Design optimized for mobile devices

Security Design Considerations: - **Authentication:** Multi-factor authentication and secure session management - **Authorization:** Role-based access control with principle of least privilege - **Data Protection:** Encryption of sensitive data at rest and in transit - **Input Validation:** Comprehensive validation and sanitization of user input - **Audit Logging:** Complete audit trail of all system activities - **Secure Communication:** HTTPS/TLS for all data transmissions

Performance Design Considerations: - **Caching Strategy:** Multiple levels of caching for improved response times - **Database Optimization:** Query optimization and connection pooling - **Load Balancing:** Distribution of load across multiple servers - **Content Delivery:** CDN for static assets and improved loading times - **Asynchronous Processing:** Background processing for long-running tasks - **Resource Optimization:** Minification and compression of web assets

Integration Considerations: - **API Design:** RESTful APIs with proper versioning and documentation - **Third-Party Integration:** Support for email services, payment gateways - **Data Exchange:** Standard formats for data import and export - **Webhook Support:** Event-driven integration with external systems - **SSO Integration:** Single sign-on compatibility with corporate systems - **Legacy System Integration:** Compatibility with existing HR systems

Deployment Considerations: - **Cloud Deployment:** Support for major cloud platforms (AWS, Azure, GCP) - **Containerization:** Docker containers for consistent deployment - **Infrastructure as Code:** Automated infrastructure provisioning - **Monitoring and Logging:** Comprehensive system monitoring and log analysis - **Disaster Recovery:** Backup and recovery procedures for business continuity - **Scaling Strategy:** Horizontal and vertical scaling capabilities

Maintenance Considerations: - **Code Quality:** Standards, reviews, and automated testing - **Documentation:** Comprehensive technical and user documentation - **Version Control:** Git-based version control with branching strategy - **Continuous Integration:** Automated build and testing processes - **Deployment Automation:** Continuous deployment with rollback capabilities - **Monitoring and Alerting:** Proactive monitoring and alerting system

These design considerations provide a comprehensive framework for developing a robust, scalable, and maintainable HR Management System that meets all stakeholder requirements while adhering to industry best practices.

CHAPTER 3

HYPERTEXT PREPROCESSOR (PHP)

3.1 Introduction to PHP

PHP (Hypertext Preprocessor) is a server-side scripting language designed specifically for web development. Originally created by Rasmus Lerdorf in 1994, PHP has evolved into one of the most popular programming languages for web development, powering millions of websites and web applications worldwide. PHP is particularly well-suited for developing dynamic, database-driven web applications like the HR Management System.

PHP Definition and Purpose: PHP is an open-source, general-purpose scripting language that is especially suited for web development and can be embedded into HTML. The primary purpose of PHP is to create dynamic web pages that interact with databases, process user input, and generate customized content based on user requests and business logic.

Key Characteristics: - **Server-Side Execution:** PHP code is executed on the server, generating HTML that is sent to the client's browser - **Cross-Platform Compatibility:** PHP runs on various operating systems including Windows, Linux, macOS, and Unix - **Database Integration:** Excellent support for various database systems, particularly MySQL - **Web Server Integration:** Works seamlessly with Apache, Nginx, IIS, and other web servers - **Extensive Community Support:** Large developer community and comprehensive documentation

Why PHP for HR Management System: The selection of PHP for the HR Management System project is based on several compelling factors that make it an ideal choice for this type of application:

Rapid Development: - PHP's simple syntax and extensive built-in functions enable faster development cycles - Rich ecosystem of frameworks and libraries accelerates common development tasks - Extensive documentation and community resources reduce learning curve and debugging time

Database Integration: - Native support for MySQL through PDO and mysqli extensions - Excellent performance for database-driven applications - Built-in functions for common database operations

Cost-Effectiveness: - Open-source nature eliminates licensing costs - Wide availability of affordable hosting solutions - Large pool of PHP developers reduces development costs

Scalability and Performance: - Proven scalability for enterprise applications - Efficient memory management and execution speed - Support for caching mechanisms and optimization techniques

Security Features: - Built-in security functions and filters - Regular security updates and patches - Extensive security best practices and guidelines

3.2 History and Evolution of PHP

Understanding the historical development of PHP provides valuable context for its current capabilities and future directions. PHP has undergone significant evolution since its inception, adapting to changing web development needs and technological advancements.

Early Years (1994-1998): - **1994:** Rasmus Lerdorf creates Personal Home Page Tools (PHP Tools) to track visitors to his online resume - **1995:** PHP Tools evolves into Personal Home Page Construction Kit (PHP/FI) - **1997:** PHP/FI 2.0 released with C implementation and database support - **1998:** PHP 3.0 released with new parser, improved performance, and MySQL support

PHP 4 Era (2000-2004): - **2000:** PHP 4.0 released with Zend Engine, significantly improving performance and reliability - **2002:** PHP 4.1 introduces superglobals ($_GET, $_POST, $_SESSION) improving security - **2004:** PHP 4.3.0 released with improved object-oriented programming support - PHP 4 established PHP as a serious web development language with enterprise capabilities

PHP 5 Revolution (2004-2012): - **2004:** PHP 5.0 released with complete object-oriented programming model - **2005:** Introduction of PHP Data Objects (PDO) for database abstraction - **2006:** PHP 5.1 adds improved performance and exception handling - **2008:** PHP 5.3 introduces namespaces, late static binding, and closures - **2012:** PHP 5.4 released with traits, shorter array syntax, and built-in web server

PHP 7 Performance Era (2015-2018): - **2015:** PHP 7.0 released with major performance improvements (2x faster than PHP 5) - **2016:** PHP 7.1 adds nullable types and void return type - **2017:** PHP 7.2 introduces parameter type widening and Argon2 password hashing - **2018:** PHP 7.3 adds flexible heredoc/nowdoc syntax and improved garbage collection

Modern PHP (2018-Present): - **2018:** PHP 7.4 released with typed properties, arrow functions, and improved error handling - **2020:** PHP 8.0 introduces Just-In-Time compilation, union types, and attributes - **2021:** PHP 8.1 adds enums, fibers, and readonly properties - **2022:** PHP 8.2 introduces readonly classes, disjoint union types, and null/false types - **2023:** PHP 8.3 adds typed class constants and randomizer extension

Current Status and Future: PHP continues to evolve with regular releases focusing on: - Performance optimization and memory efficiency - Enhanced type safety and error handling - Modern language features and syntax improvements - Security enhancements and vulnerability fixes - Better integration with modern web technologies

3.3 PHP Features and Characteristics

PHP offers a comprehensive set of features that make it particularly suitable for developing complex web applications like the HR Management System. These features span from basic programming constructs to advanced capabilities that support enterprise-level application development.

Core Language Features:

Dynamic Typing: - Variables are dynamically typed, allowing flexibility in development - Type juggling and automatic type conversion - Optional strict typing for better code reliability - Support for type hints and return type declarations

Object-Oriented Programming: - Complete OOP support with classes, objects, inheritance, and polymorphism - Interfaces, abstract classes, and traits for code organization - Visibility modifiers (public, private, protected) for encapsulation - Magic methods for dynamic behavior and method overloading

Web-Specific Features: - Built-in session management for user authentication and state management - Cookie and header manipulation for web interactions - File upload handling with security considerations - Form processing and data validation capabilities

Database Integration: - Native support for multiple database systems - PDO (PHP Data Objects) for database abstraction - MySQLi extension for optimized MySQL operations - Built-in functions for common database operations

Error Handling: - Exception handling with try-catch blocks - Custom exception classes for specific error types - Error reporting levels and logging capabilities - Graceful error recovery and user-friendly messages

Security Features: - Input filtering and sanitization functions - Password hashing and encryption functions - CSRF protection mechanisms - SQL injection prevention through prepared statements

Performance Characteristics:

Execution Speed: - Compiled execution through Zend Engine - Opcode caching for improved performance - Memory-efficient variable handling - Optimized string and array operations

Scalability: - Support for multi-threaded web servers - Efficient database connection management - Caching mechanisms and optimization techniques - Load balancing compatibility

Memory Management: - Automatic memory management with garbage collection - Reference counting for efficient memory usage - Memory limit controls and monitoring - Resource cleanup and leak prevention

Development Features:

Rich Standard Library: - Extensive built-in functions for common tasks - String manipulation and regular expression support - Date/time handling and formatting - File system operations and directory management

Extensibility: - Support for custom extensions and modules - PEAR (PHP Extension and Application Repository) - Composer for package management - Framework ecosystem (Laravel, Symfony, CodeIgniter)

Debugging and Profiling: - Built-in error reporting and logging - Xdebug integration for advanced debugging - Performance profiling and optimization tools - Stack traces and error context

Web Server Integration: - Apache module (mod_php) for direct integration - FastCGI support for Nginx and other servers - Built-in web server for development - CLI (Command Line Interface) for scripting

Platform Independence: - Cross-platform compatibility (Windows, Linux, macOS) - Consistent behavior across different environments - Environment-specific configuration support - Deployment flexibility and portability

3.4 PHP Syntax and Structure

PHP syntax is designed to be familiar to programmers with C, Java, or Perl background while being accessible to beginners. The language structure supports both procedural and object-oriented programming paradigms, making it versatile for different development approaches.

Basic Syntax Elements:

PHP Tags: PHP code is embedded in HTML using special tags:

```php
<?php
    // PHP code goes here
?>
```

Variables and Data Types:

```php
<?php
    // Variable declaration
    $name = "John Doe";
    $age = 30;
    $salary = 50000.50;
    $is_active = true;

    // Type declarations (PHP 7.4+)
    function calculateBonus(float $salary): float {
        return $salary * 0.1;
    }
?>
```

Control Structures:

```php
<?php
    // Conditional statements
    if ($age >= 18) {
        echo "Adult";
    } elseif ($age >= 13) {
        echo "Teenager";
    } else {
        echo "Child";
    }

    // Loops
    for ($i = 0; $i < 10; $i++) {
        echo $i . "<br>";
    }

    foreach ($employees as $employee) {
        echo $employee['name'] . "<br>";
    }
?>
```

Functions and Methods:

```php
<?php
    // Function definition
    function calculateSalary(int $base, float $bonus = 0.0): float {
        return $base + $bonus;
    }

    // Function call
    $total_salary = calculateSalary(50000, 5000);

    // Arrow functions (PHP 7.4+)
    $multiply = fn($x, $y) => $x * $y;
?>
```

Object-Oriented Syntax:

```php
<?php
    class Employee {
        private string $name;
        private int $id;
        private float $salary;

        public function __construct(int $id, string $name, float $salary) {
            $this->id = $id;
            $this->name = $name;
            $this->salary = $salary;
        }
```

```php
        public function getAnnualSalary(): float {
            return $this->salary * 12;
        }

        public function setSalary(float $salary): void {
            $this->salary = $salary;
        }
    }

    $employee = new Employee(1, "John Doe", 50000.0);
    echo $employee->getAnnualSalary();
?>
```

Advanced Syntax Features:

Namespaces:

```php
<?php
    namespace HRMS\Models;

    class Employee {
        // Class implementation
    }

    namespace HRMS\Controllers;

    class EmployeeController {
        // Controller implementation
    }
?>
```

Traits:

```php
<?php
    trait Timestampable {
        public function getCreatedAt(): string {
            return $this->created_at;
        }
    }

    class Employee {
        use Timestampable;
        // Other methods
    }
?>
```

Error Handling:

```php
<?php
    try {
        $result = riskyOperation();
    } catch (DatabaseException $e) {
        error_log("Database error: " . $e->getMessage());
        return false;
    } catch (Exception $e) {
        error_log("General error: " . $e->getMessage());
        return false;
    }
?>
```

Web-Specific Syntax:

Session Management:

```php
<?php
    session_start();

    $_SESSION['user_id'] = $user_id;
    $_SESSION['login_time'] = time();

    // Check session
    if (isset($_SESSION['user_id'])) {
        echo "Welcome back, User " . $_SESSION['user_id'];
    }
?>
```

Form Processing:

```php
<?php
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $name = filter_input(INPUT_POST, 'name', FILTER_SANITIZE_STRING);
        $email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);

        if ($name && $email) {
            // Process valid data
        }
    }
?>
```

File Operations:

```php
<?php
    // File upload handling
    if (isset($_FILES['resume'])) {
        $file = $_FILES['resume'];
        $upload_dir = 'uploads/';
```

```
        if (move_uploaded_file($file['tmp_name'], $upload_dir . $file['name'])) {
            echo "File uploaded successfully";
        }
    }
?>
```

Best Practices: - Use proper opening and closing tags - Follow PSR-12 coding standards - Implement proper error handling - Use type hints for better code clarity - Apply security measures for user input - Use prepared statements for database queries - Implement proper exception handling - Follow object-oriented principles for complex applications

3.5 Variables and Data Types in PHP

PHP supports a wide range of data types and provides flexible variable handling that makes it suitable for developing dynamic web applications. Understanding PHP's type system is crucial for building robust and secure HR Management System.

Variable Declaration and Naming:

```
<?php
    // Variable rules
    $employee_name = "John Doe";        // Valid: starts with $
    $empAge = 30;                     // Valid: camelCase
    $_SESSION['user_id'] = 123;        // Valid: superglobal
    $department-sales = 50000;           // Invalid: hyphen not allowed

    // Variable assignment
    $salary = 75000.50;
    $is_active = true;
    $join_date = "2024-01-15";
?>
```

Scalar Data Types:

Boolean:

```
<?php
    $is_active = true;
    $has_access = false;

    // Type casting
    $result = (bool)$number;  // 0 becomes false, non-zero becomes true
?>
```

Integer:

```
<?php
    $employee_id = 12345;
```

```php
    $department_count = 5;
    $max_file_size = 10485760;  // 10MB in bytes

    // Integer limits (platform dependent)
    echo PHP_INT_MAX;  // Maximum integer value
    echo PHP_INT_MIN;  // Minimum integer value
?>
```

Float (Double):

```php
<?php
    $salary = 75000.50;
    $tax_rate = 0.15;
    $annual_bonus = 2500.75;

    // Precision considerations
    $result = 0.1 + 0.2;  // May not equal 0.3 due to floating point precision
?>
```

String:

```php
<?php
    $employee_name = "John Doe";
    $department = 'Human Resources';
    $description = "Employee with ID: {$employee_id}";

    // String functions
    $uppercase = strtoupper($employee_name);
    $substring = substr($description, 0, 10);
    $length = strlen($employee_name);

    // Heredoc and Nowdoc
    $email_template = <<<EMAIL
    Dear {$employee_name},

    Your leave request has been processed.
    EMAIL;
?>
```

Compound Data Types:

Array:

```php
<?php
    // Indexed array
    $departments = ["HR", "IT", "Finance", "Sales"];

    // Associative array
    $employee = [
```

```php
        "id" => 123,
        "name" => "John Doe",
        "email" => "john@company.com",
        "salary" => 75000.50
    ];

    // Multidimensional array
    $employees = [
        [
            "id" => 1,
            "name" => "John Doe",
            "skills" => ["PHP", "MySQL", "JavaScript"]
        ],
        [
            "id" => 2,
            "name" => "Jane Smith",
            "skills" => ["Python", "PostgreSQL", "React"]
        ]
    ];

    // Array operations
    array_push($departments, "Marketing");
    $found = in_array("HR", $departments);
    $keys = array_keys($employee);
?>
```

Object:

```php
<?php
    class Employee {
        public $id;
        public $name;
        public $department;

        public function __construct($id, $name, $department) {
            $this->id = $id;
            $this->name = $name;
            $this->department = $department;
        }

        public function getDisplayName() {
            return "{$this->name} ({$this->department})";
        }
    }

    $employee = new Employee(1, "John Doe", "IT");
    echo $employee->getDisplayName();
```

```
?>
```

Special Data Types:

NULL:

```
<?php
    $middle_name = null;  // No middle name
    $spouse = null;       // Not married

    // Checking for null
    if ($middle_name === null) {
        echo "No middle name provided";
    }

    // Null coalescing operator (PHP 7.0+)
    $display_name = $middle_name ?? $first_name;
?>
```

Resource:

```
<?php
    // Database connection
    $connection = mysqli_connect("localhost", "user", "pass", "database");

    // File handle
    $file_handle = fopen("employees.csv", "r");

    // Image resource
    $image = imagecreatefromjpeg("photo.jpg");

    // Always close resources
    mysqli_close($connection);
    fclose($file_handle);
    imagedestroy($image);
?>
```

Type Juggling and Casting:

```
<?php
    // Automatic type conversion
    $result = "10" + 5;        // Results in 15 (integer)
    $result = "10 apples" + 5;   // Results in 15 (integer)
    $result = "test" + 5;        // Results in 5 (integer)

    // Explicit type casting
    $string_value = (string)$number;
    $int_value = (int)$float_value;
    $bool_value = (bool)$string;
```

```php
    // Type checking
    if (is_int($value)) {
        // Handle integer
    } elseif (is_string($value)) {
        // Handle string
    }
?>
```

Type Hints and Declarations (PHP 7.0+):

```php
<?php
    // Function parameter and return type hints
    function calculateBonus(float $salary, int $performance_rating): float {
        $bonus_percentage = $performance_rating * 0.02;
        return $salary * $bonus_percentage;
    }

    // Class property types (PHP 7.4+)
    class Employee {
        public int $id;
        public string $name;
        public ?float $bonus = null;  // Nullable float
    }

    // Union types (PHP 8.0+)
    function processValue(int|string $value): void {
        // Implementation
    }
?>
```

Variable Scope:

```php
<?php
    $global_var = "Global";

    function testScope() {
        $local_var = "Local";
        global $global_var;  // Access global variable

        echo $local_var;    // Outputs: Local
        echo $global_var;    // Outputs: Global
    }

    // Static variables
    function counter() {
        static $count = 0;
        $count++;
```

```php
        return $count;
    }
?>
```

Superglobal Variables:

```php
<?php
    // $_GET - URL parameters
    $page = $_GET['page'] ?? 'dashboard';

    // $_POST - Form data
    $username = $_POST['username'] ?? '';

    // $_SESSION - Session data
    $_SESSION['user_id'] = $user_id;

    // $_SERVER - Server and execution environment
    $request_method = $_SERVER['REQUEST_METHOD'];
    $user_agent = $_SERVER['HTTP_USER_AGENT'];

    // $_FILES - Uploaded files
    $resume = $_FILES['resume'] ?? null;
?>
```

3.6 Control Structures

PHP provides comprehensive control structures that enable developers to create complex logic and flow control in their applications. These structures are essential for implementing business rules and user interactions in the HR Management System.

Conditional Statements:

If-Elseif-Else:

```php
<?php
    function checkEmployeeStatus($employee) {
        if ($employee['status'] === 'active') {
            return "Employee is currently active";
        } elseif ($employee['status'] === 'inactive') {
            return "Employee is inactive";
        } elseif ($employee['status'] === 'terminated') {
            return "Employee has been terminated";
        } else {
            return "Unknown employee status";
        }
    }
?>
```

Switch Statement:

```php
<?php
    function getDepartmentName($department_id) {
        switch ($department_id) {
            case 1:
                return "Human Resources";
            case 2:
                return "Information Technology";
            case 3:
                return "Finance";
            case 4:
                return "Sales";
            default:
                return "Unknown Department";
        }
    }

    // Switch with multiple cases (PHP 7.0+)
    function getLeaveTypeName($type) {
        switch ($type) {
            case 'sick':
            case 'medical':
                return "Sick Leave";
            case 'annual':
            case 'vacation':
                return "Annual Leave";
            default:
                return "Other Leave";
        }
    }
?>
```

Ternary Operator:

```php
<?php
    $status = $is_active ? 'Active' : 'Inactive';

    // Nested ternary (use with caution for readability)
    $message = $score >= 90 ? 'Excellent' :
               ($score >= 80 ? 'Good' :
               ($score >= 70 ? 'Average' : 'Poor'));

    // Null coalescing operator (PHP 7.0+)
    $display_name = $nickname ?? $first_name ?? 'Unknown';
?>
```

Match Expression (PHP 8.0+):

```php
<?php
    function getLeaveStatusColor($status) {
        return match($status) {
            'approved' => '#28a745',      // Green
            'pending' => '#ffc107',      // Yellow
            'rejected' => '#dc3545',      // Red
            default => '#6c757d'           // Gray
        };
    }
?>
```

Looping Structures:

For Loop:

```php
<?php
    // Generate employee IDs
    $employee_ids = [];
    for ($i = 1; $i <= 100; $i++) {
        $employee_ids[] = 'EMP' . str_pad($i, 4, '0', STR_PAD_LEFT);
    }

    // Process array with index
    for ($i = 0; $i < count($employees); $i++) {
        echo "Employee {$i}: {$employees[$i]['name']}\n";
    }
?>
```

Foreach Loop:

```php
<?php
    // Iterate over associative array
    foreach ($employees as $employee) {
        echo "Name: {$employee['name']}, Department: {$employee['department']}\n";
    }

    // Iterate with key-value
    foreach ($employee as $key => $value) {
        echo "$key: $value\n";
    }

    // Iterate by reference (PHP 5.0+)
    foreach ($employees as &$employee) {
        $employee['processed'] = true;
    }
    unset($employee);  // Break reference
?>
```

While Loop:

```php
<?php
    // Process database results
    $result = mysqli_query($connection, "SELECT * FROM employees");
    while ($row = mysqli_fetch_assoc($result)) {
        processEmployee($row);
    }

    // Input validation loop
    while (empty($username)) {
        echo "Please enter username: ";
        $username = trim(fgets(STDIN));
    }
?>
```

Do-While Loop:

```php
<?php
    do {
        echo "Enter employee salary (must be > 0): ";
        $salary = trim(fgets(STDIN));
    } while ($salary <= 0);
?>
```

Jump Statements:

Break and Continue:

```php
<?php
    foreach ($employees as $employee) {
        if ($employee['status'] === 'terminated') {
            continue;  // Skip terminated employees
        }

        if ($employee['id'] === $target_id) {
            echo "Found employee: {$employee['name']}";
            break;  // Exit loop
        }
    }

    // Break with level (nested loops)
    foreach ($departments as $department) {
        foreach ($department['employees'] as $employee) {
            if ($employee['id'] === $search_id) {
                echo "Found in {$department['name']}";
                break 2;  // Break out of both loops
            }
        }
    }
```

```
?>
```

Goto (Use Sparingly):

```php
<?php
    function processLeaveRequest($request) {
        if (empty($request['employee_id'])) {
            goto validation_error;
        }

        if (empty($request['start_date'])) {
            goto validation_error;
        }

        // Process valid request
        return true;

        validation_error:
        return false;
    }
?>
```

Include and Require:

```php
<?php
    // Include files (continues on error)
    include 'config/database.php';
    include_once 'helpers/functions.php';

    // Require files (stops on error)
    require 'models/Employee.php';
    require_once 'config/constants.php';

    // Conditional includes
    if ($environment === 'development') {
        include 'debug/debug.php';
    }
?>
```

Declare Statements:

```php
<?php
    // Strict typing
    declare(strict_types=1);

    function calculateSalary(float $base, float $bonus): float {
        return $base + $bonus;
    }
```

```php
    // Ticks (for profiling)
    declare(ticks=1);
    function profiler() {
        // Profile code execution
    }
    register_tick_function('profiler');

    // Encoding
    declare(encoding='UTF-8');
?>
```

Error Control Operators:

```php
<?php
    // Suppress warnings and errors
    $connection = @mysqli_connect($host, $user, $pass, $db);

    if (!$connection) {
        die("Database connection failed");
    }

    // Custom error handler
    set_error_handler(function($severity, $message, $file, $line) {
        error_log("Error: $message in $file on line $line");
    });
?>
```

Best Practices for Control Structures: - Use meaningful variable and function names - Keep conditional logic simple and readable - Avoid deep nesting of control structures - Use early returns to reduce complexity - Prefer foreach over for when iterating arrays - Use strict comparison operators (===, !==) - Implement proper error handling - Consider performance implications of different structures - Document complex logic with comments

3.7 Functions and Object-Oriented Programming

PHP's support for functions and object-oriented programming (OOP) enables developers to create modular, maintainable, and scalable applications. The HR Management System leverages these features extensively to organize code logically and promote reusability.

Functions:

Basic Function Definition:

```php
<?php
    // Simple function
    function greetEmployee($name) {
        return "Hello, $name!";
```

```php
    }

    // Function with parameters and default values
    function calculateLeaveBalance($total_days, $used_days = 0) {
        return $total_days - $used_days;
    }

    // Function with type hints
    function formatSalary(float $salary, string $currency = 'USD'): string {
        return number_format($salary, 2) . ' ' . $currency;
    }
?>
```

Variable Functions:

```php
<?php
    function add($a, $b) { return $a + $b; }
    function multiply($a, $b) { return $a * $b; }

    $operation = 'add';
    $result = $operation(5, 3);  // Calls add() function

    $operation = 'multiply';
    $result = $operation(5, 3);  // Calls multiply() function
?>
```

Anonymous Functions and Closures:

```php
<?php
    // Anonymous function
    $calculateBonus = function($salary, $percentage) {
        return $salary * ($percentage / 100);
    };

    // Closure with use of external variables
    function createEmployeeFormatter($company_name) {
        return function($employee) use ($company_name) {
            return "{$employee['name']} - {$company_name}";
        };
    }

    $formatter = createEmployeeFormatter("Tech Corp");
    echo $formatter(['name' => 'John Doe']);

    // Arrow functions (PHP 7.4+)
    $square = fn($x) => $x * $x;
    $addTax = fn($amount, $rate) => $amount * (1 + $rate);
?>
```

Generator Functions:

```php
<?php
    function generateEmployeeIds($start, $end) {
        for ($i = $start; $i <= $end; $i++) {
            yield 'EMP' . str_pad($i, 4, '0', STR_PAD_LEFT);
        }
    }

    foreach (generateEmployeeIds(1, 10) as $id) {
        echo $id . "\n";
    }
?>
```

Object-Oriented Programming:

Class Definition:

```php
<?php
    class Employee {
        // Properties
        private int $id;
        private string $firstName;
        private string $lastName;
        private string $email;
        private float $salary;
        private DateTime $hireDate;

        // Constructor
        public function __construct(int $id, string $firstName, string $lastName,
                                    string $email, float $salary) {
            $this->id = $id;
            $this->firstName = $firstName;
            $this->lastName = $lastName;
            $this->email = $email;
            $this->salary = $salary;
            $this->hireDate = new DateTime();
        }

        // Getter methods
        public function getId(): int {
            return $this->id;
        }

        public function getFullName(): string {
            return $this->firstName . ' ' . $this->lastName;
        }
```

```php
        public function getEmail(): string {
            return $this->email;
        }

        public function getSalary(): float {
            return $this->salary;
        }

        // Setter methods
        public function setSalary(float $salary): void {
            $this->salary = $salary;
        }

        // Business logic methods
        public function getAnnualSalary(): float {
            return $this->salary * 12;
        }

        public function getYearsOfService(): int {
            $now = new DateTime();
            return $now->diff($this->hireDate)->y;
        }

        // Magic methods
        public function __toString(): string {
            return "Employee: {$this->getFullName()} (ID: {$this->id})";
        }
    }
?>
```

Inheritance:

```php
<?php
    class Person {
        protected string $firstName;
        protected string $lastName;
        protected string $email;

        public function __construct(string $firstName, string $lastName, string $email) {
            $this->firstName = $firstName;
            $this->lastName = $lastName;
            $this->email = $email;
        }

        public function getFullName(): string {
            return $this->firstName . ' ' . $this->lastName;
        }
```

```php
    }

    class Employee extends Person {
        private float $salary;
        private string $department;

        public function __construct(string $firstName, string $lastName, string $email,
                                    float $salary, string $department) {
            parent::__construct($firstName, $lastName, $email);
            $this->salary = $salary;
            $this->department = $department;
        }

        public function getSalary(): float {
            return $this->salary;
        }

        public function getDepartment(): string {
            return $this->department;
        }

        public function getEmployeeInfo(): string {
            return parent::getFullName() . " - {$this->department} - \${$this->salary}";
        }
    }
?>
```

Interfaces:

```php
<?php
    interface DatabaseInterface {
        public function connect(string $host, string $user, string $password, string $databa
        public function query(string $sql): array;
        public function disconnect(): void;
    }

    interface EmployeeRepositoryInterface {
        public function findById(int $id): ?Employee;
        public function save(Employee $employee): bool;
        public function delete(int $id): bool;
        public function findAll(): array;
    }
?>
```

Abstract Classes:

```php
<?php
    abstract class ReportGenerator {
```

```php
        abstract protected function getData(): array;
        abstract protected function formatData(array $data): string;

        public function generateReport(): string {
            $data = $this->getData();
            return $this->formatData($data);
        }

        protected function getHeader(): string {
            return "HR Management System Report\n";
        }

        protected function getFooter(): string {
            return "\nGenerated on: " . date('Y-m-d H:i:s');
        }
    }

    class EmployeeReport extends ReportGenerator {
        protected function getData(): array {
            // Fetch employee data from database
            return [];
        }

        protected function formatData(array $data): string {
            $output = $this->getHeader();
            foreach ($data as $employee) {
                $output .= $employee['name'] . " - " . $employee['department'] . "\n";
            }
            $output .= $this->getFooter();
            return $output;
        }
    }
?>
```

Traits:

```php
<?php
    trait Timestampable {
        private DateTime $createdAt;
        private DateTime $updatedAt;

        public function setCreatedAt(DateTime $date): void {
            $this->createdAt = $date;
        }

        public function setUpdatedAt(DateTime $date): void {
            $this->updatedAt = $date;
```

```php
        }

        public function getCreatedAt(): DateTime {
            return $this->createdAt;
        }

        public function getUpdatedAt(): DateTime {
            return $this->updatedAt;
        }
    }

    trait SoftDeletable {
        private ?DateTime $deletedAt = null;

        public function delete(): void {
            $this->deletedAt = new DateTime();
        }

        public function isDeleted(): bool {
            return $this->deletedAt !== null;
        }
    }

    class Employee {
        use Timestampable, SoftDeletable;
        // Other properties and methods
    }
?>
```

Static Members:

```php
<?php
    class Employee {
        private static int $count = 0;
        private static array $departments = [];

        public function __construct() {
            self::$count++;
        }

        public static function getCount(): int {
            return self::$count;
        }

        public static function addDepartment(string $department): void {
            if (!in_array($department, self::$departments)) {
                self::$departments[] = $department;
```

```php
            }
        }

        public static function getDepartments(): array {
            return self::$departments;
        }

        public static function findByEmail(string $email): ?Employee {
            // Database lookup logic
            return null;
        }
    }
?>
```

Namespaces and Autoloading:

```php
<?php
    // File: src/Models/Employee.php
    namespace HRMS\Models;

    class Employee {
        // Class implementation
    }

    // File: src/Controllers/EmployeeController.php
    namespace HRMS\Controllers;

    use HRMS\Models\Employee;

    class EmployeeController {
        public function show(int $id): void {
            $employee = new Employee($id);
            // Display employee
        }
    }

    // Autoloading (composer.json)
    // {
    //     "autoload": {
    //         "psr-4": {
    //             "HRMS\\": "src/"
    //         }
    //     }
    // }
?>
```

Exception Handling:

```php
<?php
    class HRException extends Exception {
        private array $context;

        public function __construct(string $message, array $context = [], int $code = 0) {
            parent::__construct($message, $code);
            $this->context = $context;
        }

        public function getContext(): array {
            return $this->context;
        }
    }

    class EmployeeNotFoundException extends HRException {
        public function __construct(int $employeeId) {
            parent::__construct("Employee with ID {$employeeId} not found", ['employee_id' =
        }
    }

    class EmployeeService {
        public function findEmployee(int $id): Employee {
            $employee = $this->repository->findById($id);

            if ($employee === null) {
                throw new EmployeeNotFoundException($id);
            }

            return $employee;
        }
    }
?>
```

Design Patterns in PHP:

Singleton Pattern:

```php
<?php
    class DatabaseConnection {
        private static ?DatabaseConnection $instance = null;
        private PDO $connection;

        private function __construct() {
            $this->connection = new PDO(
                "mysql:host=localhost;dbname=hrms",
                "username",
                "password"
```

```
            );
        }

        public static function getInstance(): DatabaseConnection {
            if (self::$instance === null) {
                self::$instance = new self();
            }
            return self::$instance;
        }

        public function getConnection(): PDO {
            return $this->connection;
        }

        private function __clone() {}
        public function __wakeup() {}
    }
?>
```

Factory Pattern:

```
<?php
    interface ReportFactoryInterface {
        public function createReport(string $type): ReportGenerator;
    }

    class ReportFactory implements ReportFactoryInterface {
        public function createReport(string $type): ReportGenerator {
            switch ($type) {
                case 'employee':
                    return new EmployeeReport();
                case 'attendance':
                    return new AttendanceReport();
                case 'leave':
                    return new LeaveReport();
                default:
                    throw new InvalidArgumentException("Unknown report type: {$type}");
            }
        }
    }
?>
```

These OOP features and patterns enable the development of a well-structured, maintainable, and scalable HR Management System that follows industry best practices and design principles.

CHAPTER 4

MYSQL

4.1 Introduction to MySQL

MySQL is an open-source relational database management system (RDBMS) that has become one of the most popular database solutions for web applications worldwide. Originally developed by Michael "Monty" Widenius, David Axmark, and Allan Larsson in 1994, MySQL is now owned by Oracle Corporation and continues to be a cornerstone of the LAMP (Linux, Apache, MySQL, PHP) stack.

MySQL Definition and Purpose: MySQL is a database server that provides multi-user access to numerous databases. It is a relational database that stores data in separate tables rather than putting all data in one big storeroom. This structure enables users to maintain relationships between different data elements and provides fast access, flexibility, and reliable management of large amounts of data.

Key Characteristics: - **Relational Model:** Data is organized in tables with relationships defined through foreign keys - **SQL Standard:** Uses standard SQL for data manipulation and querying - **Client-Server Architecture:** Database server handles multiple client connections - **Multi-Threading:** Supports multiple simultaneous connections and queries - **Cross-Platform:** Runs on various operating systems including Windows, Linux, and macOS - **Open Source:** Free to use and modify with active community development

Why MySQL for HR Management System: The selection of MySQL for the HR Management System is based on several compelling factors that make it an ideal choice for this type of application:

Performance and Scalability: - Optimized for read-heavy operations typical in HR applications - Supports large datasets with millions of records - Efficient indexing strategies for fast query performance - Proven scalability for enterprise-level applications

Integration with PHP: - Native support through PHP's PDO and mysqli extensions - Extensive documentation and community support - Mature, stable integration with minimal configuration - Optimized specifically for web application workloads

Cost-Effectiveness: - Open-source license eliminates database licensing costs - Wide availability of affordable hosting solutions - Lower total cost of ownership compared to commercial alternatives - Large pool of experienced MySQL administrators

Reliability and Maturity: - Proven track record in production environments - Regular security updates and bug fixes - Extensive testing and quality assurance - Strong community and commercial support options

Features Suited for HR Applications: - ACID compliance for transaction integrity - Support for complex queries and joins - Robust user permission system - Excellent backup and recovery capabilities

4.2 History and Development of MySQL

Understanding MySQL's historical development provides valuable context for its current capabilities and evolution. MySQL has undergone significant changes since its inception, adapting to growing demands of web applications and enterprise data management.

Early Years (1994-1995): - **1994:** MySQL 1.0 released by Michael Widenius for personal use - **1995:** MySQL 3.11 released with initial SQL support and basic features - Introduction of client-server architecture - Basic table creation and data manipulation capabilities - Limited to simple SELECT, INSERT, UPDATE, DELETE operations

MySQL AB Era (1996-2008): - **1996:** MySQL AB founded to commercialize MySQL development - **1996:** Version 3.20 released with improved SQL support - **1998:** MySQL 3.21 adds new functions and better performance - **2000:** MySQL 3.23 introduces InnoDB storage engine - **2003:** MySQL 4.0 released with major performance improvements - **2004:** MySQL 4.1 adds subqueries, UNION operations, and better character set support

Oracle Acquisition (2008-2010): - **2008:** Sun Microsystems acquires MySQL AB for $1 billion - **2008:** MySQL 5.1 released with stored procedures, triggers, and views - **2009:** MySQL 5.4 adds partitioning, event scheduling, and row-based replication - **2010:** Oracle Corporation acquires Sun Microsystems

MySQL 5.5+ Era (2010-2017): - **2010:** MySQL 5.5 released with semi-synchronous replication and performance schema - **2011:** MySQL 5.6 introduces InnoDB improvements and better performance - **2012:** MySQL 5.6 adds NoSQL access through memcached API and online DDL operations - **2013:** MySQL 5.7 released with enhanced performance and new features - **2015:** MySQL 5.7 focuses on security improvements and JSON support

MySQL 8.0 Revolution (2018-Present): - **2018:** MySQL 8.0 released with major architectural changes - Introduction of MySQL Document Store for NoSQL capabilities - Window functions for advanced analytics - Common Table Expressions for improved query performance - Enhanced security with default authentication methods - Improved InnoDB storage engine with better performance - **2019:** MySQL 8.0.18 focuses on stability and bug fixes - **2020:** MySQL 8.0.19 adds minor features and improvements - **2021:** MySQL 8.0.26 focuses on security patches and performance - **2022:** MySQL 8.0.30 introduces new features and optimizations - **2023:** MySQL 8.0.33 focuses on stability and performance improvements

Current Status and Future: MySQL continues to evolve with regular releases focusing on: - Performance optimization and query execution improvements

- Enhanced security features and vulnerability fixes - Better integration with cloud platforms and containerization - Improved JSON and document store capabilities - Enhanced monitoring and management tools - Better support for high-availability and clustering

4.3 MySQL Architecture

MySQL architecture is designed to provide high performance, reliability, and scalability for database-driven applications. Understanding this architecture is crucial for optimizing database design and performance in the HR Management System.

Core Components:

MySQL Server: - **Query Engine:** Parses SQL queries and generates execution plans - **Storage Engines:** Pluggable storage engine architecture (InnoDB, MyISAM, Memory) - **Connection Manager:** Handles client connections and authentication - **Cache Manager:** Manages various cache layers for performance - **Replication Engine:** Supports master-slave and group replication

Storage Engine Architecture:

```sql
-- InnoDB (Default and Recommended for HRMS)
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
) ENGINE=InnoDB;

-- MyISAM (Legacy, limited features)
CREATE TABLE legacy_data (
    id INT PRIMARY KEY AUTO_INCREMENT,
    data TEXT
) ENGINE=MyISAM;
```

Query Execution Pipeline: 1. **Parsing:** SQL query syntax validation and parsing 2. **Optimization:** Query optimization and execution plan generation 3. **Execution:** Plan execution with storage engine interaction 4. **Caching:** Result caching for repeated queries 5. **Return:** Result set formation and client response

Connection Architecture: - **Connection Pooling:** Efficient management of database connections - **Thread Management:** Multi-threaded connection handling - **Authentication:** Multiple authentication methods (native, PAM, LDAP) - **SSL/TLS Support:** Encrypted client-server communication - **Connection Limits:** Configurable connection and resource limits

Memory Architecture: - **Buffer Pool:** InnoDB buffer pool for caching data and indexes - **Log Buffer:** Transaction log buffer for write operations - **Query**

Cache: Cache for frequently executed SELECT queries - **Key Buffer:** MyISAM key buffer for index caching - **Sort Buffer:** Memory for sorting and grouping operations

Storage Engine Comparison:

InnoDB (Recommended for HRMS): - ACID compliant for transaction integrity - Row-level locking for better concurrency - Foreign key constraints and referential integrity - Crash recovery and automatic repair - Supports large data volumes and high concurrency

MyISAM (Legacy): - Table-level locking (limited concurrency) - Faster for read-heavy, write-light workloads - No transaction support - Full-text search capabilities - Simpler structure, less memory usage

Memory Storage Engine: - In-memory storage for maximum speed - Volatile (data lost on server restart) - Useful for temporary tables and caching - No transaction support - Fixed-size rows

Replication Architecture: - **Statement-Based Replication:** Replicates SQL statements - **Row-Based Replication:** Replicates individual row changes - **Mixed-Mode Replication:** Automatically switches between modes - **Semi-Synchronous Replication:** Balances performance and data safety - **Group Replication:** Multi-master replication for high availability

Security Architecture: - **Authentication Plugins:** Pluggable authentication system - **Access Control:** Granular user and permission management - **Encryption:** Data-at-rest encryption and SSL/TLS for transmission - **Audit Logging:** Comprehensive logging of database activities - **Password Validation:** Strong password policies and validation

Performance Schema: - **Instrumentation:** Detailed performance monitoring and analysis - **Query Analysis:** Query execution statistics and performance data - **Index Usage:** Index efficiency analysis and optimization suggestions - **Table I/O:** Table and index access statistics - **Wait Events:** Resource contention and bottleneck identification

4.4 MySQL Features and Capabilities

MySQL offers a comprehensive set of features that make it particularly suitable for developing complex data-driven applications like the HR Management System. These features span from basic data storage to advanced capabilities that support enterprise-level application requirements.

Core Database Features:

Data Types Support:

```sql
-- Numeric types
CREATE TABLE employee_data (
    id INT PRIMARY KEY AUTO_INCREMENT,
```

```sql
    salary DECIMAL(10,2),
    bonus FLOAT,
    hours_worked INT,
    performance_rating TINYINT
);

-- String types
CREATE TABLE employee_profiles (
    id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    bio TEXT,
    skills JSON
);

-- Date/Time types
CREATE TABLE attendance_records (
    id INT PRIMARY KEY,
    employee_id INT,
    check_in TIME,
    check_out TIME,
    date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Indexing and Performance:

```sql
-- Primary key indexes (automatic)
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(100) UNIQUE
);

-- Secondary indexes for performance
CREATE INDEX idx_employee_name ON employees(last_name, first_name);
CREATE INDEX idx_employee_department ON employees(department_id);
CREATE INDEX idx_employee_status ON employees(status);

-- Composite indexes
CREATE INDEX idx_employee_dept_status ON employees(department_id, status);

-- Full-text indexes for search
CREATE FULLTEXT INDEX idx_resume_text ON resumes(resume_content);
```

Relationships and Constraints:

```sql
-- Foreign key constraints
```

```sql
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    department_id INT,
    manager_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE SET NULL,
    FOREIGN KEY (manager_id) REFERENCES employees(id) ON DELETE SET NULL
);

-- Check constraints
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(100) NOT NULL,
    salary DECIMAL(10,2) CHECK (salary >= 0),
    CONSTRAINT chk_email_format CHECK (email LIKE '%@%.%')
);

-- Unique constraints
ALTER TABLE employees ADD CONSTRAINT uk_email UNIQUE (email);
ALTER TABLE employees ADD CONSTRAINT uk_employee_code UNIQUE (employee_code);
```

Advanced SQL Features:

Joins and Subqueries:

```sql
-- Complex joins for reporting
SELECT
    e.first_name, e.last_name, d.name as department,
    COUNT(a.id) as attendance_days,
    AVG(pr.rating) as avg_performance
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id
LEFT JOIN attendance a ON e.id = a.employee_id
LEFT JOIN performance_reviews pr ON e.id = pr.employee_id
WHERE e.status = 'active'
GROUP BY e.id, e.first_name, e.last_name, d.name
ORDER BY e.last_name;

-- Subqueries for filtering
SELECT * FROM employees
WHERE id IN (
    SELECT employee_id FROM attendance
    WHERE date >= DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY)
    GROUP BY employee_id
    HAVING COUNT(*) >= 20
);
```

Window Functions (MySQL 8.0+):

```sql
-- Running totals and averages
SELECT
    employee_id,
    date,
    hours_worked,
    SUM(hours_worked) OVER (PARTITION BY employee_id ORDER BY date) as running_total,
    AVG(hours_worked) OVER (PARTITION BY employee_id ORDER BY date ROWS BETWEEN 6 PRECEDING
FROM attendance
ORDER BY employee_id, date;

-- Row numbering and ranking
SELECT
    e.*,
    ROW_NUMBER() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) as dept_rank,
    DENSE_RANK() OVER (ORDER BY e.hire_date DESC) as overall_rank
FROM employees e;
```

JSON Support (MySQL 5.7+):

```sql
-- JSON data storage
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    skills JSON,
    metadata JSON
);

-- JSON functions for querying
SELECT
    name,
    JSON_EXTRACT(skills, '$.technical[0]') as primary_skill,
    JSON_LENGTH(skills) as skill_count,
    JSON_CONTAINS(metadata, '"key":"value"') as has_special_flag
FROM employees
WHERE JSON_EXTRACT(skills, '$.language') = '"PHP"';
```

Stored Programs:

```sql
-- Stored procedures
DELIMITER //
CREATE PROCEDURE calculate_employee_bonus(IN employee_id INT, IN performance_rating DECIMAL(
BEGIN
    DECLARE base_salary DECIMAL(10,2);
    DECLARE bonus_amount DECIMAL(10,2);

    SELECT salary INTO base_salary FROM employees WHERE id = employee_id;

    SET bonus_amount = base_salary * (performance_rating / 100);
```

```sql
    UPDATE employees
    SET bonus = bonus_amount,
        last_bonus_calculated = CURRENT_TIMESTAMP
    WHERE id = employee_id;

    SELECT bonus_amount as calculated_bonus;
END //
DELIMITER ;

-- Triggers for automatic updates
CREATE TRIGGER update_leave_balance
AFTER INSERT ON leaves
FOR EACH ROW
BEGIN
    UPDATE leave_balance
    SET used = used + NEW.total_days
    WHERE employee_id = NEW.employee_id
    AND leave_type_id = NEW.leave_type_id
    AND year = YEAR(NEW.start_date);
END;
```

Views and Virtual Tables:

```sql
-- Simplified views for complex queries
CREATE VIEW employee_summary AS
SELECT
    e.id,
    e.first_name,
    e.last_name,
    e.email,
    d.name as department,
    COUNT(a.id) as total_attendance_days,
    SUM(CASE WHEN a.status = 'present' THEN 1 ELSE 0 END) as present_days,
    AVG(pr.rating) as avg_performance_rating
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id
LEFT JOIN attendance a ON e.id = a.employee_id
LEFT JOIN performance_reviews pr ON e.id = pr.employee_id
GROUP BY e.id, e.first_name, e.last_name, e.email, d.name;

-- Updatable views for data modification
CREATE VIEW active_employees AS
SELECT id, first_name, last_name, email, department_id
FROM employees
WHERE status = 'active'
WITH CHECK OPTION;
```

Transaction Support:

```sql
-- ACID transactions for data integrity
START TRANSACTION;

-- Update employee salary
UPDATE employees SET salary = 55000.00 WHERE id = 123;

-- Insert salary history record
INSERT INTO salary_history (employee_id, old_salary, new_salary, change_date)
VALUES (123, 50000.00, 55000.00, CURRENT_DATE);

-- Update department budget
UPDATE departments SET total_salary = total_salary + 5000.00 WHERE id =
    (SELECT department_id FROM employees WHERE id = 123);

COMMIT;

-- Rollback on error
ROLLBACK;
```

Partitioning and Sharding:

```sql
-- Range partitioning for large tables
CREATE TABLE attendance (
    id INT,
    employee_id INT,
    date DATE,
    status VARCHAR(20),
    PRIMARY KEY (id, date)
)
PARTITION BY RANGE (YEAR(date)) (
    PARTITION p2020 VALUES LESS THAN (2021),
    PARTITION p2021 VALUES LESS THAN (2022),
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION pmax VALUES LESS THAN MAXVALUE
);

-- List partitioning for specific values
CREATE TABLE employees (
    id INT,
    name VARCHAR(100),
    department VARCHAR(50),
    PRIMARY KEY (id)
)
PARTITION BY LIST (department) (
    PARTITION p_hr VALUES IN ('HR', 'Administration'),
```

```sql
    PARTITION p_it VALUES IN ('IT', 'Development'),
    PARTITION p_sales VALUES IN ('Sales', 'Marketing'),
    PARTITION p_other VALUES IN (DEFAULT)
);
```

4.5 Data Types in MySQL

MySQL provides a comprehensive set of data types that allow developers to store various kinds of information efficiently and appropriately. Understanding these data types is crucial for designing an optimal database schema for the HR Management System.

Numeric Data Types:

Integer Types:

```sql
-- TINYINT (-128 to 127 or 0 to 255)
CREATE TABLE settings (
    max_leave_days TINYINT UNSIGNED,  -- 0 to 255 days
    is_active BOOLEAN  -- Alias for TINYINT(1)
);

-- SMALLINT (-32768 to 32767 or 0 to 65535)
CREATE TABLE departments (
    id SMALLINT PRIMARY KEY AUTO_INCREMENT,  -- Up to 65,535 departments
    employee_count SMALLINT UNSIGNED  -- Up to 65,535 employees
);

-- INT (-2 billion to 2 billion or 0 to 4 billion)
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,  -- Primary key
    employee_code INT,  -- Employee numbers
    department_id INT  -- Foreign key references
);

-- BIGINT (-9 quintillion to 9 quintillion)
CREATE TABLE audit_logs (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  -- Large audit tables
    timestamp BIGINT  -- Unix timestamps
);
```

Floating-Point Types:

```sql
-- FLOAT (Single precision)
CREATE TABLE performance_metrics (
    employee_id INT,
    efficiency_score FLOAT,  -- Approximate values
    satisfaction_rating FLOAT
```

```sql
);

-- DOUBLE/REAL (Double precision)
CREATE TABLE salary_data (
    employee_id INT,
    base_salary DECIMAL(10,2),  -- Fixed-point for money
    bonus_percentage DECIMAL(5,2),  -- Precise percentages
    annual_salary DECIMAL(12,2)  -- Large monetary values
);
```

String Data Types:

Fixed-Length Strings:

```sql
-- CHAR (Fixed length, padded with spaces)
CREATE TABLE employee_codes (
    id INT PRIMARY KEY,
    code CHAR(10)  -- Always 10 characters, padded if shorter
);

-- BINARY (Fixed length binary)
CREATE TABLE file_hashes (
    id INT PRIMARY KEY,
    hash BINARY(32)  -- MD5 hashes
);
```

Variable-Length Strings:

```sql
-- VARCHAR (Variable length, 0-65,535 characters)
CREATE TABLE employees (
    id INT PRIMARY KEY,
    first_name VARCHAR(50),    -- First names up to 50 chars
    last_name VARCHAR(50),     -- Last names up to 50 chars
    email VARCHAR(100),       -- Email addresses up to 100 chars
    phone VARCHAR(20)         -- Phone numbers up to 20 chars
);

-- TEXT (Variable length, up to 65,535 characters)
CREATE TABLE employee_profiles (
    id INT PRIMARY KEY,
    bio TEXT,               -- Long biographies
    notes TEXT               -- Administrative notes
);

-- LONGTEXT (Very long text, up to 4GB)
CREATE TABLE document_storage (
    id INT PRIMARY KEY,
    content LONGTEXT  -- Large documents
```

```
);
```

Date and Time Types:

```
-- DATE (YYYY-MM-DD format)
CREATE TABLE employees (
    id INT PRIMARY KEY,
    birth_date DATE,           -- Employee birth dates
    hire_date DATE,            -- Employment start dates
    termination_date DATE       -- Employment end dates
);

-- TIME (HH:MM:SS format)
CREATE TABLE attendance (
    id INT PRIMARY KEY,
    employee_id INT,
    check_in TIME,             -- Clock-in times
    check_out TIME,            -- Clock-out times
    break_duration TIME          -- Break times
);

-- DATETIME (YYYY-MM-DD HH:MM:SS format)
CREATE TABLE audit_trail (
    id INT PRIMARY KEY,
    action_time DATETIME,      -- Precise timestamps
    description VARCHAR(255)
);

-- TIMESTAMP (Automatic timestamping)
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  -- Record creation
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  -- Last upd
);
```

Special Data Types:

JSON Data Type (MySQL 5.7+):

```
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    skills JSON,               -- Flexible skill storage
    preferences JSON,          -- User preferences
    metadata JSON             -- Extensible metadata
);
```

```sql
-- JSON functions for querying
SELECT
    name,
    JSON_EXTRACT(skills, '$.languages[0]') as primary_language,
    JSON_EXTRACT(skills, '$.experience[0].years') as experience_years,
    JSON_CONTAINS(preferences, '"notifications": true') as wants_notifications
FROM employees;
```

ENUM and SET Types:

```sql
-- ENUM (Single value from predefined list)
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    status ENUM('active', 'inactive', 'terminated', 'on_leave'),  -- Employee status
    employment_type ENUM('full_time', 'part_time', 'contract', 'intern')   -- Employment typ
);

-- SET (Multiple values from predefined list)
CREATE TABLE employee_permissions (
    id INT PRIMARY KEY,
    employee_id INT,
    permissions SET('read', 'write', 'delete', 'admin', 'report')  -- Multiple permissions
);
```

Spatial Data Types:

```sql
-- For location-based features (if needed)
CREATE TABLE office_locations (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    coordinates POINT,        -- Geographic coordinates
    region POLYGON           -- Office regions
);
```

Data Type Selection Best Practices: - Use the smallest appropriate data type to save storage space - Consider future growth when choosing data types - Use UNSIGNED for non-negative values - Prefer VARCHAR over CHAR for variable-length strings - Use appropriate date/time types for temporal data - Consider using JSON for flexible, schema-less data - Use ENUM for fields with limited, predefined values

4.6 MySQL Commands and Syntax

MySQL uses standard SQL (Structured Query Language) with some extensions and optimizations. Understanding MySQL's specific commands and syntax is essential for effectively managing the HR Management System database.

Data Definition Language (DDL):

CREATE TABLE:

```sql
-- Basic table creation
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    department_id INT,
    hire_date DATE NOT NULL,
    salary DECIMAL(10,2) DEFAULT 0.00,
    status ENUM('active', 'inactive', 'terminated') DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table with specific storage engine
CREATE TABLE attendance_logs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT NOT NULL,
    log_date DATE NOT NULL,
    check_in TIME,
    check_out TIME,
    notes TEXT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Temporary table
CREATE TEMPORARY TABLE temp_attendance AS
SELECT employee_id, COUNT(*) as days_present
FROM attendance
WHERE date >= '2024-01-01'
GROUP BY employee_id;
```

ALTER TABLE:

```sql
-- Add new column
ALTER TABLE employees
ADD COLUMN middle_name VARCHAR(50) AFTER last_name;

-- Modify column definition
ALTER TABLE employees
MODIFY COLUMN salary DECIMAL(12,2);

-- Drop column
ALTER TABLE employees
DROP COLUMN middle_name;

-- Add index
```

```sql
ALTER TABLE employees
ADD INDEX idx_employee_name (last_name, first_name);

-- Add foreign key
ALTER TABLE employees
ADD CONSTRAINT fk_employee_department
FOREIGN KEY (department_id) REFERENCES departments(id)
ON DELETE SET NULL;

-- Rename table
ALTER TABLE old_employees RENAME TO employees_archive;
```

DROP TABLE:

```sql
-- Drop table (permanent deletion)
DROP TABLE IF EXISTS employees;

-- Drop with foreign key checks
DROP TABLE IF EXISTS attendance
CASCADE;  -- Also drops dependent tables
```

Data Manipulation Language (DML):

INSERT:

```sql
-- Single row insert
INSERT INTO employees (first_name, last_name, email, department_id, hire_date, salary)
VALUES ('John', 'Doe', 'john@company.com', 1, '2024-01-15', 75000.00);

-- Multiple row insert
INSERT INTO employees (first_name, last_name, email, department_id, hire_date, salary)
VALUES
    ('Jane', 'Smith', 'jane@company.com', 1, '2024-01-20', 68000.00),
    ('Bob', 'Johnson', 'bob@company.com', 2, '2024-02-01', 72000.00),
    ('Alice', 'Brown', 'alice@company.com', 1, '2024-02-15', 65000.00);

-- Insert from select
INSERT INTO employee_archive (id, first_name, last_name, email, hire_date)
SELECT id, first_name, last_name, email, hire_date
FROM employees
WHERE status = 'terminated';

-- Insert with on duplicate key update
INSERT INTO departments (id, name, description)
VALUES (5, 'Marketing', 'Marketing and sales department')
ON DUPLICATE KEY UPDATE
SET name = VALUES(name), description = VALUES(description);
```

UPDATE:

```sql
-- Single row update
UPDATE employees
SET salary = 80000.00, updated_at = CURRENT_TIMESTAMP
WHERE id = 123;

-- Update multiple columns
UPDATE employees
SET
    salary = salary * 1.05,
    status = 'active',
    last_review_date = CURRENT_DATE
WHERE department_id = 1 AND status = 'inactive';

-- Update with join
UPDATE employees e
JOIN departments d ON e.department_id = d.id
SET e.salary = e.salary * 1.10
WHERE d.name = 'Information Technology';

-- Conditional update
UPDATE employees
SET
    salary = CASE
        WHEN department_id = 1 THEN salary * 1.10
        WHEN department_id = 2 THEN salary * 1.08
        WHEN department_id = 3 THEN salary * 1.12
        ELSE salary * 1.05
    END
WHERE status = 'active';
```

DELETE:

```sql
-- Delete specific rows
DELETE FROM employees
WHERE id = 123;

-- Delete with condition
DELETE FROM attendance
WHERE date < DATE_SUB(CURRENT_DATE, INTERVAL 2 YEAR);

-- Delete with join
DELETE e FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE d.name = 'Closed Department';

-- Truncate table (delete all rows)
TRUNCATE TABLE attendance_logs;
```

Data Query Language (DQL):

Basic SELECT:

```sql
-- Select all columns
SELECT * FROM employees;

-- Select specific columns
SELECT id, first_name, last_name, email, department_id
FROM employees
WHERE status = 'active';

-- Column aliases
SELECT
    e.first_name,
    e.last_name,
    d.name AS department_name,
    CONCAT(e.first_name, ' ', e.last_name) AS full_name
FROM employees e
JOIN departments d ON e.department_id = d.id;
```

Advanced SELECT:

```sql
-- Aggregate functions
SELECT
    d.name AS department,
    COUNT(e.id) AS employee_count,
    AVG(e.salary) AS avg_salary,
    MAX(e.salary) AS max_salary,
    MIN(e.salary) AS min_salary,
    SUM(e.salary) AS total_salary_cost
FROM employees e
JOIN departments d ON e.department_id = d.id
GROUP BY d.id, d.name
ORDER BY employee_count DESC;

-- Subqueries
SELECT * FROM employees
WHERE id IN (
    SELECT employee_id FROM performance_reviews
    WHERE rating >= 4.5
    AND review_date >= DATE_SUB(CURRENT_DATE, INTERVAL 6 MONTH)
);

-- Window functions (MySQL 8.0+)
SELECT
    employee_id,
    review_date,
```

```sql
    rating,
    ROW_NUMBER() OVER (PARTITION BY employee_id ORDER BY review_date DESC) as review_rank,
    AVG(rating) OVER (PARTITION BY employee_id ORDER BY review_date ROWS BETWEEN 2 PRECEDING
FROM performance_reviews;
```

Joins:

```sql
-- Inner join
SELECT e.first_name, e.last_name, d.name AS department
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;

-- Left join
SELECT e.first_name, e.last_name, d.name AS department, COUNT(a.id) AS attendance_days
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id
LEFT JOIN attendance a ON e.id = a.employee_id
GROUP BY e.id, e.first_name, e.last_name, d.name;

-- Multiple joins
SELECT
    e.first_name, e.last_name,
    d.name AS department,
    l.type AS leave_type,
    COUNT(l.id) AS leave_count
FROM employees e
JOIN departments d ON e.department_id = d.id
LEFT JOIN leaves l ON e.id = l.employee_id
LEFT JOIN leave_types lt ON l.leave_type_id = lt.id
GROUP BY e.id, e.first_name, e.last_name, d.name, l.type;
```

Transaction Control:

```sql
-- Start transaction
START TRANSACTION;

-- Multiple operations
INSERT INTO salary_history (employee_id, old_salary, new_salary, change_date)
VALUES (123, 50000.00, 55000.00, CURRENT_DATE);

UPDATE employees
SET salary = 55000.00, updated_at = CURRENT_TIMESTAMP
WHERE id = 123;

-- Commit or rollback
COMMIT;
-- ROLLBACK;  -- If error occurs
```

Index Management:

```sql
-- Create indexes
CREATE INDEX idx_employee_email ON employees(email);
CREATE INDEX idx_employee_name ON employees(last_name, first_name);
CREATE INDEX idx_employee_department ON employees(department_id, status);

-- Full-text index
CREATE FULLTEXT INDEX idx_resume_content ON resumes(content);

-- Drop index
DROP INDEX idx_employee_email ON employees;

-- Show indexes
SHOW INDEX FROM employees;
```

User and Privilege Management:

```sql
-- Create user
CREATE USER 'hrms_user'@'localhost' IDENTIFIED BY 'secure_password';

-- Grant privileges
GRANT SELECT, INSERT, UPDATE, DELETE ON hrms_db.* TO 'hrms_user'@'localhost';

-- Grant specific privileges
GRANT SELECT ON employees TO 'report_user'@'localhost';
GRANT INSERT, UPDATE ON attendance TO 'hr_user'@'localhost';

-- Revoke privileges
REVOKE DELETE ON employees FROM 'hr_user'@'localhost';

-- Show grants
SHOW GRANTS FOR 'hrms_user'@'localhost';
```

4.7 Database Design Principles

Database design principles are fundamental guidelines for creating efficient, maintainable, and scalable database schemas. Applying these principles to the HR Management System ensures optimal performance, data integrity, and ease of maintenance.

Normalization Principles:

First Normal Form (1NF): - Each table cell contains atomic (indivisible) values - No repeating groups or arrays in single columns - Each row is uniquely identifiable

```sql
-- Bad: Repeating groups
CREATE TABLE employee_skills_bad (
```

```sql
    employee_id INT,
    skill1 VARCHAR(50),
    skill2 VARCHAR(50),
    skill3 VARCHAR(50)
);

-- Good: Atomic values
CREATE TABLE employee_skills (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT,
    skill_name VARCHAR(50),
    skill_level ENUM('beginner', 'intermediate', 'advanced', 'expert')
);
```

Second Normal Form (2NF): - Meets 1NF requirements - No partial dependencies on composite primary keys - All non-key attributes depend on entire primary key

```sql
-- Bad: Partial dependency
CREATE TABLE employee_schedule_bad (
    employee_id INT,
    day_of_week ENUM('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'),
    shift_type ENUM('morning', 'afternoon', 'evening'),
    shift_location VARCHAR(50),
    PRIMARY KEY (employee_id, day_of_week)
);

-- Good: No partial dependency
CREATE TABLE shifts (
    id INT PRIMARY KEY AUTO_INCREMENT,
    day_of_week ENUM('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'),
    shift_type ENUM('morning', 'afternoon', 'evening'),
    shift_location VARCHAR(50)
);

CREATE TABLE employee_schedule (
    employee_id INT,
    shift_id INT,
    PRIMARY KEY (employee_id, shift_id),
    FOREIGN KEY (shift_id) REFERENCES shifts(id)
);
```

Third Normal Form (3NF): - Meets 2NF requirements - No transitive dependencies - Non-key attributes depend only on primary key

```sql
-- Bad: Transitive dependency
CREATE TABLE employee_bad (
    employee_id INT PRIMARY KEY,
```

```sql
    department_name VARCHAR(50),
    department_location VARCHAR(100),
    department_manager VARCHAR(50)  -- Depends on department_name
);

-- Good: Eliminated transitive dependency
CREATE TABLE departments (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    location VARCHAR(100),
    manager_id INT
);

CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(id)
);
```

Database Design Best Practices:

Primary Key Design:

```sql
-- Surrogate keys (recommended)
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,  -- Surrogate key
    employee_code VARCHAR(20) UNIQUE,      -- Natural key as unique constraint
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE
);

-- Composite primary keys
CREATE TABLE employee_qualifications (
    employee_id INT,
    qualification_id INT,
    date_obtained DATE,
    PRIMARY KEY (employee_id, qualification_id)  -- Composite key
);
```

Foreign Key Relationships:

```sql
-- One-to-many relationship
CREATE TABLE departments (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL
);

CREATE TABLE employees (
```

```sql
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(id)
    ON DELETE SET NULL ON UPDATE CASCADE
);

-- Many-to-many relationship
CREATE TABLE leave_types (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    max_days_per_year INT
);

CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE employee_leave_balance (
    employee_id INT,
    leave_type_id INT,
    year YEAR,
    total_allocated INT,
    used INT,
    PRIMARY KEY (employee_id, leave_type_id, year),
    FOREIGN KEY (employee_id) REFERENCES employees(id) ON DELETE CASCADE,
    FOREIGN KEY (leave_type_id) REFERENCES leave_types(id) ON DELETE CASCADE
);
```

Indexing Strategy:

```sql
-- Primary key indexes (automatic)
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100)
);

-- Secondary indexes for performance
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100),
    department_id INT,
    INDEX idx_employee_name (name),
    INDEX idx_employee_email (email),
```

```sql
    INDEX idx_employee_department (department_id),
    INDEX idx_employee_status (status, department_id)  -- Composite index
);

-- Full-text indexes for search
CREATE TABLE resumes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    candidate_id INT,
    content TEXT,
    FULLTEXT INDEX ft_resume (content)
);
```

Data Integrity Constraints:

```sql
-- NOT NULL constraints
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,  -- Must have value
    email VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NULL          -- Optional phone
);

-- UNIQUE constraints
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(100) NOT NULL UNIQUE,  -- Unique email
    employee_code VARCHAR(20) NOT NULL UNIQUE  -- Unique employee code
);

-- CHECK constraints
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    salary DECIMAL(10,2) CHECK (salary >= 0),  -- Positive salary
    hire_date DATE CHECK (hire_date <= CURRENT_DATE),  -- Not future date
    status ENUM('active', 'inactive', 'terminated') DEFAULT 'active'
);
```

Naming Conventions:

```sql
-- Table naming (plural, snake_case)
CREATE TABLE employees;
CREATE TABLE departments;
CREATE TABLE attendance_records;
CREATE TABLE performance_reviews;

-- Column naming (snake_case, descriptive)
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
```

```sql
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email_address VARCHAR(100),
    hire_date DATE,
    created_at TIMESTAMP
);

-- Index naming (descriptive prefix)
CREATE INDEX idx_employee_last_name ON employees(last_name);
CREATE INDEX idx_employee_department ON employees(department_id);
CREATE INDEX idx_employee_email ON employees(email);
```

4.8 Indexing and Performance Optimization

Database performance optimization is crucial for ensuring the HR Management System responds quickly to user queries and handles concurrent access efficiently. MySQL provides various indexing and optimization techniques that significantly improve query performance.

Index Types and Strategies:

Primary Key Indexes:

```sql
-- Automatic indexing for primary keys
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,  -- Automatically indexed
    name VARCHAR(100),
    email VARCHAR(100)
);

-- Composite primary key
CREATE TABLE employee_daily_attendance (
    employee_id INT,
    attendance_date DATE,
    status ENUM('present', 'absent', 'late', 'half_day'),
    notes TEXT,
    PRIMARY KEY (employee_id, attendance_date)  -- Composite index
);
```

Secondary Indexes:

```sql
-- Single-column indexes
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100),
    department_id INT,
    INDEX idx_employee_name (name),           -- For name searches
```

```sql
    INDEX idx_employee_email (email),          -- For email lookups
    INDEX idx_employee_department (department_id)  -- For department joins
);

-- Composite indexes for multi-column queries
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    department_id INT,
    status ENUM('active', 'inactive', 'terminated'),
    INDEX idx_employee_dept_status (department_id, status),  -- For filtering by dept and s
    INDEX idx_employee_status_dept (status, department_id)   -- Different column order
);
```

Unique Indexes:

```sql
-- Enforce data uniqueness
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(100) NOT NULL,
    employee_code VARCHAR(20) NOT NULL,
    UNIQUE KEY uk_employee_email (email),           -- Unique email
    UNIQUE KEY uk_employee_code (employee_code)         -- Unique employee code
);

-- Composite unique index
CREATE TABLE employee_qualifications (
    employee_id INT,
    qualification_id INT,
    date_obtained DATE,
    UNIQUE KEY uk_emp_qual (employee_id, qualification_id)  -- Unique combination
);
```

Full-Text Indexes:

```sql
-- For text searching
CREATE TABLE resumes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    candidate_id INT,
    content TEXT,
    FULLTEXT INDEX ft_resume_content (content)
);

-- Search with full-text index
SELECT candidate_id, content
FROM resumes
WHERE MATCH(content) AGAINST('PHP MySQL JavaScript' IN NATURAL LANGUAGE MODE);
```

```sql
-- Boolean mode search
SELECT candidate_id, content,
    MATCH(content) AGAINST('PHP MySQL' IN BOOLEAN MODE) as relevance_score
FROM resumes
WHERE MATCH(content) AGAINST('PHP MySQL' IN BOOLEAN MODE)
ORDER BY relevance_score DESC;
```

Performance Analysis Tools:

EXPLAIN Command:

```sql
-- Analyze query execution plan
EXPLAIN SELECT * FROM employees
WHERE department_id = 1 AND status = 'active'
ORDER BY hire_date DESC;

-- Detailed execution plan
EXPLAIN FORMAT=JSON
SELECT e.*, d.name
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE e.salary > 50000;
```

Performance Schema:

```sql
-- Query performance analysis
SELECT
    DIGEST_TEXT,
    COUNT_STAR,
    SUM_TIMER_WAIT,
    AVG_TIMER_WAIT
FROM performance_schema.events_statements_summary_by_digest
WHERE DIGEST_TEXT LIKE '%employees%'
ORDER BY SUM_TIMER_WAIT DESC
LIMIT 10;

-- Table access analysis
SELECT
    table_name,
    rows_read,
    rows_inserted,
    rows_updated,
    rows_deleted
FROM performance_schema.table_io_waits_summary
WHERE object_schema = 'hrms_db'
ORDER BY rows_read + rows_inserted + rows_updated + rows_deleted DESC;
```

Slow Query Log:

```sql
-- Enable slow query logging
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 2;  -- Log queries taking > 2 seconds

-- Analyze slow queries
SELECT
    start_time,
    query_time,
    lock_time,
    rows_sent,
    sql_text
FROM mysql.slow_log
WHERE sql_text LIKE '%employees%'
ORDER BY query_time DESC
LIMIT 20;
```

Query Optimization Techniques:

Index Usage:

```sql
-- Ensure queries use indexes
-- Good: Uses index on department_id
SELECT * FROM employees WHERE department_id = 5;

-- Bad: Function on indexed column prevents index usage
SELECT * FROM employees WHERE YEAR(hire_date) = 2024;  -- Can't use hire_date index

-- Solution: Range query instead
SELECT * FROM employees
WHERE hire_date >= '2024-01-01' AND hire_date < '2025-01-01';
```

JOIN Optimization:

```sql
-- Use indexed columns in joins
SELECT e.*, d.name
FROM employees e
INNER JOIN departments d ON e.department_id = d.id  -- department_id indexed
WHERE e.status = 'active';

-- Covering indexes for better performance
CREATE INDEX idx_employee_covering ON employees(department_id, status, hire_date);
-- This index can satisfy queries without accessing table data
```

Subquery Optimization:

```sql
-- Use EXISTS instead of IN for better performance
SELECT e.*
FROM employees e
WHERE EXISTS (
```

```sql
    SELECT 1 FROM attendance a
    WHERE a.employee_id = e.id
    AND a.date = CURRENT_DATE
);

-- Use JOIN instead of subquery in WHERE clause
SELECT e.*
FROM employees e
JOIN attendance a ON e.id = a.employee_id
WHERE a.date = CURRENT_DATE;
```

Database Configuration:

Memory Settings:

```sql
-- InnoDB buffer pool size
SET GLOBAL innodb_buffer_pool_size = 1073741824;  -- 1GB

-- Key buffer size
SET GLOBAL key_buffer_size = 268435456;  -- 256MB

-- Query cache size
SET GLOBAL query_cache_size = 67108864;  -- 64MB

-- Table cache size
SET GLOBAL table_open_cache = 4000;
```

Connection Settings:

```sql
-- Maximum connections
SET GLOBAL max_connections = 200;

-- Connection timeout
SET GLOBAL wait_timeout = 28800;  -- 8 hours

-- Interactive timeout
SET GLOBAL interactive_timeout = 3600;  -- 1 hour
```

Performance Monitoring:

```sql
-- Show process list
SHOW PROCESSLIST;

-- Show server status
SHOW STATUS LIKE 'Handler%';
SHOW STATUS LIKE 'Innodb%';

-- Show engine status
SHOW ENGINE INNODB STATUS;
```

4.9 MySQL Security

Security is a critical aspect of database management, especially for HR systems containing sensitive employee information. MySQL provides comprehensive security features that, when properly configured, protect data from unauthorized access and ensure compliance with data protection regulations.

Authentication and Authorization:

User Management:

```sql
-- Create database-specific user
CREATE USER 'hrms_app'@'localhost' IDENTIFIED BY 'StrongP@ssw0rd!';

-- Create user with specific privileges
CREATE USER 'hr_readonly'@'192.168.1.%' IDENTIFIED BY 'ReadOnlyP@ss';

-- Grant database privileges
GRANT SELECT, INSERT, UPDATE, DELETE ON hrms_db.* TO 'hrms_app'@'localhost';

-- Grant specific table privileges
GRANT SELECT ON employees TO 'hr_reports'@'localhost';
GRANT SELECT ON attendance TO 'hr_reports'@'localhost';
GRANT SELECT ON leaves TO 'hr_reports'@'localhost';

-- Grant with GRANT OPTION
GRANT SELECT ON hrms_db.* TO 'hr_manager'@'localhost' WITH GRANT OPTION;
```

Password Security:

```sql
-- Set password policy
SET GLOBAL validate_password_policy = 'STRONG';
SET GLOBAL validate_password_length = 12;

-- Require password changes
SET GLOBAL password_expired = 180;  -- Password expires every 180 days

-- Password history
SET GLOBAL password_history = 5;  -- Remember last 5 passwords
```

Access Control:

```sql
-- Host-based access control
CREATE USER 'hrms_user'@'192.168.1.100' IDENTIFIED BY 'password';  -- Specific IP
CREATE USER 'hrms_user'@'%.company.com' IDENTIFIED BY 'password';   -- Domain wildcard

-- Limit connections per user
GRANT USAGE ON *.* TO 'hrms_user'@'localhost'
WITH MAX_USER_CONNECTIONS 5;
```

```sql
-- Resource limits
GRANT SELECT ON hrms_db.* TO 'hrms_user'@'localhost'
WITH MAX_QUERIES_PER_HOUR 1000
MAX_UPDATES_PER_HOUR 500
MAX_CONNECTIONS_PER_HOUR 10;
```

Data Encryption:

Transport Layer Security:

```sql
-- Require SSL for connections
GRANT SELECT ON hrms_db.* TO 'hrms_user'@'localhost' REQUIRE SSL;

-- Require X509 certificate
GRANT SELECT ON hrms_db.* TO 'hrms_user'@'localhost'
REQUIRE SUBJECT '/C=US/ST=California/L=San Francisco/O=Company/CN=hrms-cert';
```

Data-at-Rest Encryption:

```sql
-- Enable table encryption
CREATE TABLE sensitive_data (
    id INT PRIMARY KEY AUTO_INCREMENT,
    ssn VARCHAR(11) ENCRYPTED,
    salary DECIMAL(10,2) ENCRYPTED,
    bank_account VARCHAR(20) ENCRYPTED
) ENCRYPTION='Y';

-- Column-level encryption
ALTER TABLE employees
ADD COLUMN ssn VARBINARY(255) ENCRYPTED;
```

Audit and Logging:

```sql
-- Enable general query log
SET GLOBAL general_log = 'ON';
SET GLOBAL general_log_file = '/var/log/mysql/general.log';

-- Enable slow query log
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL slow_query_log_file = '/var/log/mysql/slow.log';

-- Enable binary log for replication
SET GLOBAL log_bin = 'ON';
SET GLOBAL binlog_format = 'ROW';

-- Audit plugin installation
INSTALL PLUGIN audit_log SONAME 'audit_log.so';
```

SQL Injection Prevention:

```sql
-- Use prepared statements (application level)
PREPARE stmt FROM 'SELECT * FROM employees WHERE id = ? AND status = ?';
EXECUTE stmt USING @employee_id, @status;

-- Use parameterized queries
SELECT * FROM employees
WHERE id = ? AND department_id = ? AND hire_date >= ?;

-- Avoid dynamic SQL construction
-- Bad: Vulnerable to injection
SET @sql = CONCAT('SELECT * FROM employees WHERE name = ''', user_input, '''');
PREPARE stmt FROM @sql;
EXECUTE stmt;

-- Good: Safe parameterized query
PREPARE stmt FROM 'SELECT * FROM employees WHERE name = ?';
EXECUTE stmt USING @user_input;
```

Privilege Management:

```sql
-- Review current grants
SHOW GRANTS FOR CURRENT_USER;

-- Revoke unnecessary privileges
REVOKE ALL PRIVILEGES ON *.* FROM 'test_user'@'localhost';

-- Grant minimal required privileges
GRANT SELECT ON hrms_db.employees TO 'report_user'@'localhost';
GRANT SELECT ON hrms_db.attendance TO 'report_user'@'localhost';
```

Security Best Practices: - Use strong, unique passwords for all database users - Implement principle of least privilege (grant only necessary permissions) - Regularly review and revoke unnecessary privileges - Enable SSL/TLS for all database connections - Use prepared statements to prevent SQL injection - Implement comprehensive audit logging - Regular security updates and patching - Encrypt sensitive data both at rest and in transit - Use network-level access controls and firewalls - Regular security audits and vulnerability assessments

4.10 MySQL Administration and Maintenance

Database administration and maintenance are essential for ensuring the HR Management System remains reliable, performant, and secure over time. MySQL provides comprehensive tools and commands for effective database management.

Backup and Recovery:

Logical Backups:

```
-- Complete database backup
mysqldump -u root -p hrms_db > hrms_backup_$(date +%Y%m%d).sql

-- Specific table backup
mysqldump -u root -p hrms_db employees departments > hrms_tables_backup.sql

-- Backup with stored procedures and triggers
mysqldump -u root -p --routines --triggers hrms_db > hrms_complete_backup.sql

-- Compressed backup
mysqldump -u root -p hrms_db | gzip > hrms_backup_$(date +%Y%m%d).sql.gz
```

Physical Backups:

```
-- Hot backup (InnoDB)
mysqlhotcopy /var/lib/mysql/hrms_db /backup/hrms_db_$(date +%Y%m%d)

-- File system backup
cp -r /var/lib/mysql/hrms_db /backup/mysql_files/hrms_db_$(date +%Y%m%d)

-- Binary backup
mysqlbinlog --read-from-remote-server --host=localhost \
    --raw --stop-never --result-file=/backup/binlog.000001
```

Automated Backup Scripts:

```
#!/bin/bash
# backup_script.sh

DB_NAME="hrms_db"
DB_USER="root"
BACKUP_DIR="/backup/mysql"
DATE=$(date +%Y%m%d_%H%M%S)

# Create backup directory
mkdir -p $BACKUP_DIR

# Logical backup
mysqldump -u $DB_USER -p --single-transaction --routines --triggers \
    $DB_NAME | gzip > $BACKUP_DIR/hrms_backup_$DATE.sql.gz

# Keep last 7 days of backups
find $BACKUP_DIR -name "hrms_backup_*.sql.gz" -mtime +7 -delete

echo "Backup completed: hrms_backup_$DATE.sql.gz"
```

Database Monitoring:

Performance Monitoring:

```sql
-- Show running processes
SHOW PROCESSLIST;

-- Kill long-running queries
KILL 12345;  -- Process ID from SHOW PROCESSLIST

-- Show server status
SHOW STATUS;
SHOW ENGINE INNODB STATUS;

-- Performance schema queries
SELECT * FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC LIMIT 10;
```

Resource Usage Monitoring:

```sql
-- Connection usage
SHOW STATUS LIKE 'Threads%';
SHOW STATUS LIKE 'Connections%';

-- Memory usage
SHOW STATUS LIKE 'Innodb_buffer_pool%';
SHOW STATUS LIKE 'Key%';

-- Query cache performance
SHOW STATUS LIKE 'Qcache%';
```

Maintenance Operations:

Table Maintenance:

```sql
-- Analyze table for query optimizer
ANALYZE TABLE employees;
ANALYZE TABLE attendance;
ANALYZE TABLE performance_reviews;

-- Optimize table for defragmentation
OPTIMIZE TABLE employees;
OPTIMIZE TABLE attendance;

-- Check table for errors
CHECK TABLE employees;
CHECK TABLE attendance;

-- Repair corrupted table
REPAIR TABLE employees;
```

Index Maintenance:

```sql
-- Rebuild indexes
ALTER TABLE employees ENGINE=InnoDB;

-- Add missing indexes
ALTER TABLE employees ADD INDEX idx_employee_hire_date (hire_date);

-- Remove unused indexes
DROP INDEX idx_unused_index ON employees;

-- Monitor index usage
SELECT * FROM performance_schema.table_io_waits_summary
WHERE object_name = 'employees';
```

Log Management:

```sql
-- Rotate binary logs
FLUSH LOGS;

-- Purge old binary logs
PURGE BINARY LOGS BEFORE DATE_SUB(NOW(), INTERVAL 7 DAY);

-- Clear general log
SET GLOBAL general_log = 'OFF';
TRUNCATE TABLE mysql.general_log;
SET GLOBAL general_log = 'ON';
```

Configuration Management:

```sql
-- Show current configuration
SHOW VARIABLES;
SHOW VARIABLES LIKE 'innodb%';
SHOW VARIABLES LIKE 'query_cache%';

-- Set configuration parameters
SET GLOBAL innodb_buffer_pool_size = 1073741824;  -- 1GB
SET GLOBAL query_cache_size = 67108864;           -- 64MB
SET GLOBAL max_connections = 200;
```

Scheduled Maintenance:

```bash
#!/bin/bash
# maintenance_script.sh

# Daily optimization
mysql -u root -p -e "
    ANALYZE TABLE hrms_db.employees;
    ANALYZE TABLE hrms_db.attendance;
    ANALYZE TABLE hrms_db.performance_reviews;
"
```

```
# Weekly optimization
mysql -u root -p -e "
    OPTIMIZE TABLE hrms_db.employees;
    OPTIMIZE TABLE hrms_db.attendance;
    OPTIMIZE TABLE hrms_db.performance_reviews;
"

# Monthly check
mysql -u root -p -e "
    CHECK TABLE hrms_db.employees;
    CHECK TABLE hrms_db.attendance;
    CHECK TABLE hrms_db.performance_reviews;
"
```

4.11 MySQL vs Other Database Systems

Choosing the right database system is crucial for the success of the HR Management System. MySQL competes with various other database systems, each with distinct advantages and use cases. Understanding these differences helps in making informed technology decisions.

Relational Database Systems Comparison:

MySQL vs PostgreSQL:

```
-- MySQL: Simpler setup, better for read-heavy workloads
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    skills JSON  -- Native JSON support
);

-- PostgreSQL: More advanced features, better for complex queries
CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    skills JSONB,  -- Binary JSON with indexing
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- PostgreSQL advanced features
CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    skills JSONB,
    salary_range INT4RANGE,  -- Custom range types
```

```sql
    CONSTRAINT check_salary CHECK (salary > 0)
);
```

MySQL vs SQL Server:

```sql
-- MySQL: Open-source, cross-platform, lower TCO
-- SQL Server: Commercial, Windows-centric, advanced features

-- MySQL syntax
SELECT JSON_EXTRACT(skills, '$.language') FROM employees;

-- SQL Server syntax
SELECT JSON_VALUE(skills, '$.language') FROM employees;
```

MySQL vs Oracle:

```sql
-- MySQL: Lightweight, easier administration
-- Oracle: Enterprise features, higher scalability, complex administration

-- MySQL simple replication
CHANGE MASTER TO master_host;
START SLAVE;

-- Oracle advanced replication (Data Guard)
-- Complex setup with RAC and standby databases
```

NoSQL Databases:

MySQL vs MongoDB:

```sql
-- MySQL: Structured data, ACID transactions, complex queries
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    skills JSON,
    INDEX idx_employee_name (name)
);

-- MongoDB: Document-oriented, flexible schema, horizontal scaling
db.employees.insertOne({
    name: "John Doe",
    skills: ["PHP", "MySQL", "JavaScript"],
    department: "IT"
});

-- MongoDB aggregation
db.employees.aggregate([
    { $match: { department: "IT" } },
    { $group: { _id: "$department", avgSalary: { $avg: "$salary" } } }
```

```
]);
```

Cloud Databases:

MySQL vs Amazon RDS:

```
-- On-premises MySQL
-- Full control, no managed services
-- Manual backups and maintenance
-- Lower cost for small deployments

-- Amazon RDS MySQL
-- Managed service, automated backups
-- High availability with Multi-AZ
-- Read replicas for scaling
-- Pay-as-you-go pricing
```

Selection Criteria for HRMS:

Performance Requirements: - Read-heavy operations (employee lookups, reports) - Complex joins across multiple tables - Need for fast query response times - Concurrent user access

Data Characteristics: - Structured relational data - Need for ACID compliance - Complex relationships between entities - Requirement for data integrity

Operational Considerations: - Ease of administration - Community support and documentation - Development team expertise - Total cost of ownership - Scalability requirements

MySQL Advantages for HRMS: - Proven reliability in production environments - Excellent PHP integration - Large community and talent pool - Lower total cost of ownership - Mature ecosystem of tools and utilities - Good performance for read-heavy workloads

When to Consider Alternatives: - Need for advanced analytics (PostgreSQL) - Requirement for document storage (MongoDB) - Large-scale distributed deployment (cloud databases) - Complex transaction requirements (Oracle) - Microsoft-centric environment (SQL Server)

This comprehensive analysis of MySQL provides the foundation for understanding how the database system supports the HR Management System's requirements for performance, reliability, and scalability.

CHAPTER 6

HR-MANAGEMENT-SYSTEM

6.1 System Architecture and Design

The HR Management System is designed with a modular, scalable architecture that supports the complex requirements of modern human resource management. The system follows a three-tier architecture pattern, separating presentation, business logic, and data access layers to ensure maintainability, scalability, and security.

System Architecture Overview:

Three-Tier Architecture: - **Presentation Layer:** User interface components including web pages, forms, and dashboards - **Business Logic Layer:** Core application logic, validation, and processing rules - **Data Access Layer:** Database interactions, data persistence, and query optimization

Component-Based Design: The HR Management System is organized into functional modules that can be developed, tested, and maintained independently:

Core Modules: 1. **Authentication Module:** User login, session management, and access control 2. **Employee Management Module:** Employee records, personal information, and employment details 3. **Department Management Module:** Organizational structure and department hierarchy 4. **Attendance Module:** Time tracking, attendance monitoring, and reporting 5. **Leave Management Module:** Leave requests, approvals, and balance tracking 6. **Performance Management Module:** Performance reviews, ratings, and improvement plans 7. **Recruitment Module:** Job postings, candidate management, and hiring workflow 8. **Reporting Module:** Analytics, dashboards, and business intelligence

Technical Architecture:

Frontend Architecture:

```
<!-- Responsive Web Design -->
<div class="container-fluid">
    <div class="row">
        <div class="col-md-3">
            <!-- Sidebar Navigation -->
            <nav class="sidebar">
                <ul class="nav flex-column">
                    <li class="nav-item">
                        <a class="nav-link" href="dashboard.php">Dashboard</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="employees.php">Employees</a>
```

```
                    </li>
                </ul>
            </nav>
        </div>
        <div class="col-md-9">
            <!-- Main Content Area -->
            <main class="main-content">
                <!-- Dynamic content loaded here -->
            </main>
        </div>
    </div>
</div>

<!-- JavaScript Framework Integration -->
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script src="assets/js/app.js"></script>
```

Backend Architecture:

```
<?php
// MVC Pattern Implementation
class EmployeeController {
    private $employeeModel;
    private $view;

    public function __construct() {
        $this->employeeModel = new EmployeeModel();
        $this->view = new View();
    }

    public function index() {
        $employees = $this->employeeModel->getAllEmployees();
        $this->view->render('employees/index', ['employees' => $employees]);
    }

    public function create() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $data = $this->validateEmployeeData($_POST);
            if ($this->employeeModel->create($data)) {
                header('Location: /employees.php?success=1');
                exit;
            }
        }
        $this->view->render('employees/create');
    }

    private function validateEmployeeData($data) {
```

```
        $errors = [];

        if (empty($data['first_name'])) {
            $errors['first_name'] = 'First name is required';
        }

        if (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
            $errors['email'] = 'Valid email is required';
        }

        return $errors ? ['errors' => $errors] : $data;
    }
}

// Database Layer
class EmployeeModel {
    private $db;

    public function __construct() {
        $this->db = Database::getInstance();
    }

    public function getAllEmployees() {
        $stmt = $this->db->prepare("
            SELECT e.*, d.name as department_name
            FROM employees e
            LEFT JOIN departments d ON e.department_id = d.id
            ORDER BY e.last_name, e.first_name
        ");
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    public function create($data) {
        $stmt = $this->db->prepare("
            INSERT INTO employees (first_name, last_name, email, department_id, hire_date, s
            VALUES (?, ?, ?, ?, ?, ?)
        ");
        return $stmt->execute([
            $data['first_name'],
            $data['last_name'],
            $data['email'],
            $data['department_id'],
            $data['hire_date'],
            $data['salary']
        ]);
```

```
    }
}
?>
```

Database Architecture:

```sql
-- Core Database Schema
CREATE DATABASE hrms_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Employees Table
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employee_code VARCHAR(20) UNIQUE NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20),
    department_id INT,
    position VARCHAR(100),
    salary DECIMAL(10,2),
    hire_date DATE NOT NULL,
    status ENUM('active', 'inactive', 'terminated', 'on_leave') DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE SET NULL
);

-- Departments Table
CREATE TABLE departments (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    manager_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (manager_id) REFERENCES employees(id) ON DELETE SET NULL
);

-- Attendance Table
CREATE TABLE attendance (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT NOT NULL,
    date DATE NOT NULL,
    check_in TIME,
    check_out TIME,
    break_duration INT DEFAULT 0,
    status ENUM('present', 'absent', 'late', 'half_day', 'holiday') DEFAULT 'present',
    notes TEXT,
```

```sql
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (employee_id) REFERENCES employees(id) ON DELETE CASCADE,
    UNIQUE KEY uk_employee_date (employee_id, date)
);

-- Leave Types Table
CREATE TABLE leave_types (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    description TEXT,
    max_days_per_year INT DEFAULT 0,
    requires_approval BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Leaves Table
CREATE TABLE leaves (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT NOT NULL,
    leave_type_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    total_days DECIMAL(4,1) NOT NULL,
    reason TEXT,
    status ENUM('pending', 'approved', 'rejected', 'cancelled') DEFAULT 'pending',
    approved_by INT,
    approved_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (employee_id) REFERENCES employees(id) ON DELETE CASCADE,
    FOREIGN KEY (leave_type_id) REFERENCES leave_types(id) ON DELETE RESTRICT,
    FOREIGN KEY (approved_by) REFERENCES employees(id) ON DELETE SET NULL
);

-- Performance Reviews Table
CREATE TABLE performance_reviews (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT NOT NULL,
    reviewer_id INT NOT NULL,
    review_date DATE NOT NULL,
    rating DECIMAL(3,1) NOT NULL CHECK (rating >= 1 AND rating <= 5),
    strengths TEXT,
    weaknesses TEXT,
    goals TEXT,
    comments TEXT,
    status ENUM('draft', 'submitted', 'reviewed', 'completed') DEFAULT 'draft',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (employee_id) REFERENCES employees(id) ON DELETE CASCADE,
    FOREIGN KEY (reviewer_id) REFERENCES employees(id) ON DELETE RESTRICT
);
```

6.2 Module Implementation

The HR Management System implements each functional module with specific features, workflows, and user interfaces. Each module is designed to handle specific HR processes while maintaining integration with other system components.

Authentication and Security Module:

User Authentication System:

```php
<?php
class AuthenticationController {
    private $userModel;

    public function login() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $username = sanitize_input($_POST['username']);
            $password = $_POST['password'];

            $user = $this->userModel->authenticate($username, $password);

            if ($user) {
                // Set session variables
                $_SESSION['user_id'] = $user['id'];
                $_SESSION['username'] = $user['username'];
                $_SESSION['role'] = $user['role'];
                $_SESSION['login_time'] = time();

                // Set remember me cookie if requested
                if (isset($_POST['remember'])) {
                    $token = generate_auth_token($user['id']);
                    setcookie('remember_token', $token, time() + (30 * 24 * 60 * 60), '/');
                }

                // Log successful login
                $this->logActivity($user['id'], 'login', 'Successful login');

                header('Location: dashboard.php');
                exit;
            } else {
                $error = 'Invalid username or password';
                $this->logActivity(null, 'login_failed', 'Failed login attempt for: ' . $use
```

```
            }
        }

        include 'views/login.php';
    }

    public function logout() {
        // Log activity
        $this->logActivity($_SESSION['user_id'], 'logout', 'User logged out');

        // Clear session
        session_destroy();

        // Clear remember me cookie
        setcookie('remember_token', '', time() - 3600, '/');

        header('Location: login.php');
        exit;
    }

    private function logActivity($userId, $action, $description) {
        $stmt = $this->db->prepare("
            INSERT INTO activity_logs (user_id, action, description, ip_address, user_agent)
            VALUES (?, ?, ?, ?, ?)
        ");
        $stmt->execute([
            $userId,
            $action,
            $description,
            $_SERVER['REMOTE_ADDR'],
            $_SERVER['HTTP_USER_AGENT']
        ]);
    }
}
?>
```

Role-Based Access Control:

```
<?php
class Authorization {
    private static $permissions = [
        'admin' => [
            'employees' => ['create', 'read', 'update', 'delete'],
            'departments' => ['create', 'read', 'update', 'delete'],
            'attendance' => ['create', 'read', 'update', 'delete'],
            'leaves' => ['create', 'read', 'update', 'delete', 'approve'],
            'reports' => ['read', 'generate'],
```

```
            'settings' => ['read', 'update']
        ],
        'hr_manager' => [
            'employees' => ['create', 'read', 'update'],
            'departments' => ['read'],
            'attendance' => ['create', 'read', 'update'],
            'leaves' => ['create', 'read', 'update', 'approve'],
            'reports' => ['read', 'generate']
        ],
        'employee' => [
            'employees' => ['read'], // Only own profile
            'attendance' => ['read', 'create'], // Clock in/out
            'leaves' => ['create', 'read'], // Own leave requests
            'reports' => ['read'] // Limited reports
        ]
    ];

    public static function can($role, $resource, $action) {
        return isset(self::$permissions[$role][$resource]) &&
               in_array($action, self::$permissions[$role][$resource]);
    }

    public static function requireAuth($resource, $action) {
        if (!isset($_SESSION['user_id'])) {
            header('Location: login.php');
            exit;
        }

        $role = $_SESSION['role'];
        if (!self::can($role, $resource, $action)) {
            header('HTTP/1.0 403 Forbidden');
            include 'views/403.php';
            exit;
        }
    }
}
?>
```

Employee Management Module:

Employee CRUD Operations:

```
<?php
class EmployeeController {
    public function index() {
        Authorization::requireAuth('employees', 'read');
```

```
        $page = isset($_GET['page']) ? (int)$_GET['page'] : 1;
        $search = sanitize_input($_GET['search'] ?? '');
        $department = sanitize_input($_GET['department'] ?? '');

        $employees = $this->employeeModel->getEmployees($page, $search, $department);
        $departments = $this->departmentModel->getAllDepartments();
        $totalPages = $this->employeeModel->getTotalPages($search, $department);

        include 'views/employees/index.php';
    }

    public function create() {
        Authorization::requireAuth('employees', 'create');

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $data = $this->validateEmployeeData($_POST);

            if (!isset($data['errors'])) {
                $employeeId = $this->employeeModel->create($data);

                // Create initial leave balance
                $this->leaveBalanceModel->initializeBalance($employeeId);

                // Log activity
                $this->logActivity($_SESSION['user_id'], 'employee_created',
                    "Created employee: {$data['first_name']} {$data['last_name']}");

                $_SESSION['success'] = 'Employee created successfully';
                header('Location: employees.php');
                exit;
            }
        }

        $departments = $this->departmentModel->getAllDepartments();
        include 'views/employees/create.php';
    }

    public function edit($id) {
        Authorization::requireAuth('employees', 'update');

        $employee = $this->employeeModel->getById($id);
        if (!$employee) {
            header('HTTP/1.0 404 Not Found');
            include 'views/404.php';
            exit;
        }
```

```
        // Check if user can edit this employee
        if ($_SESSION['role'] === 'employee' && $employee['id'] != $_SESSION['user_id']) {
            header('HTTP/1.0 403 Forbidden');
            include 'views/403.php';
            exit;
        }

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $data = $this->validateEmployeeData($_POST);

            if (!isset($data['errors'])) {
                $this->employeeModel->update($id, $data);

                $this->logActivity($_SESSION['user_id'], 'employee_updated',
                    "Updated employee: {$employee['first_name']} {$employee['last_name']}");

                $_SESSION['success'] = 'Employee updated successfully';
                header('Location: employees.php');
                exit;
            }
        }

        $departments = $this->departmentModel->getAllDepartments();
        include 'views/employees/edit.php';
    }

    private function validateEmployeeData($data) {
        $errors = [];

        // Required fields validation
        if (empty($data['first_name'])) {
            $errors['first_name'] = 'First name is required';
        } elseif (!preg_match('/^[a-zA-Z\s]{2,50}$/', $data['first_name'])) {
            $errors['first_name'] = 'First name must be 2-50 letters only';
        }

        if (empty($data['last_name'])) {
            $errors['last_name'] = 'Last name is required';
        } elseif (!preg_match('/^[a-zA-Z\s]{2,50}$/', $data['last_name'])) {
            $errors['last_name'] = 'Last name must be 2-50 letters only';
        }

        if (empty($data['email'])) {
            $errors['email'] = 'Email is required';
        } elseif (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
```

```
            $errors['email'] = 'Please enter a valid email address';
        }

        // Numeric validation
        if (isset($data['salary']) && (!is_numeric($data['salary']) || $data['salary'] < 0))
            $errors['salary'] = 'Salary must be a positive number';
        }

        // Date validation
        if (!empty($data['hire_date'])) {
            $hireDate = DateTime::createFromFormat('Y-m-d', $data['hire_date']);
            if (!$hireDate || $hireDate > new DateTime()) {
                $errors['hire_date'] = 'Please enter a valid hire date (not in future)';
            }
        }

        return $errors ? ['errors' => $errors] : $data;
    }
}
?>
```

Employee Dashboard:

```
<?php
class DashboardController {
    public function index() {
        Authorization::requireAuth('dashboard', 'read');

        $userId = $_SESSION['user_id'];
        $role = $_SESSION['role'];

        if ($role === 'admin') {
            $stats = $this->getAdminStats();
        } elseif ($role === 'hr_manager') {
            $stats = $this->getHRStats();
        } else {
            $stats = $this->getEmployeeStats($userId);
        }

        include 'views/dashboard/index.php';
    }

    private function getAdminStats() {
        return [
            'total_employees' => $this->employeeModel->getTotalCount(),
            'active_employees' => $this->employeeModel->getActiveCount(),
            'departments' => $this->departmentModel->getTotalCount(),
```

```
            'pending_leaves' => $this->leaveModel->getPendingCount(),
            'new_hires_this_month' => $this->employeeModel->getNewHiresCount(date('Y-m')),
            'attendance_today' => $this->attendanceModel->getTodayCount(),
            'avg_salary' => $this->employeeModel->getAverageSalary()
        ];
    }

    private function getEmployeeStats($userId) {
        return [
            'attendance_this_month' => $this->attendanceModel->getMonthlyCount($userId),
            'leave_balance' => $this->leaveBalanceModel->getBalance($userId),
            'pending_leaves' => $this->leaveModel->getPendingCount($userId),
            'upcoming_reviews' => $this->performanceModel->getUpcomingReviews($userId),
            'recent_activities' => $this->activityModel->getRecentActivities($userId)
        ];
    }
}
?>
```

Attendance Management Module:

Time Tracking System:

```
<?php
class AttendanceController {
    public function clockIn() {
        Authorization::requireAuth('attendance', 'create');

        $userId = $_SESSION['user_id'];
        $today = date('Y-m-d');

        // Check if already clocked in today
        if ($this->attendanceModel->hasClockedIn($userId, $today)) {
            $_SESSION['error'] = 'You have already clocked in today';
            header('Location: attendance.php');
            exit;
        }

        $data = [
            'employee_id' => $userId,
            'date' => $today,
            'check_in' => date('H:i:s'),
            'status' => 'present'
        ];

        $this->attendanceModel->create($data);
```

```
        $this->logActivity($userId, 'clock_in', 'Clocked in at ' . date('H:i:s'));

        $_SESSION['success'] = 'Successfully clocked in';
        header('Location: attendance.php');
        exit;
    }

    public function clockOut() {
        Authorization::requireAuth('attendance', 'update');

        $userId = $_SESSION['user_id'];
        $today = date('Y-m-d');

        $attendance = $this->attendanceModel->getTodayRecord($userId, $today);

        if (!$attendance || !$attendance['check_in']) {
            $_SESSION['error'] = 'You must clock in first';
            header('Location: attendance.php');
            exit;
        }

        if ($attendance['check_out']) {
            $_SESSION['error'] = 'You have already clocked out today';
            header('Location: attendance.php');
            exit;
        }

        $checkOutTime = date('H:i:s');
        $breakDuration = $this->calculateBreakDuration($attendance['check_in'], $checkOutTi

        $this->attendanceModel->updateClockOut($attendance['id'], $checkOutTime, $breakDurat

        $this->logActivity($userId, 'clock_out', 'Clocked out at ' . $checkOutTime);

        $_SESSION['success'] = 'Successfully clocked out';
        header('Location: attendance.php');
        exit;
    }

    public function report() {
        Authorization::requireAuth('attendance', 'read');

        $month = sanitize_input($_GET['month'] ?? date('Y-m'));
        $department = sanitize_input($_GET['department'] ?? '');

        if ($_SESSION['role'] === 'employee') {
```

```
            $report = $this->attendanceModel->getEmployeeReport($_SESSION['user_id'], $month
        } else {
            $report = $this->attendanceModel->getDepartmentReport($department, $month);
        }

        include 'views/attendance/report.php';
    }

    private function calculateBreakDuration($checkIn, $checkOut) {
        // Simple break calculation (can be enhanced)
        $checkInTime = strtotime($checkIn);
        $checkOutTime = strtotime($checkOut);
        $totalHours = ($checkOutTime - $checkInTime) / 3600;

        // Assume 1 hour break for shifts longer than 6 hours
        return $totalHours > 6 ? 60 : 0;
    }
}
?>
```

Leave Management Module:

Leave Request System:

```
<?php
class LeaveController {
    public function request() {
        Authorization::requireAuth('leaves', 'create');

        $userId = $_SESSION['user_id'];

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $data = $this->validateLeaveRequest($_POST);

            if (!isset($data['errors'])) {
                // Check leave balance
                $balance = $this->leaveBalanceModel->getBalance($userId, $data['leave_type_i

                if ($balance['available'] < $data['total_days']) {
                    $_SESSION['error'] = 'Insufficient leave balance';
                } else {
                    $leaveId = $this->leaveModel->create($data);

                    // Send notification to manager
                    $this->notificationModel->sendLeaveRequestNotification($leaveId);

                    $this->logActivity($userId, 'leave_requested',
```

```
                    "Leave requested: {$data['total_days']} days");

                $_SESSION['success'] = 'Leave request submitted successfully';
                header('Location: leaves.php');
                exit;
            }
        }
    }

    $leaveTypes = $this->leaveTypeModel->getAll();
    $balance = $this->leaveBalanceModel->getAllBalances($userId);
    include 'views/leaves/request.php';
}

public function approve($id) {
    Authorization::requireAuth('leaves', 'approve');

    $leave = $this->leaveModel->getById($id);

    if (!$leave || $leave['status'] !== 'pending') {
        $_SESSION['error'] = 'Invalid leave request';
        header('Location: leaves.php');
        exit;
    }

    // Check if user can approve (manager or admin)
    if (!$this->canApproveLeave($leave)) {
        header('HTTP/1.0 403 Forbidden');
        include 'views/403.php';
        exit;
    }

    $this->leaveModel->approve($id, $_SESSION['user_id']);

    // Update leave balance
    $this->leaveBalanceModel->deductLeave($leave['employee_id'],
        $leave['leave_type_id'], $leave['total_days']);

    // Send notification to employee
    $this->notificationModel->sendLeaveApprovalNotification($id);

    $this->logActivity($_SESSION['user_id'], 'leave_approved',
        "Approved leave request ID: {$id}");

    $_SESSION['success'] = 'Leave request approved';
    header('Location: leaves.php');
```

```
        exit;
    }

    private function validateLeaveRequest($data) {
        $errors = [];

        if (empty($data['leave_type_id'])) {
            $errors['leave_type_id'] = 'Please select a leave type';
        }

        if (empty($data['start_date'])) {
            $errors['start_date'] = 'Start date is required';
        }

        if (empty($data['end_date'])) {
            $errors['end_date'] = 'End date is required';
        }

        if (!empty($data['start_date']) && !empty($data['end_date'])) {
            $startDate = new DateTime($data['start_date']);
            $endDate = new DateTime($data['end_date']);
            $today = new DateTime();

            if ($startDate < $today) {
                $errors['start_date'] = 'Start date cannot be in the past';
            }

            if ($endDate < $startDate) {
                $errors['end_date'] = 'End date must be after start date';
            }

            // Calculate total days
            $interval = $startDate->diff($endDate);
            $totalDays = $interval->days + 1;
            $data['total_days'] = $totalDays;
        }

        if (empty($data['reason'])) {
            $errors['reason'] = 'Reason for leave is required';
        }

        return $errors ? ['errors' => $errors] : $data;
    }
}
?>
```

6.3 User Interface Design

The HR Management System features a modern, responsive user interface designed for usability across different devices and user roles. The UI follows Material Design principles and provides an intuitive experience for HR professionals and employees.

Responsive Design Implementation:

Mobile-First Approach:

```
/* CSS for Responsive Design */
.container-fluid {
    padding: 0;
}

.sidebar {
    min-height: 100vh;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    transition: all 0.3s ease;
}

/* Desktop Layout */
@media (min-width: 768px) {
    .sidebar {
        width: 250px;
        position: fixed;
        left: 0;
        top: 0;
        z-index: 1000;
    }

    .main-content {
        margin-left: 250px;
        padding: 20px;
    }
}

/* Mobile Layout */
@media (max-width: 767px) {
    .sidebar {
        width: 100%;
        position: fixed;
        left: -100%;
        top: 0;
        z-index: 1000;
    }
```

```
    .sidebar.active {
        left: 0;
    }

    .main-content {
        margin-left: 0;
        padding: 15px;
    }

    .mobile-menu-toggle {
        display: block;
    }
}

/* Card Components */
.card {
    border: none;
    border-radius: 10px;
    box-shadow: 0 4px 6px rgba(50, 50, 93, 0.11);
    transition: all 0.3s ease;
}

.card:hover {
    transform: translateY(-5px);
    box-shadow: 0 8px 15px rgba(50, 50, 93, 0.2);
}

.card-header {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    border-radius: 10px 10px 0 0;
    padding: 15px;
}

/* Form Styling */
.form-control {
    border: 2px solid #e0e0e0;
    border-radius: 8px;
    padding: 12px;
    transition: all 0.3s ease;
}

.form-control:focus {
    border-color: #667eea;
    box-shadow: 0 0 0 0.2rem rgba(102, 126, 234, 0.25);
```

```
}

.btn-primary {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    border: none;
    border-radius: 8px;
    padding: 12px 30px;
    font-weight: 600;
    transition: all 0.3s ease;
}

.btn-primary:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 8px rgba(102, 126, 234, 0.3);
}
```

Dashboard Interface:

```
<!-- Dashboard Layout -->
<div class="dashboard-container">
    <!-- Statistics Cards -->
    <div class="row mb-4">
        <div class="col-md-3 mb-4">
            <div class="card text-center">
                <div class="card-body">
                    <div class="stat-icon text-primary mb-3">
                        <i class="fas fa-users fa-3x"></i>
                    </div>
                    <h3 class="stat-number"><?php echo $stats['total_employees']; ?></h3>
                    <p class="stat-label">Total Employees</p>
                </div>
            </div>
        </div>

        <div class="col-md-3 mb-4">
            <div class="card text-center">
                <div class="card-body">
                    <div class="stat-icon text-success mb-3">
                        <i class="fas fa-user-check fa-3x"></i>
                    </div>
                    <h3 class="stat-number"><?php echo $stats['active_employees']; ?></h3>
                    <p class="stat-label">Active Employees</p>
                </div>
            </div>
        </div>

        <div class="col-md-3 mb-4">
```

```
            <div class="card text-center">
                <div class="card-body">
                    <div class="stat-icon text-warning mb-3">
                        <i class="fas fa-calendar-check fa-3x"></i>
                    </div>
                    <h3 class="stat-number"><?php echo $stats['pending_leaves']; ?></h3>
                    <p class="stat-label">Pending Leaves</p>
                </div>
            </div>
        </div>

        <div class="col-md-3 mb-4">
            <div class="card text-center">
                <div class="card-body">
                    <div class="stat-icon text-info mb-3">
                        <i class="fas fa-building fa-3x"></i>
                    </div>
                    <h3 class="stat-number"><?php echo $stats['departments']; ?></h3>
                    <p class="stat-label">Departments</p>
                </div>
            </div>
        </div>
    </div>

    <!-- Charts Section -->
    <div class="row">
        <div class="col-md-8 mb-4">
            <div class="card">
                <div class="card-header">
                    <h5>Employee Distribution by Department</h5>
                </div>
                <div class="card-body">
                    <canvas id="departmentChart" width="400" height="200"></canvas>
                </div>
            </div>
        </div>

        <div class="col-md-4 mb-4">
            <div class="card">
                <div class="card-header">
                    <h5>Recent Activities</h5>
                </div>
                <div class="card-body">
                    <div class="activity-feed">
                        <?php foreach ($recentActivities as $activity): ?>
                        <div class="activity-item">
```

```
                            <div class="activity-icon">
                                <i class="fas fa-circle text-<?php echo $activity['type']; ?
                            </div>
                            <div class="activity-content">
                                <p class="mb-1"><?php echo $activity['description']; ?></p>
                                <small class="text-muted"><?php echo timeAgo($activity['crea
                            </div>
                        </div>
                        <?php endforeach; ?>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>

<!-- Chart.js Integration -->
<script>
// Department Distribution Chart
const ctx = document.getElementById('departmentChart').getContext('2d');
const departmentChart = new Chart(ctx, {
    type: 'doughnut',
    data: {
        labels: <?php echo json_encode(array_column($departmentStats, 'name')); ?>,
        datasets: [{
            data: <?php echo json_encode(array_column($departmentStats, 'count')); ?>,
            backgroundColor: [
                '#667eea',
                '#764ba2',
                '#f093fb',
                '#4facfe',
                '#43e97b',
                '#fa709a',
                '#fee140'
            ],
            borderWidth: 2,
            borderColor: '#fff'
        }]
    },
    options: {
        responsive: true,
        maintainAspectRatio: false,
        plugins: {
            legend: {
                position: 'bottom'
            }
```

```
        }
    }
});
</script>
```

Employee Management Interface:

```
<!-- Employee List with Search and Filter -->
<div class="employee-management">
    <div class="row mb-4">
        <div class="col-md-6">
            <div class="input-group">
                <input type="text" class="form-control" id="searchInput"
                       placeholder="Search employees...">
                <div class="input-group-append">
                    <button class="btn btn-primary" type="button">
                        <i class="fas fa-search"></i>
                    </button>
                </div>
            </div>
        </div>

        <div class="col-md-3">
            <select class="form-control" id="departmentFilter">
                <option value="">All Departments</option>
                <?php foreach ($departments as $dept): ?>
                <option value="<?php echo $dept['id']; ?>">
                    <?php echo htmlspecialchars($dept['name']); ?>
                </option>
                <?php endforeach; ?>
            </select>
        </div>

        <div class="col-md-3">
            <button class="btn btn-success btn-block" data-toggle="modal" data-target="#addE
                <i class="fas fa-plus"></i> Add Employee
            </button>
        </div>
    </div>

    <!-- Employee Table -->
    <div class="card">
        <div class="card-body">
            <div class="table-responsive">
                <table class="table table-hover" id="employeeTable">
                    <thead>
                        <tr>
```

```
                <th>Employee ID</th>
                <th>Name</th>
                <th>Email</th>
                <th>Department</th>
                <th>Position</th>
                <th>Status</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($employees as $employee): ?>
            <tr>
                <td><?php echo htmlspecialchars($employee['employee_code']); ?><
                <td>
                    <div class="d-flex align-items-center">
                        <img src="assets/images/avatar.png" class="rounded-circl
                             width="30" height="30" alt="Avatar">
                        <?php echo htmlspecialchars($employee['first_name'] . '
                    </div>
                </td>
                <td><?php echo htmlspecialchars($employee['email']); ?></td>
                <td>
                    <span class="badge badge-info">
                        <?php echo htmlspecialchars($employee['department_name']
                    </span>
                </td>
                <td><?php echo htmlspecialchars($employee['position']); ?></td>
                <td>
                    <span class="badge badge-<?php echo $employee['status'] ===
                        <?php echo ucfirst($employee['status']); ?>
                    </span>
                </td>
                <td>
                    <div class="btn-group">
                        <button class="btn btn-sm btn-primary" onclick="viewEmpl
                            <i class="fas fa-eye"></i>
                        </button>
                        <button class="btn btn-sm btn-warning" onclick="editEmpl
                            <i class="fas fa-edit"></i>
                        </button>
                        <button class="btn btn-sm btn-danger" onclick="deleteEmp
                            <i class="fas fa-trash"></i>
                        </button>
                    </div>
                </td>
            </tr>
```

```
                        <?php endforeach; ?>
                    </tbody>
                </table>
            </div>

            <!-- Pagination -->
            <nav aria-label="Employee pagination">
                <ul class="pagination justify-content-center">
                    <?php for ($i = 1; $i <= $totalPages; $i++): ?>
                    <li class="page-item <?php echo $i === $currentPage ? 'active' : ''; ?>'
                        <a class="page-link" href="?page=<?php echo $i; ?>">
                            <?php echo $i; ?>
                        </a>
                    </li>
                    <?php endfor; ?>
                </ul>
            </nav>
        </div>
    </div>
</div>

<!-- Add Employee Modal -->
<div class="modal fade" id="addEmployeeModal" tabindex="-1">
    <div class="modal-dialog modal-lg">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Add New Employee</h5>
                <button type="button" class="close" data-dismiss="modal">
                    <span>&times;</span>
                </button>
            </div>
            <div class="modal-body">
                <form id="addEmployeeForm">
                    <div class="row">
                        <div class="col-md-6">
                            <div class="form-group">
                                <label for="firstName">First Name</label>
                                <input type="text" class="form-control" id="firstName" name=
                            </div>
                        </div>
                        <div class="col-md-6">
                            <div class="form-group">
                                <label for="lastName">Last Name</label>
                                <input type="text" class="form-control" id="lastName" name='
                            </div>
                        </div>
```

```
            </div>

            <div class="form-group">
                <label for="email">Email</label>
                <input type="email" class="form-control" id="email" name="email" rec
            </div>

            <div class="row">
                <div class="col-md-6">
                    <div class="form-group">
                        <label for="department">Department</label>
                        <select class="form-control" id="department" name="departmen
                            <option value="">Select Department</option>
                            <?php foreach ($departments as $dept): ?>
                            <option value="<?php echo $dept['id']; ?>">
                                <?php echo htmlspecialchars($dept['name']); ?>
                            </option>
                            <?php endforeach; ?>
                        </select>
                    </div>
                </div>
                <div class="col-md-6">
                    <div class="form-group">
                        <label for="position">Position</label>
                        <input type="text" class="form-control" id="position" name='
                    </div>
                </div>
            </div>

            <div class="row">
                <div class="col-md-6">
                    <div class="form-group">
                        <label for="salary">Salary</label>
                        <input type="number" class="form-control" id="salary" name='
                               step="0.01" min="0" required>
                    </div>
                </div>
                <div class="col-md-6">
                    <div class="form-group">
                        <label for="hireDate">Hire Date</label>
                        <input type="date" class="form-control" id="hireDate" name='
                    </div>
                </div>
            </div>
        </form>
    </div>
```

```
            <div class="modal-footer">
                <button type="button" class="btn btn-secondary" data-dismiss="modal">Cancel
                <button type="button" class="btn btn-primary" onclick="saveEmployee()">Save
            </div>
        </div>
    </div>
</div>
```

6.4 System Integration

The HR Management System integrates various components and external services to provide comprehensive functionality. Integration includes database connectivity, file management, email notifications, and reporting capabilities.

Database Integration:

```php
<?php
// Database Connection Class
class Database {
    private static $instance = null;
    private $connection;

    private function __construct() {
        $this->connect();
    }

    public static function getInstance() {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    private function connect() {
        try {
            $dsn = "mysql:host=" . DB_HOST . ";dbname=" . DB_NAME . ";charset=utf8mb4";
            $this->connection = new PDO($dsn, DB_USER, DB_PASSWORD, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false
            ]);
        } catch (PDOException $e) {
            error_log("Database connection failed: " . $e->getMessage());
            throw new Exception("Database connection failed");
        }
    }
```

```php
    public function getConnection() {
        return $this->connection;
    }

    public function query($sql, $params = []) {
        try {
            $stmt = $this->connection->prepare($sql);
            $stmt->execute($params);
            return $stmt;
        } catch (PDOException $e) {
            error_log("Query failed: " . $e->getMessage());
            throw new Exception("Database query failed");
        }
    }

    public function beginTransaction() {
        return $this->connection->beginTransaction();
    }

    public function commit() {
        return $this->connection->commit();
    }

    public function rollback() {
        return $this->connection->rollback();
    }
}

// Repository Pattern Implementation
class EmployeeRepository {
    private $db;

    public function __construct() {
        $this->db = Database::getInstance();
    }

    public function findById($id) {
        $stmt = $this->db->query("
            SELECT e.*, d.name as department_name
            FROM employees e
            LEFT JOIN departments d ON e.department_id = d.id
            WHERE e.id = ?
        ", [$id]);

        return $stmt->fetch();
    }
```

```php
public function create($data) {
    $sql = "
        INSERT INTO employees (first_name, last_name, email, department_id,
                               position, salary, hire_date, employee_code)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    ";

    $params = [
        $data['first_name'],
        $data['last_name'],
        $data['email'],
        $data['department_id'],
        $data['position'],
        $data['salary'],
        $data['hire_date'],
        $this->generateEmployeeCode()
    ];

    $this->db->query($sql, $params);
    return $this->db->getConnection()->lastInsertId();
}

public function update($id, $data) {
    $sql = "
        UPDATE employees
        SET first_name = ?, last_name = ?, email = ?, department_id = ?,
            position = ?, salary = ?, updated_at = CURRENT_TIMESTAMP
        WHERE id = ?
    ";

    $params = [
        $data['first_name'],
        $data['last_name'],
        $data['email'],
        $data['department_id'],
        $data['position'],
        $data['salary'],
        $id
    ];

    return $this->db->query($sql, $params);
}

private function generateEmployeeCode() {
    return 'EMP' . date('Y') . str_pad(mt_rand(1, 9999), 4, '0', STR_PAD_LEFT);
```

```
    }
}
?>
```

File Management Integration:

```
<?php
class FileManager {
    private $uploadPath;
    private $allowedTypes = ['jpg', 'jpeg', 'png', 'pdf', 'doc', 'docx'];
    private $maxFileSize = 5 * 1024 * 1024; // 5MB

    public function __construct() {
        $this->uploadPath = UPLOAD_PATH . '/';
    }

    public function uploadFile($file, $category = 'documents') {
        // Validate file
        if (!$this->validateFile($file)) {
            return false;
        }

        // Generate unique filename
        $extension = pathinfo($file['name'], PATHINFO_EXTENSION);
        $filename = uniqid() . '.' . $extension;
        $categoryPath = $this->uploadPath . $category;

        // Create directory if it doesn't exist
        if (!is_dir($categoryPath)) {
            mkdir($categoryPath, 0755, true);
        }

        $uploadPath = $categoryPath . '/' . $filename;

        if (move_uploaded_file($file['tmp_name'], $uploadPath)) {
            return [
                'success' => true,
                'filename' => $filename,
                'path' => $category . '/' . $filename,
                'size' => $file['size']
            ];
        }

        return ['success' => false, 'error' => 'Failed to upload file'];
    }

    private function validateFile($file) {
```

```php
        // Check if file was uploaded
        if ($file['error'] !== UPLOAD_ERR_OK) {
            return false;
        }

        // Check file size
        if ($file['size'] > $this->maxFileSize) {
            return false;
        }

        // Check file type
        $extension = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
        if (!in_array($extension, $this->allowedTypes)) {
            return false;
        }

        // Check if it's actually an image (for image files)
        if (in_array($extension, ['jpg', 'jpeg', 'png'])) {
            $imageInfo = getimagesize($file['tmp_name']);
            if ($imageInfo === false) {
                return false;
            }
        }

        return true;
    }

    public function deleteFile($path) {
        $fullPath = $this->uploadPath . $path;
        if (file_exists($fullPath)) {
            return unlink($fullPath);
        }
        return false;
    }

    public function getFileUrl($path) {
        return BASE_URL . '/uploads/' . $path;
    }
}

// Usage in Employee Controller
class EmployeeController {
    private $fileManager;

    public function __construct() {
        $this->fileManager = new FileManager();
```

```php
    }

    public function uploadProfilePicture($employeeId) {
        if (isset($_FILES['profile_picture'])) {
            $result = $this->fileManager->uploadFile($_FILES['profile_picture'], 'profiles')

            if ($result['success']) {
                // Update employee record with profile picture path
                $this->employeeModel->updateProfilePicture($employeeId, $result['path']);

                $_SESSION['success'] = 'Profile picture uploaded successfully';
            } else {
                $_SESSION['error'] = 'Failed to upload profile picture';
            }
        }

        header('Location: employee.php?id=' . $employeeId);
        exit;
    }
}
?>
```

Email Notification Integration:

```php
<?php
class EmailService {
    private $mailer;

    public function __construct() {
        $this->mailer = new PHPMailer(true);
        $this->configureMailer();
    }

    private function configureMailer() {
        // SMTP Configuration
        $this->mailer->isSMTP();
        $this->mailer->Host = SMTP_HOST;
        $this->mailer->SMTPAuth = true;
        $this->mailer->Username = SMTP_USERNAME;
        $this->mailer->Password = SMTP_PASSWORD;
        $this->mailer->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
        $this->mailer->Port = SMTP_PORT;

        // Default settings
        $this->mailer->setFrom(FROM_EMAIL, FROM_NAME);
        $this->mailer->isHTML(true);
    }
```

```php
public function sendLeaveNotification($leave, $employee, $manager) {
    try {
        $this->mailer->addAddress($manager['email'], $manager['name']);
        $this->mailer->Subject = 'Leave Request: ' . $employee['name'];

        $this->mailer->Body = $this->getLeaveEmailTemplate($leave, $employee);

        return $this->mailer->send();
    } catch (Exception $e) {
        error_log("Email sending failed: " . $e->getMessage());
        return false;
    }
}

public function sendLeaveApprovalNotification($leave, $employee) {
    try {
        $this->mailer->addAddress($employee['email'], $employee['name']);
        $this->mailer->Subject = 'Leave Request Approved';

        $this->mailer->Body = $this->getLeaveApprovalEmailTemplate($leave, $employee);

        return $this->mailer->send();
    } catch (Exception $e) {
        error_log("Email sending failed: " . $e->getMessage());
        return false;
    }
}

private function getLeaveEmailTemplate($leave, $employee) {
    return "
    <h2>Leave Request Notification</h2>
    <p>Dear Manager,</p>
    <p><strong>{$employee['name']}</strong> has requested leave:</p>
    <ul>
        <li><strong>Leave Type:</strong> {$leave['type']}</li>
        <li><strong>Duration:</strong> {$leave['start_date']} to {$leave['end_date']}</l
        <li><strong>Total Days:</strong> {$leave['total_days']}</li>
        <li><strong>Reason:</strong> {$leave['reason']}</li>
    </ul>
    <p>Please review and approve or reject this request in the HR Management System.</p>
    <p><a href='" . BASE_URL . "/leaves.php' class='btn btn-primary'>Review Leave Reques
    <p>Thank you,<br>HR Management System</p>
    ";
}
```

```
    private function getLeaveApprovalEmailTemplate($leave, $employee) {
        return "
        <h2>Leave Request Approved</h2>
        <p>Dear {$employee['name']},</p>
        <p>Your leave request has been approved:</p>
        <ul>
            <li><strong>Leave Type:</strong> {$leave['type']}</li>
            <li><strong>Duration:</strong> {$leave['start_date']} to {$leave['end_date']}</l
            <li><strong>Total Days:</strong> {$leave['total_days']}</li>
        </ul>
        <p>Have a great time off!</p>
        <p>Best regards,<br>HR Department</p>
        ";
    }
}

// Notification Service
class NotificationService {
    private $emailService;

    public function __construct() {
        $this->emailService = new EmailService();
    }

    public function sendLeaveRequestNotification($leaveId) {
        $leave = $this->leaveModel->getById($leaveId);
        $employee = $this->employeeModel->getById($leave['employee_id']);
        $manager = $this->employeeModel->getById($employee['manager_id']);

        if ($manager) {
            $this->emailService->sendLeaveNotification($leave, $employee, $manager);
        }

        // Also create in-app notification
        $this->createNotification($manager['id'], 'leave_request',
            "New leave request from {$employee['name']}", $leaveId);
    }

    public function sendLeaveApprovalNotification($leaveId) {
        $leave = $this->leaveModel->getById($leaveId);
        $employee = $this->employeeModel->getById($leave['employee_id']);

        $this->emailService->sendLeaveApprovalNotification($leave, $employee);

        // Create in-app notification
        $this->createNotification($employee['id'], 'leave_approved',
```

```
            "Your leave request has been approved", $leaveId);
    }

    private function createNotification($userId, $type, $message, $relatedId = null) {
        $stmt = $this->db->prepare("
            INSERT INTO notifications (user_id, type, message, related_id, created_at)
            VALUES (?, ?, ?, ?, CURRENT_TIMESTAMP)
        ");

        return $stmt->execute([$userId, $type, $message, $relatedId]);
    }
}
?>
```

This comprehensive implementation of the HR Management System demonstrates modern web development practices, including modular architecture, responsive design, database integration, file management, and email notifications. The system is designed to be scalable, maintainable, and user-friendly while meeting all functional requirements for human resource management.

CHAPTER 7

SYSTEM TESTING

7.1 Introduction to System Testing

System testing is a critical phase in the software development lifecycle that ensures the HR Management System meets all specified requirements, functions correctly, and is free from defects. Comprehensive testing validates the system's reliability, performance, security, and usability before deployment to production environment.

Purpose of System Testing: - Verify that all system requirements are met - Ensure system functionality works as expected - Identify and fix defects before deployment - Validate system performance under various conditions - Ensure security measures protect sensitive data - Verify system usability for different user roles - Confirm system integration with external services

Testing Objectives: - **Functional Testing:** Verify all features work according to specifications - **Performance Testing:** Ensure system responds within acceptable time limits - **Security Testing:** Validate security controls and data protection - **Usability Testing:** Confirm system is user-friendly and intuitive - **Compatibility Testing:** Ensure system works across different browsers and devices - **Integration Testing:** Verify proper integration with external systems and services

Testing Environment: - **Development Environment:** Local development

setup for initial testing - **Staging Environment:** Production-like environment for final validation - **Testing Database:** Separate database instance for testing purposes - **Test Data:** Anonymized sample data for comprehensive testing - **Testing Tools:** Automated testing frameworks and manual testing procedures

7.2 Testing Methodology

The HR Management System follows a comprehensive testing methodology that combines automated and manual testing approaches to ensure thorough validation of all system components and functionalities.

Testing Levels:

Unit Testing:

```php
<?php
// Example Unit Test for Employee Model
class EmployeeModelTest extends PHPUnit\Framework\TestCase {
    private $employeeModel;
    private $db;

    protected function setUp(): void {
        $this->db = $this->createMockDatabase();
        $this->employeeModel = new EmployeeModel($this->db);
    }

    public function testCreateEmployee() {
        // Arrange
        $employeeData = [
            'first_name' => 'John',
            'last_name' => 'Doe',
            'email' => 'john.doe@example.com',
            'department_id' => 1,
            'salary' => 50000.00,
            'hire_date' => '2024-01-15'
        ];

        // Act
        $result = $this->employeeModel->create($employeeData);

        // Assert
        $this->assertTrue($result);
        $this->assertIsInt($result);
    }

    public function testCreateEmployeeWithInvalidEmail() {
        // Arrange
```

```php
        $employeeData = [
            'first_name' => 'John',
            'last_name' => 'Doe',
            'email' => 'invalid-email',
            'department_id' => 1,
            'salary' => 50000.00,
            'hire_date' => '2024-01-15'
        ];

        // Act & Assert
        $this->expectException(InvalidArgumentException::class);
        $this->employeeModel->create($employeeData);
    }

    public function testGetEmployeeById() {
        // Arrange
        $employeeId = 1;
        $expectedEmployee = [
            'id' => 1,
            'first_name' => 'John',
            'last_name' => 'Doe',
            'email' => 'john.doe@example.com'
        ];

        $this->db->expects($this->once())
            ->method('query')
            ->willReturn($this->createMockStatement($expectedEmployee));

        // Act
        $result = $this->employeeModel->getById($employeeId);

        // Assert
        $this->assertEquals($expectedEmployee, $result);
    }

    private function createMockDatabase() {
        return $this->createMock(PDO::class);
    }

    private function createMockStatement($data) {
        $stmt = $this->createMock(PDOStatement::class);
        $stmt->method('fetch')->willReturn($data);
        return $stmt;
    }
}
?>
```

Integration Testing:

```php
<?php
// Example Integration Test for Employee Registration
class EmployeeRegistrationTest extends PHPUnit\Framework\TestCase {
    private $httpClient;
    private $testDatabase;

    protected function setUp(): void {
        $this->httpClient = new GuzzleHttp\Client();
        $this->testDatabase = new Database();
        $this->testDatabase->connect('test_hrms_db');
    }

    public function testEmployeeRegistrationFlow() {
        // Test complete employee registration process
        $registrationData = [
            'first_name' => 'Jane',
            'last_name' => 'Smith',
            'email' => 'jane.smith@example.com',
            'department_id' => 1,
            'position' => 'Software Developer',
            'salary' => 60000.00,
            'hire_date' => '2024-01-15'
        ];

        // Send registration request
        $response = $this->httpClient->post('http://localhost/hrms/employees.php', [
            'form_params' => $registrationData
        ]);

        // Assert successful response
        $this->assertEquals(200, $response->getStatusCode());

        // Verify employee was created in database
        $employee = $this->testDatabase->query(
            "SELECT * FROM employees WHERE email = ?",
            [$registrationData['email']]
        )->fetch();

        $this->assertNotNull($employee);
        $this->assertEquals($registrationData['first_name'], $employee['first_name']);
        $this->assertEquals($registrationData['last_name'], $employee['last_name']);

        // Verify leave balance was initialized
        $leaveBalance = $this->testDatabase->query(
```

```
            "SELECT * FROM leave_balance WHERE employee_id = ?",
            [$employee['id']]
        )->fetch();

        $this->assertNotNull($leaveBalance);
    }

    public function testEmployeeRegistrationWithDuplicateEmail() {
        // Create employee with existing email
        $existingEmail = 'existing@example.com';
        $this->testDatabase->query(
            "INSERT INTO employees (first_name, last_name, email, department_id, hire_date,
             VALUES ('Test', 'User', ?, 1, '2024-01-01', 50000)",
            [$existingEmail]
        );

        $registrationData = [
            'first_name' => 'New',
            'last_name' => 'User',
            'email' => $existingEmail, // Duplicate email
            'department_id' => 1,
            'position' => 'Software Developer',
            'salary' => 60000.00,
            'hire_date' => '2024-01-15'
        ];

        // Send registration request
        $response = $this->httpClient->post('http://localhost/hrms/employees.php', [
            'form_params' => $registrationData
        ]);

        // Assert error response
        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('Email already exists', (string)$response->getBody
    }

    protected function tearDown(): void {
        // Clean up test data
        $this->testDatabase->query("DELETE FROM employees WHERE email LIKE '%@example.com'")
    }
}
?>
```

System Testing:

```
<?php
// Example System Test for Leave Management
```

```php
class LeaveManagementSystemTest extends PHPUnit\Framework\TestCase {
    private $httpClient;
    private $testUsers;

    protected function setUp(): void {
        $this->httpClient = new GuzzleHttp\Client();
        $this->testUsers = [
            'employee' => ['username' => 'employee1', 'password' => 'password123'],
            'manager' => ['username' => 'manager1', 'password' => 'password123'],
            'admin' => ['username' => 'admin1', 'password' => 'password123']
        ];
    }

    public function testCompleteLeaveRequestWorkflow() {
        // Step 1: Employee logs in
        $loginResponse = $this->loginUser('employee');
        $this->assertEquals(200, $loginResponse->getStatusCode());

        // Step 2: Employee requests leave
        $leaveData = [
            'leave_type_id' => 1,
            'start_date' => '2024-02-01',
            'end_date' => '2024-02-05',
            'reason' => 'Personal vacation'
        ];

        $leaveResponse = $this->httpClient->post('http://localhost/hrms/leaves.php', [
            'form_params' => $leaveData
        ]);

        $this->assertEquals(200, $leaveResponse->getStatusCode());

        // Step 3: Manager logs in
        $managerLogin = $this->loginUser('manager');
        $this->assertEquals(200, $managerLogin->getStatusCode());

        // Step 4: Manager approves leave
        $approvalResponse = $this->httpClient->post('http://localhost/hrms/leaves.php?action
        $this->assertEquals(200, $approvalResponse->getStatusCode());

        // Step 5: Verify email notification was sent
        $this->assertTrue($this->checkEmailNotificationSent());

        // Step 6: Verify leave balance was updated
        $this->assertTrue($this->verifyLeaveBalanceUpdated());
    }
```

```php
    private function loginUser($userType) {
        $credentials = $this->testUsers[$userType];
        return $this->httpClient->post('http://localhost/hrms/login.php', [
            'form_params' => $credentials
        ]);
    }

    private function checkEmailNotificationSent() {
        // Check email log or mock email service
        return true; // Simplified for example
    }

    private function verifyLeaveBalanceUpdated() {
        // Verify database was updated correctly
        return true; // Simplified for example
    }
}
?>
```

7.3 Functional Testing

Functional testing verifies that the HR Management System's features work according to specified requirements. This type of testing focuses on the system's behavior from the user's perspective.

Authentication Module Testing:

Test Cases:

```php
<?php
class AuthenticationTest extends PHPUnit\Framework\TestCase {

    public function testValidLogin() {
        $credentials = [
            'username' => 'admin',
            'password' => 'admin123'
        ];

        $response = $this->post('/login.php', $credentials);

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('dashboard', $response->getHeaderLine('Location'))
    }

    public function testInvalidLogin() {
        $credentials = [
```

```
            'username' => 'admin',
            'password' => 'wrongpassword'
        ];

        $response = $this->post('/login.php', $credentials);

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('Invalid username or password', $response->getBody
    }

    public function testSessionManagement() {
        // Login
        $this->loginUser('admin');

        // Access protected page
        $response = $this->get('/dashboard.php');
        $this->assertEquals(200, $response->getStatusCode());

        // Logout
        $response = $this->get('/logout.php');
        $this->assertEquals(200, $response->getStatusCode());

        // Try to access protected page after logout
        $response = $this->get('/dashboard.php');
        $this->assertEquals(302, $response->getStatusCode());
        $this->assertStringContainsString('login.php', $response->getHeaderLine('Location'))
    }

    public function testRoleBasedAccess() {
        // Login as employee
        $this->loginUser('employee');

        // Try to access admin-only page
        $response = $this->get('/admin/settings.php');
        $this->assertEquals(403, $response->getStatusCode());

        // Login as admin
        $this->loginUser('admin');

        // Access admin page
        $response = $this->get('/admin/settings.php');
        $this->assertEquals(200, $response->getStatusCode());
    }
}
?>
```

Employee Management Testing:

Test Cases:

```php
<?php
class EmployeeManagementTest extends PHPUnit\Framework\TestCase {

    public function testCreateEmployeeWithValidData() {
        $employeeData = [
            'first_name' => 'John',
            'last_name' => 'Doe',
            'email' => 'john.doe@example.com',
            'department_id' => 1,
            'position' => 'Software Developer',
            'salary' => 50000.00,
            'hire_date' => '2024-01-15'
        ];

        $response = $this->post('/employees.php', $employeeData);

        $this->assertEquals(302, $response->getStatusCode());
        $this->assertStringContainsString('employees.php?success=1', $response->getHeaderLin

        // Verify employee was created
        $employee = $this->getEmployeeByEmail('john.doe@example.com');
        $this->assertNotNull($employee);
        $this->assertEquals('John', $employee['first_name']);
    }

    public function testCreateEmployeeWithInvalidEmail() {
        $employeeData = [
            'first_name' => 'John',
            'last_name' => 'Doe',
            'email' => 'invalid-email',
            'department_id' => 1,
            'position' => 'Software Developer',
            'salary' => 50000.00,
            'hire_date' => '2024-01-15'
        ];

        $response = $this->post('/employees.php', $employeeData);

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('Valid email is required', $response->getBody());
    }

    public function testUpdateEmployee() {
```

```
        // Create employee first
        $employeeId = $this->createTestEmployee();

        $updateData = [
            'first_name' => 'John Updated',
            'last_name' => 'Doe Updated',
            'email' => 'john.updated@example.com',
            'department_id' => 2,
            'position' => 'Senior Developer',
            'salary' => 55000.00
        ];

        $response = $this->post("/employees.php?action=edit&id={$employeeId}", $updateData);

        $this->assertEquals(302, $response->getStatusCode());

        // Verify employee was updated
        $employee = $this->getEmployeeById($employeeId);
        $this->assertEquals('John Updated', $employee['first_name']);
        $this->assertEquals(55000.00, $employee['salary']);
    }

    public function testDeleteEmployee() {
        // Create employee first
        $employeeId = $this->createTestEmployee();

        $response = $this->post("/employees.php?action=delete&id={$employeeId}", []);

        $this->assertEquals(302, $response->getStatusCode());

        // Verify employee was deleted
        $employee = $this->getEmployeeById($employeeId);
        $this->assertNull($employee);
    }

    public function testEmployeeSearch() {
        // Create test employees
        $this->createTestEmployee(['name' => 'Alice Johnson']);
        $this->createTestEmployee(['name' => 'Bob Smith']);

        // Search for Alice
        $response = $this->get('/employees.php?search=Alice');

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('Alice Johnson', $response->getBody());
        $this->assertStringNotContainsString('Bob Smith', $response->getBody());
```

```
    }
}
?>
```

Attendance Management Testing:

Test Cases:

```
<?php
class AttendanceTest extends PHPUnit\Framework\TestCase {

    public function testClockIn() {
        $this->loginUser('employee');

        $clockInData = [
            'action' => 'clock_in'
        ];

        $response = $this->post('/attendance.php', $clockInData);

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('Successfully clocked in', $response->getBody());

        // Verify attendance record was created
        $attendance = $this->getTodayAttendance($this->getCurrentUserId());
        $this->assertNotNull($attendance);
        $this->assertNotNull($attendance['check_in']);
        $this->assertNull($attendance['check_out']);
    }

    public function testClockInTwiceSameDay() {
        $this->loginUser('employee');

        // Clock in first time
        $this->clockIn();

        // Try to clock in again
        $response = $this->post('/attendance.php', ['action' => 'clock_in']);

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('already clocked in today', $response->getBody());
    }

    public function testClockOut() {
        $this->loginUser('employee');

        // Clock in first
```

```
        $this->clockIn();

        $clockOutData = [
            'action' => 'clock_out'
        ];

        $response = $this->post('/attendance.php', $clockOutData);

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('Successfully clocked out', $response->getBody());

        // Verify attendance record was updated
        $attendance = $this->getTodayAttendance($this->getCurrentUserId());
        $this->assertNotNull($attendance['check_out']);
        $this->assertTrue($attendance['check_out'] > $attendance['check_in']);
    }

    public function testClockOutWithoutClockIn() {
        $this->loginUser('employee');

        $response = $this->post('/attendance.php', ['action' => 'clock_out']);

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('must clock in first', $response->getBody());
    }

    public function testAttendanceReport() {
        $this->loginUser('manager');

        $response = $this->get('/attendance.php?action=report&month=2024-01');

        $this->assertEquals(200, $response->getStatusCode());
        $this->assertStringContainsString('Attendance Report', $response->getBody());
        $this->assertStringContainsString('January 2024', $response->getBody());
    }
}
?>
```

7.4 Performance Testing

Performance testing ensures the HR Management System meets performance requirements and can handle expected user loads without degradation in response times or system stability.

Load Testing:

Test Scenarios:

```php
<?php
class PerformanceTest extends PHPUnit\Framework\TestCase {

    public function testConcurrentUserLogin() {
        $concurrentUsers = 50;
        $successCount = 0;
        $responseTimes = [];

        for ($i = 0; $i < $concurrentUsers; $i++) {
            $startTime = microtime(true);

            $response = $this->httpClient->post('http://localhost/hrms/login.php', [
                'form_params' => [
                    'username' => 'testuser' . $i,
                    'password' => 'password123'
                ],
                'timeout' => 10
            ]);

            $endTime = microtime(true);
            $responseTime = ($endTime - $startTime) * 1000; // Convert to milliseconds

            $responseTimes[] = $responseTime;

            if ($response->getStatusCode() === 200) {
                $successCount++;
            }
        }

        // Assert performance metrics
        $this->assertGreaterThan($concurrentUsers * 0.95, $successCount); // 95% success ra
        $this->assertLessThan(2000, array_sum($responseTimes) / count($responseTimes)); // A
        $this->assertLessThan(5000, max($responseTimes)); // Max < 5 seconds
    }

    public function testEmployeeSearchPerformance() {
        $searchTerms = ['John', 'Mary', 'Development', 'HR', 'Manager'];
        $responseTimes = [];

        foreach ($searchTerms as $term) {
            $startTime = microtime(true);

            $response = $this->httpClient->get('http://localhost/hrms/employees.php?search=

            $endTime = microtime(true);
            $responseTime = ($endTime - $startTime) * 1000;
```

```
        $responseTimes[] = $responseTime;

        $this->assertEquals(200, $response->getStatusCode());
    }

    // Assert search performance
    $this->assertLessThan(1000, array_sum($responseTimes) / count($responseTimes)); // A
    $this->assertLessThan(2000, max($responseTimes)); // Max < 2 seconds
}

public function testDatabaseQueryPerformance() {
    $queries = [
        'SELECT COUNT(*) FROM employees',
        'SELECT * FROM employees WHERE department_id = 1',
        'SELECT e.*, d.name FROM employees e JOIN departments d ON e.department_id = d.i
        'SELECT * FROM attendance WHERE date >= CURDATE() - INTERVAL 30 DAY'
    ];

    foreach ($queries as $sql) {
        $iterations = 100;
        $totalTime = 0;

        for ($i = 0; $i < $iterations; $i++) {
            $startTime = microtime(true);

            $this->db->query($sql);

            $endTime = microtime(true);
            $totalTime += ($endTime - $startTime);
        }

        $averageTime = ($totalTime / $iterations) * 1000; // Convert to milliseconds

        // Assert query performance
        $this->assertLessThan(100, $averageTime); // Average < 100ms
    }
}

public function testPageLoadPerformance() {
    $pages = [
        '/dashboard.php',
        '/employees.php',
        '/attendance.php',
        '/leaves.php',
        '/reports.php'
    ];
```

```php
        foreach ($pages as $page) {
            $responseTimes = [];

            for ($i = 0; $i < 10; $i++) {
                $startTime = microtime(true);

                $response = $this->httpClient->get('http://localhost/hrms' . $page);

                $endTime = microtime(true);
                $responseTimes[] = ($endTime - $startTime) * 1000;

                $this->assertEquals(200, $response->getStatusCode());
            }

            $averageTime = array_sum($responseTimes) / count($responseTimes);

            // Assert page load performance
            $this->assertLessThan(1500, $averageTime); // Average < 1.5 seconds
        }
    }
}
?>
```

Stress Testing:

```php
<?php
class StressTest extends PHPUnit\Framework\TestCase {

    public function testHighVolumeDataOperations() {
        $operations = [
            'create_employees' => 1000,
            'update_employees' => 500,
            'delete_employees' => 100,
            'attendance_records' => 10000,
            'leave_requests' => 500
        ];

        $startTime = microtime(true);
        $memoryBefore = memory_get_usage();

        // Perform stress operations
        $this->performStressOperations($operations);

        $endTime = microtime(true);
        $memoryAfter = memory_get_usage();
```

```
        $totalTime = ($endTime - $startTime);
        $memoryUsed = ($memoryAfter - $memoryBefore) / 1024 / 1024; // MB

        // Assert system stability under stress
        $this->assertLessThan(300, $totalTime); // Complete within 5 minutes
        $this->assertLessThan(100, $memoryUsed); // Use less than 100MB additional memory
    }

    public function testMemoryLeakDetection() {
        $initialMemory = memory_get_usage();

        // Perform repeated operations
        for ($i = 0; $i < 1000; $i++) {
            $this->performEmployeeSearch('test');
            $this->performLeaveRequest();
            $this->performAttendanceCheck();
        }

        $finalMemory = memory_get_usage();
        $memoryIncrease = ($finalMemory - $initialMemory) / 1024 / 1024;

        // Assert no significant memory leak
        $this->assertLessThan(50, $memoryIncrease); // Less than 50MB increase
    }

    private function performStressOperations($operations) {
        // Create employees
        for ($i = 0; $i < $operations['create_employees']; $i++) {
            $this->createTestEmployee(['index' => $i]);
        }

        // Update employees
        for ($i = 0; $i < $operations['update_employees']; $i++) {
            $this->updateTestEmployee($i);
        }

        // Delete employees
        for ($i = 0; $i < $operations['delete_employees']; $i++) {
            $this->deleteTestEmployee($i);
        }

        // Create attendance records
        for ($i = 0; $i < $operations['attendance_records']; $i++) {
            $this->createAttendanceRecord($i);
        }
```

```
        // Create leave requests
        for ($i = 0; $i < $operations['leave_requests']; $i++) {
            $this->createLeaveRequest($i);
        }
    }
}
?>
```

7.5 Security Testing

Security testing ensures the HR Management System protects sensitive data, prevents unauthorized access, and is resistant to common security vulnerabilities.

Authentication Security Testing:

```
<?php
class SecurityTest extends PHPUnit\Framework\TestCase {

    public function testSQLInjectionPrevention() {
        $maliciousInputs = [
            "' OR '1'='1",
            "'; DROP TABLE employees; --",
            "' UNION SELECT username, password FROM admins --",
            "1' AND (SELECT COUNT(*) FROM employees) > 0 --"
        ];

        foreach ($maliciousInputs as $input) {
            $response = $this->post('/login.php', [
                'username' => $input,
                'password' => 'password123'
            ]);

            // Should not allow SQL injection
            $this->assertEquals(200, $response->getStatusCode());
            $this->assertStringContainsString('Invalid username or password', $response->get
        }
    }

    public function testXSSPrevention() {
        $xssPayloads = [
            '<script>alert("XSS")</script>',
            '<img src="x" onerror="alert(\'XSS\')">',
            'javascript:alert("XSS")',
            '<svg onload="alert(\'XSS\')">'
        ];
```

```
        foreach ($xssPayloads as $payload) {
            // Test XSS in employee creation
            $response = $this->post('/employees.php', [
                'first_name' => $payload,
                'last_name' => 'Test',
                'email' => 'test@example.com',
                'department_id' => 1,
                'position' => 'Test',
                'salary' => 50000,
                'hire_date' => '2024-01-15'
            ]);

            // Verify XSS is not executed
            $this->assertStringNotContainsString('<script>', $response->getBody());
            $this->assertStringNotContainsString('javascript:', $response->getBody());
            $this->assertStringContainsString('&lt;', $response->getBody()); // Should be es
        }
    }

    public function testCSRFProtection() {
        // Test without CSRF token
        $response = $this->post('/employees.php', [
            'first_name' => 'Test',
            'last_name' => 'User',
            'email' => 'test@example.com',
            'department_id' => 1,
            'position' => 'Test',
            'salary' => 50000,
            'hire_date' => '2024-01-15'
        ]);

        // Should require CSRF token
        $this->assertEquals(403, $response->getStatusCode());
        $this->assertStringContainsString('CSRF', $response->getBody());
    }

    public function testSessionSecurity() {
        // Test session fixation
        $sessionId = 'test_session_id';

        // Set session ID
        $response = $this->get('/login.php', [], [
            'cookies' => ['PHPSESSID' => $sessionId]
        ]);

        // Login and get new session
```

```php
    $this->post('/login.php', [
        'username' => 'admin',
        'password' => 'admin123'
    ]);

    // Verify session ID changed
    $newSessionId = $this->extractSessionId($response);
    $this->assertNotEquals($sessionId, $newSessionId);
}

public function testInputValidation() {
    $invalidInputs = [
        'email' => 'not-an-email',
        'salary' => '-50000',
        'hire_date' => '2025-01-01', // Future date
        'phone' => 'abc123' // Invalid characters
    ];

    foreach ($invalidInputs as $field => $value) {
        $response = $this->post('/employees.php', [
            'first_name' => 'Test',
            'last_name' => 'User',
            'email' => $field === 'email' ? $value : 'test@example.com',
            'salary' => $field === 'salary' ? $value : 50000,
            'hire_date' => $field === 'hire_date' ? $value : '2024-01-15',
            'phone' => $field === 'phone' ? $value : '123-456-7890',
            'department_id' => 1,
            'position' => 'Test'
        ]);

        // Should show validation error
        $this->assertStringContainsString('error', $response->getBody());
    }
}

public function testFileUploadSecurity() {
    $maliciousFiles = [
        [
            'name' => 'malicious.php',
            'type' => 'application/x-php',
            'tmp_name' => '/tmp/malicious.php',
            'error' => UPLOAD_ERR_OK,
            'size' => 1000
        ],
        [
            'name' => 'huge-file.jpg',
```

```
                'type' => 'image/jpeg',
                'tmp_name' => '/tmp/huge-file.jpg',
                'error' => UPLOAD_ERR_OK,
                'size' => 100 * 1024 * 1024 // 100MB
            ]
        ];

        foreach ($maliciousFiles as $file) {
            $response = $this->post('/employees.php?action=upload', [
                'multipart' => [
                    'profile_picture' => $file
                ]
            ]);

            // Should reject malicious files
            $this->assertStringContainsString('error', $response->getBody());
        }
    }
}
?>
```

Access Control Testing:

```
<?php
class AccessControlTest extends PHPUnit\Framework\TestCase {

    public function testUnauthorizedAccess() {
        $protectedPages = [
            '/dashboard.php',
            '/employees.php',
            '/attendance.php',
            '/leaves.php',
            '/reports.php'
        ];

        foreach ($protectedPages as $page) {
            // Access without login
            $response = $this->get($page);

            // Should redirect to login
            $this->assertEquals(302, $response->getStatusCode());
            $this->assertStringContainsString('login.php', $response->getHeaderLine('Locatio
        }
    }

    public function testRoleBasedAccessControl() {
        $rolePermissions = [
```

```
        'employee' => [
            '/employees.php' => ['read' => true, 'create' => false, 'update' => false, '
            '/leaves.php' => ['read' => true, 'create' => true, 'approve' => false],
            '/reports.php' => ['read' => true, 'create' => false]
        ],
        'manager' => [
            '/employees.php' => ['read' => true, 'create' => true, 'update' => true, 'de
            '/leaves.php' => ['read' => true, 'create' => true, 'approve' => true],
            '/reports.php' => ['read' => true, 'create' => true]
        ],
        'admin' => [
            '/employees.php' => ['read' => true, 'create' => true, 'update' => true, 'de
            '/leaves.php' => ['read' => true, 'create' => true, 'approve' => true, 'dele
            '/reports.php' => ['read' => true, 'create' => true, 'delete' => true]
        ]
    ];

    foreach ($rolePermissions as $role => $permissions) {
        $this->loginUser($role);

        foreach ($permissions as $page => $pagePermissions) {
            // Test read access
            $response = $this->get($page);
            if ($pagePermissions['read']) {
                $this->assertEquals(200, $response->getStatusCode());
            } else {
                $this->assertEquals(403, $response->getStatusCode());
            }

            // Test write access
            if ($pagePermissions['create']) {
                $response = $this->post($page, ['action' => 'create']);
                $this->assertNotEquals(403, $response->getStatusCode());
            } else {
                $response = $this->post($page, ['action' => 'create']);
                $this->assertEquals(403, $response->getStatusCode());
            }
        }
    }
}

public function testDataPrivacy() {
    // Test that users can only see their own data
    $this->loginUser('employee1');

    // Try to access another employee's data
```

```
        $response = $this->get('/employees.php?action=view&id=2'); // Another employee's ID

        $this->assertEquals(403, $response->getStatusCode());

        // Test that users can only see their own attendance
        $response = $this->get('/attendance.php?action=report&employee_id=2');

        $this->assertEquals(403, $response->getStatusCode());

        // Test that managers can see their department's data
        $this->loginUser('manager1');

        $response = $this->get('/attendance.php?action=report&department_id=1'); // Their de

        $this->assertEquals(200, $response->getStatusCode());

        $response = $this->get('/attendance.php?action=report&department_id=2'); // Another

        $this->assertEquals(403, $response->getStatusCode());
    }
}
?>
```

7.6 Usability Testing

Usability testing ensures the HR Management System is user-friendly, intuitive, and provides a positive user experience across different user roles and devices.

User Interface Testing:

```
<?php
class UsabilityTest extends PHPUnit\Framework\TestCase {

    public function testNavigationConsistency() {
        $pages = [
            '/dashboard.php',
            '/employees.php',
            '/attendance.php',
            '/leaves.php',
            '/reports.php'
        ];

        foreach ($pages as $page) {
            $response = $this->get($page);

            // Check for consistent navigation
            $this->assertStringContainsString('<nav', $response->getBody());
```

```
        $this->assertStringContainsString('Dashboard', $response->getBody());
        $this->assertStringContainsString('Employees', $response->getBody());
        $this->assertStringContainsString('Attendance', $response->getBody());
        $this->assertStringContainsString('Leaves', $response->getBody());
    }
}

public function testResponsiveDesign() {
    $testDevices = [
        ['width' => 1920, 'height' => 1080, 'user_agent' => 'Desktop'],
        ['width' => 768, 'height' => 1024, 'user_agent' => 'Tablet'],
        ['width' => 375, 'height' => 667, 'user_agent' => 'Mobile']
    ];

    foreach ($testDevices as $device) {
        $response = $this->get('/dashboard.php', [
            'headers' => ['User-Agent' => $device['user_agent']]
        ]);

        // Check responsive elements
        $this->assertStringContainsString('container-fluid', $response->getBody());
        $this->assertStringContainsString('table-responsive', $response->getBody());
        $this->assertStringContainsString('btn', $response->getBody());
    }
}

public function testFormUsability() {
    $forms = [
        '/employees.php?action=create',
        '/leaves.php?action=request',
        '/login.php'
    ];

    foreach ($forms as $form) {
        $response = $this->get($form);

        // Check for form usability elements
        $this->assertStringContainsString('<form', $response->getBody());
        $this->assertStringContainsString('label', $response->getBody());
        $this->assertStringContainsString('placeholder', $response->getBody());
        $this->assertStringContainsString('required', $response->getBody());

        // Check for error handling
        $this->assertStringContainsString('alert', $response->getBody());
        $this->assertStringContainsString('error', $response->getBody());
    }
```

```
    }

    public function testDataVisualization() {
        $pagesWithCharts = [
            '/dashboard.php',
            '/reports.php?action=employee_distribution',
            '/reports.php?action=attendance_trends'
        ];

        foreach ($pagesWithCharts as $page) {
            $response = $this->get($page);

            // Check for chart elements
            $this->assertStringContainsString('canvas', $response->getBody());
            $this->assertStringContainsString('Chart.js', $response->getBody());
            $this->assertStringContainsString('chart-', $response->getBody());
        }
    }

    public function testAccessibility() {
        $pages = ['/dashboard.php', '/employees.php', '/login.php'];

        foreach ($pages as $page) {
            $response = $this->get($page);

            // Check for accessibility features
            $this->assertStringContainsString('alt=', $response->getBody()); // Alt text fo
            $this->assertStringContainsString('aria-', $response->getBody()); // ARIA labels
            $this->assertStringContainsString('role=', $response->getBody()); // Semantic r
            $this->assertStringContainsString('tabindex', $response->getBody()); // Keyboar
        }
    }
}
?>
```

User Experience Testing:

```
<?php
class UserExperienceTest extends PHPUnit\Framework\TestCase {

    public function testErrorHandling() {
        // Test various error scenarios
        $errorScenarios = [
            'invalid_login' => '/login.php',
            'duplicate_email' => '/employees.php?action=create',
            'insufficient_leave_balance' => '/leaves.php?action=request',
            'invalid_date' => '/attendance.php?action=report'
```

```
        ];

        foreach ($errorScenarios as $scenario => $page) {
            $response = $this->simulateErrorScenario($scenario, $page);

            // Check for user-friendly error messages
            $this->assertStringContainsString('error', $response->getBody());
            $this->assertStringContainsString('alert', $response->getBody());
            $this->assertNotContainsString('SQL', $response->getBody()); // No technical er
            $this->assertNotContainsString('Exception', $response->getBody()); // No techni
        }
    }

    public function testSuccessFeedback() {
        $successScenarios = [
            'successful_login' => '/login.php',
            'employee_created' => '/employees.php?action=create',
            'leave_approved' => '/leaves.php?action=request'
        ];

        foreach ($successScenarios as $scenario => $page) {
            $response = $this->simulateSuccessScenario($scenario, $page);

            // Check for success feedback
            $this->assertStringContainsString('success', $response->getBody());
            $this->assertStringContainsString('alert-success', $response->getBody());
            $this->assertStringContainsString('Successfully', $response->getBody());
        }
    }

    public function testLoadingIndicators() {
        $pagesWithAjax = [
            '/employees.php',
            '/leaves.php',
            '/attendance.php'
        ];

        foreach ($pagesWithAjax as $page) {
            $response = $this->get($page);

            // Check for loading indicators
            $this->assertStringContainsString('loading', $response->getBody());
            $this->assertStringContainsString('spinner', $response->getBody());
            $this->assertStringContainsString('disabled', $response->getBody()); // Disable
        }
    }
```

```
    public function testHelpAndDocumentation() {
        $pages = ['/dashboard.php', '/employees.php', '/leaves.php'];

        foreach ($pages as $page) {
            $response = $this->get($page);

            // Check for help elements
            $this->assertStringContainsString('help', $response->getBody());
            $this->assertStringContainsString('tooltip', $response->getBody());
            $this->assertStringContainsString('question-circle', $response->getBody()); // I
        }
    }

    private function simulateErrorScenario($scenario, $page) {
        switch ($scenario) {
            case 'invalid_login':
                return $this->post($page, ['username' => 'invalid', 'password' => 'wrong']);
            case 'duplicate_email':
                return $this->post($page, ['email' => 'existing@example.com']);
            case 'insufficient_leave_balance':
                return $this->post($page, ['leave_type_id' => 1, 'total_days' => 30]);
            case 'invalid_date':
                return $this->get($page . '?date=invalid-date');
        }
    }

    private function simulateSuccessScenario($scenario, $page) {
        switch ($scenario) {
            case 'successful_login':
                return $this->post($page, ['username' => 'admin', 'password' => 'admin123']);
            case 'employee_created':
                return $this->post($page, $this->getValidEmployeeData());
            case 'leave_approved':
                return $this->post($page . '?action=approve&id=1', []);
        }
    }
}
?>
```

7.7 Compatibility Testing

Compatibility testing ensures the HR Management System works correctly across different browsers, devices, and operating systems.

Browser Compatibility Testing:

```php
<?php
class BrowserCompatibilityTest extends PHPUnit\Framework\TestCase {

    private $browsers = [
        'Chrome' => ['user_agent' => 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/
        'Firefox' => ['user_agent' => 'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:91.0) Ge
        'Safari' => ['user_agent' => 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleW
        'Edge' => ['user_agent' => 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/53
    ];

    public function testCoreFunctionalityAcrossBrowsers() {
        $corePages = [
            '/dashboard.php',
            '/employees.php',
            '/attendance.php',
            '/leaves.php'
        ];

        foreach ($this->browsers as $browser => $config) {
            foreach ($corePages as $page) {
                $response = $this->get($page, [
                    'headers' => ['User-Agent' => $config['user_agent']]
                ]);

                // Check basic functionality
                $this->assertEquals(200, $response->getStatusCode());
                $this->assertStringContainsString('<!DOCTYPE html>', $response->getBody());
                $this->assertStringContainsString('</html>', $response->getBody());

                // Check JavaScript functionality
                $this->assertStringContainsString('script', $response->getBody());
                $this->assertStringContainsString('Chart.js', $response->getBody());

                // Check CSS rendering
                $this->assertStringContainsString('bootstrap', $response->getBody());
                $this->assertStringContainsString('style', $response->getBody());
            }
        }
    }

    public function testFormSubmissionAcrossBrowsers() {
        foreach ($this->browsers as $browser => $config) {
            // Test employee creation form
            $response = $this->get('/employees.php?action=create', [
                'headers' => ['User-Agent' => $config['user_agent']]
            ]);
```

```
            $this->assertStringContainsString('<form', $response->getBody());
            $this->assertStringContainsString('input', $response->getBody());
            $this->assertStringContainsString('button', $response->getBody());

            // Test form submission
            $submitResponse = $this->post('/employees.php', [
                'first_name' => 'Test',
                'last_name' => 'User',
                'email' => 'test@example.com',
                'department_id' => 1,
                'position' => 'Test',
                'salary' => 50000,
                'hire_date' => '2024-01-15'
            ], [
                'headers' => ['User-Agent' => $config['user_agent']]
            ]);

            $this->assertNotEquals(500, $submitResponse->getStatusCode());
        }
    }

    public function testResponsiveDesignAcrossBrowsers() {
        $viewports = [
            ['width' => 1920, 'height' => 1080], // Desktop
            ['width' => 768, 'height' => 1024],  // Tablet
            ['width' => 375, 'height' => 667]   // Mobile
        ];

        foreach ($this->browsers as $browser => $config) {
            foreach ($viewports as $viewport) {
                $response = $this->get('/dashboard.php', [
                    'headers' => [
                        'User-Agent' => $config['user_agent']
                    ]
                ]);

                // Check responsive elements
                $this->assertStringContainsString('container-fluid', $response->getBody());
                $this->assertStringContainsString('table-responsive', $response->getBody());
                $this->assertStringContainsString('btn', $response->getBody());
            }
        }
    }
}
?>
```

Device Compatibility Testing:

```php
<?php
class DeviceCompatibilityTest extends PHPUnit\Framework\TestCase {

    public function testMobileFunctionality() {
        $mobileDevices = [
            ['user_agent' => 'Mozilla/5.0 (iPhone; CPU iPhone OS 14_7_1 like Mac OS X) Apple
            ['user_agent' => 'Mozilla/5.0 (Linux; Android 11; SM-G991B) AppleWebKit/537.36']
            ['user_agent' => 'Mozilla/5.0 (iPad; CPU OS 14_7_1 like Mac OS X) AppleWebKit/60
        ];

        foreach ($mobileDevices as $device) {
            // Test mobile navigation
            $response = $this->get('/dashboard.php', [
                'headers' => ['User-Agent' => $device['user_agent']]
            ]);

            $this->assertStringContainsString('mobile-menu-toggle', $response->getBody());
            $this->assertStringContainsString('sidebar', $response->getBody());

            // Test touch interactions
            $this->assertStringContainsString('ontouch', $response->getBody());
            $this->assertStringContainsString('touch', $response->getBody());

            // Test mobile forms
            $this->assertStringContainsString('form-control', $response->getBody());
            $this->assertStringContainsString('btn', $response->getBody());
        }
    }

    public function testTabletFunctionality() {
        $tabletDevices = [
            ['user_agent' => 'Mozilla/5.0 (iPad; CPU OS 14_7_1 like Mac OS X) AppleWebKit/60
            ['user_agent' => 'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, lik
        ];

        foreach ($tabletDevices as $device) {
            $response = $this->get('/dashboard.php', [
                'headers' => ['User-Agent' => $device['user_agent']]
            ]);

            // Test tablet layout
            $this->assertStringContainsString('col-md-', $response->getBody());
            $this->assertStringContainsString('container', $response->getBody());
```

```
            // Test tablet-specific features
            $this->assertStringContainsString('table-responsive', $response->getBody());
        }
    }

    public function testDesktopFunctionality() {
        $desktopDevices = [
            ['user_agent' => 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36']
            ['user_agent' => 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/53
        ];

        foreach ($desktopDevices as $device) {
            $response = $this->get('/dashboard.php', [
                'headers' => ['User-Agent' => $device['user_agent']]
            ]);

            // Test desktop layout
            $this->assertStringContainsString('col-md-', $response->getBody());
            $this->assertStringContainsString('sidebar', $response->getBody());

            // Test desktop-specific features
            $this->assertStringContainsString('hover', $response->getBody());
            $this->assertStringContainsString('dropdown', $response->getBody());
        }
    }
}
?>
```

7.8 Test Documentation and Reporting

Comprehensive test documentation ensures that all testing activities are properly recorded, tracked, and reported for future reference and continuous improvement.

Test Case Documentation:

```
<?php
/**
 * Test Case: Employee Registration
 *
 * Description: Verify that employees can be successfully registered with valid data
 * and that appropriate validation occurs with invalid data.
 *
 * Test ID: TC-EMP-001
 * Module: Employee Management
 * Priority: High
 *
```

```
 * Preconditions:
 * - User is logged in as HR Manager
 * - Database is in clean state
 * - All required modules are active
 *
 * Test Steps:
 * 1. Navigate to employee creation page
 * 2. Fill form with valid data
 * 3. Submit form
 * 4. Verify success message
 * 5. Verify employee in database
 * 6. Verify email notification sent
 *
 * Expected Results:
 * - Employee created successfully
 * - Success message displayed
 * - Employee record in database
 * - Email notification sent to employee
 *
 * Actual Results:
 * - [To be filled during testing]
 *
 * Status: [Pass/Fail]
 *
 * Test Data:
 * - Valid: John, Doe, john.doe@example.com, IT, Developer, 50000, 2024-01-15
 * - Invalid: Invalid email, negative salary, future date
 */
class EmployeeRegistrationTest {

    public function testValidRegistration() {
        // Test implementation
    }

    public function testInvalidEmailRegistration() {
        // Test implementation
    }

    public function testNegativeSalaryRegistration() {
        // Test implementation
    }

    public function testFutureDateRegistration() {
        // Test implementation
    }
}
```

?>

Test Execution Reports: “‘php $this->generateSummary($testResults), ‘test_cases’ => $testResults, ‘coverage’ => $this->calculateCoverage($testResults), ‘defects’ => $this->categorizeDefects($testResults), ‘recommendations’ => $this->generateRecommendations($testResults)];

```
    return $report;
}

private function generateSummary($testResults) {
    $total = count($testResults);
    $passed = 0;
    $failed = 0;
    $blocked = 0;

    foreach ($testResults as $result) {
        switch ($result['status']) {
            case 'pass':
                $passed++;
                break;
            case 'fail':
                $failed++;
                break;
            case 'blocked':
                $blocked++;
                break;
        }
    }

    return [
        'total_tests' => $total,
        'passed' => $passed,
        'failed' => $failed,
        'blocked' => $blocked,
        'pass_rate' => $total > 0 ? round(($passed / $total) * 100, 2) : 0,
        'execution_time' => $this->calculateTotalExecutionTime($testResults)
    ];
}

private function calculateCoverage($testResults) {
    $modules = [];
    $totalCoverage = 0;
    $coveredModules = 0;

    foreach ($testResults as $result) {
        $module = $result['module'];
```

```
        if (!isset($modules[$module])) {
            $modules[$module] = ['total' => 0, 'covered' => 0];
        }

        $modules[$module]['total']++;
        if ($result['status'] === 'pass') {
            $modules[$module]['covered']++;
        }
    }

    foreach ($modules as $module => $stats) {
        $totalCoverage += $stats['total'];
        if ($stats['covered'] > 0) {
            $coveredModules++;
        }
    }

    return [
        'total_modules' => count($modules),
        'covered_modules' => $coveredModules,
        'module_coverage' => $totalCoverage > 0 ? round(($coveredModules / count($modules))
        'functional_coverage' => $totalCoverage > 0 ? round(($totalCoverage - array_sum(arra
    ];
}

private function categorizeDefects($testResults) {
    $defects = [
        'critical' => 0,
        'high' => 0,
        'medium' => 0,
        'low' => 0
    ];

    foreach ($testResults as $result) {
        if ($result['status'] === 'fail') {
            $severity = $this->determineDefectSeverity($result);
            $defects[$severity]++;
        }
    }

    return $defects;
}

private function generateRecommendations($testResults) {
    $recommendations = [];
```

```
    // Analyze failure patterns
    $failurePatterns = $this->analyzeFailurePatterns($testResults);

    foreach ($failurePatterns as $pattern => $count) {
        if ($count > 5) { // More than 5 failures of same type
            $recommendations[] = $this->getRecommendationForPattern($pattern);
        }
    }

    return $recommendations;
}

public function exportToHTML($report) {
    $html = '<!DOCTYPE html>
```

Test Report - HR Management System

```
    <h1>Test Report - HR Management System</h1>
    <p>Generated on: ' . date('Y-m-d H:i:s') . '</p>
</div>

<div class="summary">
    <div class="metric">
        <div class="metric-value pass">' . $report['summary']['passed'] . '</div>
        <div class="metric-label">Passed</div>
    </div>
    <div class="metric">
        <div class="metric-value fail">' . $report['summary']['failed'] . '</div>
        <div class="metric-label">Failed</div>
    </div>
    <div class="metric">
        <div class="metric-value blocked">' . $report['summary']['blocked'] . '</div>
        <div class="metric-label">Blocked</div>
    </div>
    <div class="metric">
        <div class="metric-value">' . $report['summary']['pass_rate'] . '%</div>
        <div class="metric-label">Pass Rate</div>
    </div>
</div>

<h2>Test Results</h2>
<table>
    <tr>
        <th>Test ID</th>
        <th>Module</th>
        <th>Test Case</th>
```

```
        <th>Status</th>
        <th>Execution Time</th>
        <th>Comments</th>
    </tr>';

    foreach ($report['test_cases'] as $test) {
        $statusClass = 'status-' . $test['status'];
        $html .= '<tr>
            <td>' . $test['id'] . '</td>
            <td>' . $test['module'] . '</td>
            <td>' . $test['description'] . '</td>
            <td class="' . $statusClass . '">' . ucfirst($test['status']) . '</td>
            <td>' . $test['execution_time'] . 'ms</td>
            <td>' . ($test['comments'] ?? '') . '</td>
        </tr>';
    }

    $html .= '</table>
';

    return $html;
```

CONCLUSION

8.1 Project Summary

The HR Management System represents a comprehensive solution for modern human resource management challenges. This project successfully demonstrates the integration of various technologies and methodologies to create a robust, scalable, and user-friendly system that addresses the complex needs of HR departments in today's dynamic business environment.

Key Achievements:

Technical Excellence: - Successfully implemented a three-tier architecture following industry best practices - Developed modular system components that can be independently maintained and scaled - Integrated modern web technologies including PHP, MySQL, HTML5, CSS3, JavaScript, and Chart.js - Implemented comprehensive security measures to protect sensitive employee data - Created responsive design that works across multiple devices and browsers

Functional Completeness: - Delivered all core HR management modules: authentication, employee management, attendance tracking, leave management, performance appraisal, and reporting - Implemented role-based access control ensuring data privacy and security - Created automated workflows for leave

requests and approvals - Developed comprehensive reporting and analytics capabilities - Integrated email notifications for important system events

Quality Assurance: - Conducted extensive testing covering functional, performance, security, usability, and compatibility aspects - Achieved high test coverage across all system modules - Implemented automated testing frameworks to ensure continuous quality - Documented all test cases and results for future reference

8.2 Technical Contributions

This project contributes to the field of web-based HR management systems through several technical innovations and implementations:

Architecture and Design Patterns: - **MVC Pattern Implementation:** Separation of concerns through Model-View-Controller architecture - **Repository Pattern:** Abstracted data access layer for improved maintainability - **Singleton Pattern:** Database connection management for resource efficiency - **Factory Pattern:** Dynamic object creation for flexible system behavior

Security Implementation: - **Multi-layered Authentication:** Username/password with session management and remember me functionality - **Role-Based Access Control:** Granular permissions for different user roles - **Input Validation and Sanitization:** Protection against SQL injection and XSS attacks - **CSRF Protection:** Prevents cross-site request forgery attacks - **Secure File Upload:** Validates file types and sizes to prevent malicious uploads

Performance Optimization: - **Database Indexing:** Optimized queries for improved response times - **Caching Strategy:** Implemented caching for frequently accessed data - **Lazy Loading:** Optimized page load times through progressive content loading - **Connection Pooling:** Efficient database connection management

8.3 Business Value and Impact

The HR Management System delivers significant business value through automation, efficiency improvements, and data-driven decision making capabilities.

Operational Efficiency: - **Time Savings:** Automated processes reduce manual HR tasks by an estimated 60-70% - **Error Reduction:** Digital workflows minimize human errors in data entry and calculations - **Process Standardization:** Consistent procedures across all HR operations - **Real-time Information:** Immediate access to current HR data and metrics

Cost Benefits: - **Reduced Administrative Overhead:** Fewer manual processes mean lower operational costs - **Paperless Operations:** Digital records reduce printing and storage costs - **Improved Resource Allocation:** Better

visibility into workforce distribution and utilization - **Compliance Management:** Automated tracking reduces risk of regulatory violations

Strategic Advantages: - **Data-Driven Decisions:** Analytics and reporting support strategic HR planning - **Employee Satisfaction:** Self-service capabilities improve employee experience - **Scalability:** System grows with organizational needs - **Competitive Edge:** Modern HR tools attract and retain talent

8.4 Learning Outcomes and Experience

This project provided valuable learning experiences across multiple domains of software development and project management.

Technical Skills Development: - **Full-Stack Development:** Comprehensive understanding of frontend and backend technologies - **Database Design:** Advanced knowledge of relational database design and optimization - **Security Implementation:** Practical experience with web security best practices - **Testing Methodologies:** Expertise in various testing approaches and frameworks - **Performance Tuning:** Skills in optimizing application performance

Project Management Experience: - **Requirements Analysis:** Practice in gathering and analyzing business requirements - **System Design:** Experience in translating requirements into technical solutions - **Quality Assurance:** Understanding of testing strategies and quality metrics - **Documentation:** Skills in creating comprehensive technical documentation - **Version Control:** Proficiency in Git-based development workflows

Problem-Solving Capabilities: - **System Integration:** Experience in connecting multiple system components - **User Experience Design:** Understanding of creating intuitive user interfaces - **Scalability Planning:** Knowledge of designing systems for future growth - **Troubleshooting:** Advanced debugging and problem resolution skills

8.5 Challenges and Solutions

Throughout the development process, several challenges were encountered and successfully resolved:

Technical Challenges:

Database Performance: - **Challenge:** Complex queries with multiple joins causing slow response times - **Solution:** Implemented strategic indexing and query optimization - **Result:** 70% improvement in query performance

Security Implementation: - **Challenge:** Balancing security with user experience - **Solution:** Implemented multi-layered security with minimal user friction - **Result:** Robust security without sacrificing usability

Cross-Browser Compatibility: - **Challenge:** Ensuring consistent behavior across different browsers - **Solution:** Comprehensive testing and progressive enhancement approach - **Result:** Consistent user experience across all major browsers

Project Management Challenges:

Scope Management: - **Challenge:** Managing feature creep while meeting deadlines - **Solution:** Agile development methodology with clear prioritization - **Result:** On-time delivery with all core features implemented

Quality Assurance: - **Challenge:** Ensuring comprehensive testing within time constraints - **Solution:** Automated testing frameworks and parallel test execution - **Result:** High test coverage without project delays

8.6 Future Enhancements and Recommendations

The HR Management System provides a solid foundation for future enhancements and improvements. Several areas offer opportunities for continued development:

Immediate Enhancements:

Mobile Application: - Develop native mobile applications for iOS and Android - Implement push notifications for important events - Add offline capabilities for field operations - Include biometric authentication options

Advanced Analytics: - Implement machine learning for predictive analytics - Add workforce trend analysis and forecasting - Create customizable dashboards with drag-and-drop functionality - Integrate with business intelligence tools

Integration Capabilities: - Connect with payroll systems for seamless data flow - Integrate with recruitment platforms for candidate management - Add API support for third-party application integration - Implement single sign-on (SSO) capabilities

Long-term Strategic Enhancements:

Artificial Intelligence Integration: - AI-powered employee performance analysis - Automated leave approval based on historical patterns - Intelligent candidate matching for recruitment - Predictive workforce planning tools

Cloud Migration: - Cloud-native architecture for improved scalability - Multi-tenant support for SaaS deployment - Enhanced disaster recovery and backup capabilities - Global deployment and multi-region support

Blockchain Integration: - Secure credential verification - Immutable audit trails for compliance - Smart contracts for automated HR processes - Decentralized identity management

8.7 Lessons Learned

This project provided valuable insights that can be applied to future software development endeavors:

Technical Lessons: - **Early Architecture Planning:** Investing time in initial architecture design pays dividends throughout development - **Security First Approach:** Integrating security from the beginning is more effective than retrofitting - **Performance Monitoring:** Continuous performance monitoring prevents degradation over time - **Modular Design:** Modular architecture facilitates testing, maintenance, and future enhancements

Process Lessons: - **Iterative Development:** Regular iterations with user feedback lead to better outcomes - **Comprehensive Testing:** Investing in testing reduces maintenance costs and improves quality - **Documentation Importance:** Good documentation accelerates development and knowledge transfer - **User-Centered Design:** Focusing on user experience drives adoption and satisfaction

Business Lessons: - **Stakeholder Engagement:** Regular communication with stakeholders ensures alignment - **Change Management:** Proper change management is crucial for system adoption - **Training Investment:** User training maximizes system value and effectiveness - **Continuous Improvement:** Systems should evolve with changing business needs

8.8 Final Assessment

The HR Management System successfully meets all project objectives and delivers significant value to its users. The system demonstrates:

Technical Excellence: - Robust architecture following industry best practices - Comprehensive security measures protecting sensitive data - Optimized performance ensuring responsive user experience - Scalable design supporting future growth

Functional Completeness: - All required HR management modules implemented - Automated workflows reducing manual intervention - Comprehensive reporting and analytics capabilities - User-friendly interface promoting adoption

Quality Assurance: - Extensive testing ensuring system reliability - High code quality following established standards - Comprehensive documentation supporting maintenance - Performance metrics meeting or exceeding requirements

Business Value: - Significant operational efficiency improvements - Cost reduction through process automation - Enhanced decision-making through data analytics - Improved employee satisfaction and engagement

8.9 Acknowledgments

This project represents the culmination of extensive research, development, and testing efforts. The successful completion of the HR Management System was made possible through:

Technical Resources: - Open-source technologies providing robust development foundations - Development frameworks accelerating implementation - Testing tools ensuring quality and reliability - Documentation resources supporting best practices

Knowledge Sources: - Academic research in software engineering principles - Industry best practices in HR management systems - Community contributions and open-source collaboration - Professional experience in web application development

Development Environment: - Modern development tools and IDEs - Version control systems enabling collaborative development - Testing frameworks supporting comprehensive validation - Deployment tools facilitating system delivery

8.10 Conclusion

The HR Management System stands as a testament to the successful application of modern software development principles to solve real-world business challenges. Through careful planning, systematic development, and comprehensive testing, this project delivers a solution that not only meets current requirements but also provides a foundation for future enhancements.

The system's modular architecture, robust security measures, and user-friendly interface demonstrate the successful integration of technical excellence with practical business utility. The comprehensive testing approach ensures reliability and performance under various conditions, while the detailed documentation supports future maintenance and development.

This project contributes to the field of HR management systems by providing a reference implementation that balances functionality, security, performance, and usability. The lessons learned and challenges overcome during development provide valuable insights for future software development endeavors.

The HR Management System is ready for deployment and will deliver significant value to organizations seeking to modernize their HR operations, improve efficiency, and leverage data for strategic decision-making. As technology continues to evolve, the system's architecture and design principles provide a solid foundation for future enhancements and adaptations to emerging business needs.